

Unit 11

PROGRAMMING

96 items · 540 marks

Syllabus sections in this unit

- 11.1 Programming Basics
- 11.2 Constructs
- 11.3 Structured Programming

Unit 11: Programming

1. M/J 26/P21/Q1/a,b

1 Study this pseudocode:

```
FOR i ← 3 TO 12 STEP 3
    OUTPUT i, i * i
NEXT i
```

(a) Identify the type of construct used in this pseudocode.

[1]

(b) State how many iterations are performed by this pseudocode.

[1]

2. M/J 26/P21/Q5/a,b

5 Electric light bulbs are advertised online with the price showing:

\$1.75 each or 20 bulbs for \$30.00

A pseudocode function `BulbCost()`

- takes the quantity of bulbs purchased as its single parameter
- returns the total cost in dollars.

Pseudocode for the main program:

- inputs the number of bulbs purchased
- returns the total cost from the function `BulbCost()`
- outputs the total cost.

(a) Write pseudocode for the function `BulbCost()`

[4]

(b) The main program accepts a user input for the quantity of bulbs purchased and calls function `BulbCost()`

Complete the pseudocode for the main program.

```
// Main program
DECLARE NumberPurchased : INTEGER
DECLARE TotalCost : REAL

OUTPUT "How many bulbs? "
INPUT NumberPurchased

TotalCost ←

OUTPUT '$' , TotalCost
// end main program
```

[1]

3. M/J 26/P21/Q6/a,b

- 6 Fence parts are manufactured by a number of similar machines. The length of the part must fall within a certain range. Parts **not** within the length range are rejected.

The data for each rejected part forms a data packet made up of three data items, each separated by the '/' character delimiter. The format of a data packet is:

<length>/<machine ID>/<description of the fault>

- The length is 3 digits (in millimetres).
- The machine ID is a single character ('A', 'B', or 'C').
- The description of the fault is a variable length string.

A pseudocode program will process the data for several data packets.

The program contains the statement:

DataPacket ← "563/B/the fence length is too long"

- (a) Complete the **two** pseudocode expressions which extract the appropriate data item.

PartLength ←

MachineID ←

[2]

- (b) The program will create a module `RejectDescription()` to return the fault description.

Write the pseudocode for the module `RejectDescription()`

[4]

4. M/J 26/P22/Q1/a(ii,iii)

- 1 A program is being developed to control a simple alarm system to be used in a house. The alarm has a control panel with a keypad and a screen to display text.

The keypad consists of 36 character keys:

- 26 upper-case letter keys, allowing A to Z to be entered
- 10 digit keys, allowing 0 to 9 to be entered.

- (a) When the alarm system is initially installed, a password must be created. When the password is initially entered, it is checked to ensure that it follows a set of validation rules. It is then permanently stored in the alarm system.

For a password to be valid it must follow these validation rules:

Rule 1: contain between six and eight characters inclusive.

Rule 2: contain at least one letter and one digit.

Rule 3: each character can only occur once.

The global variable `Pass` is used to store the password entered.

- (ii) Complete the pseudocode to check that a password contains between **six** and **eight** characters inclusive (rule 1).

IF (Pass) < 6 OR (Pass) > THEN

OUTPUT "Error"

OUTPUT "The password has to contain between 6 and 8 characters."

ENDIF

[2]

- (iii) Complete the pseudocode to check that a password entered contains at least one letter and one digit character (rule 2).

```
DECLARE Index : INTEGER
DECLARE Character : CHAR
DECLARE LetterFlag, DigitFlag : .....
DigitFlag ← FALSE
LetterFlag ← .....
FOR Index ← 1 TO ..... (Pass)
    Character ← ..... (Pass, ..... , ..... )
    IF ..... (Character) = TRUE THEN
        DigitFlag ← .....
    ELSE
        LetterFlag ← .....
    ENDIF
NEXT Index
IF ..... (LetterFlag AND DigitFlag) THEN
    OUTPUT "Error"
    OUTPUT "Password needs at least one letter and one digit."
ENDIF
```

[7]

5. M/J 26/P22/Q5/c

- 5 A programmer has defined a global 1D array `Numbers` of type `INTEGER`, which can store 1000 integers. The lower bound of the array `Numbers` is set to 1. Each element of the array has been initialised with a random integer in the range 200 to 99999.

The programmer has written a bubble sort algorithm to sort the array `Numbers` into ascending order.

Study algorithm A.

Assume all the variables have been correctly declared.

algorithm A:

```
FOR LoopCounter ← 1 TO 999                                //Outer Loop
  FOR Index ← 1 TO 999                                    //Inner Loop
    IF Numbers[Index] > Numbers[Index + 1] THEN
      //Start of block to swap neighbouring elements in the array
      Temp ← Numbers[Index]
      Numbers[Index] ← Numbers[Index + 1]
      Numbers[Index + 1] ← Temp
      //End of block to swap neighbouring elements in the array
    ENDIF
  NEXT Index
NEXT LoopCounter
```

- (c) The programmer has defined a new procedure:

```
PROCEDURE Swap (Num1 : INTEGER, Num2 : INTEGER)
  DECLARE Temp : INTEGER
  Temp ← Num1
  Num1 ← Num2
  Num2 ← Temp
ENDPROCEDURE
```

The programmer replaces the block of pseudocode in the algorithm that swaps the values in two neighbouring elements in the array with a call to this procedure.

The block of code:

```
Temp ← Numbers[Index]
Numbers[Index] ← Numbers[Index + 1]
Numbers[Index + 1] ← Temp
```

Is replaced by:

```
CALL Swap (Numbers[Index], Numbers[Index + 1])
```

The `Swap ()` procedure will **not** work as expected when used in the bubble sort algorithm.

- (i) Explain why the procedure `Swap ()` will **not** work as expected.

[2]

- (ii) Write pseudocode to amend the version of the `Swap ()` procedure given so that it will work as expected.

The amended version of the `Swap ()` procedure must **not** use global variables.

[2]

6. M/J 26/P22/Q6/a

6 A program is being developed to encrypt a message.

The following terms are used in the encryption technique to be used:

term	definition
plain text	The original message before it is encrypted.
cipher text	The encrypted version of the plain text.

The encryption method to be used is based on the algorithm for the Caesar Box Cipher. All plain text messages to be encrypted must be 25 characters in length or less. If a message is less than 25 characters in length, then spaces are added to the end to make the padded plain text the required length.

A square grid of five columns and five rows is used. The padded plain text message is put into the grid row by row. The cipher text is obtained by reading the grid column by column.

For example:

plain text: This is a message

The number of characters in the original plain text, including spaces, is 17.

plain text with added spaces at the end: This△is△a△message△△△△△△△△

The character △ is used to indicate a space in the padded plain text.

The grid filled, row by row, with the padded plain text.

T	h	i	s	△
i	s	△	a	△
m	e	s	s	a
g	e	△	△	△
△	△	△	△	△

The cipher text is obtained by reading the grid column by column.

cipher text: Timg△hsee△i△s△△sas△△△△a△△

The programmer has defined the first program module as:

module	description
AddPadding()	• user prompt for and input of plain text • check plain text contains between 1 and 25 characters inclusive • re-input of plain text until it contains between 1 and 25 characters • if required, additional space characters are added to the end of the plain text to ensure it contains exactly 25 characters • return of amended plain text

(a) Write pseudocode for module AddPadding()

7. M/J 26/P23/Q1/a,b

1 Study the pseudocode.

```
i ← 1
REPEAT
    OUTPUT i*i
    i ← i + 1
UNTIL i = 6
```

(a) Identify the type of programming construct used in the pseudocode.

[1]

(b) State how many iterations are carried out by the pseudocode.

[1]

8. M/J 26/P23/Q5/a

5 Data is stored about people who have attended a job interview. Each interview is held at one of three office locations.

Each person's data is stored as a data packet made up of four data items:

<initial><shortened surname><office code><summary of the interview>

- The initial is a single character.
- The shortened surname is the first five characters of the surname.
 - You can assume all surnames are at least five characters.
- The office code is a single character 'A', 'B' or 'C'.
- The interview summary text is of variable length.

Example:

Peter Watkins was interviewed at office 'B'. After the interview it was decided to invite him for a second interview.

His data packet is stored by variable `Applicant`:

```
Applicant ← "PWatkiBSecond interview"
```

(a) A pseudocode function `InterviewSummary()` will:

- take the data packet as a single parameter
- return the interview summary text.

Write the pseudocode for the function `InterviewSummary()`

[4]

9. O/N 25/P21/Q1/a

Uses the insert (Appendix)

1 A program is being developed to meet a particular customer requirement.**(a)** The program contains these variables:

Variable	Data type
MyChar	CHAR
MyString	STRING
MyInt	INTEGER
MyDOB	DATE

Complete the table by filling in the gaps using functions and/or operators from the **insert**.

Each expression must be valid.

Expression
MyString ← 'X' MyString
MyChar ← ("ABCD", ,)
MyString ← (..... (MyDOB))
MyInt ← (..... (MyString) / 2)

[4]

10. O/N 25/P21/Q5/b

- 5** A program is being designed in pseudocode.

The program contains the following declaration for the global array `MyData`:

```
DECLARE MyData : ARRAY[1:10000] OF STRING
```

A function `FindFirst()` is written to search the array for a given string and to return the index of the first element where that string is found, or to return `-1` if the string is **not** found.

The function is written in pseudocode as shown:

```
FUNCTION FindFirst(SearchString : STRING) RETURNS INTEGER
  DECLARE Index, FoundAt : INTEGER
  FoundAt ← -1
  FOR Index ← 1 TO 10000
    IF MyData[Index] = SearchString THEN // outer conditional clause
      IF FoundAt = -1 THEN               // inner conditional clause
        FoundAt ← Index
      ENDIF
    ENDIF
  NEXT Index
  RETURN FoundAt
ENDFUNCTION
```

- (b)** The pseudocode does **not** use the most appropriate loop construct.

- (i)** Explain why this is **not** the most appropriate loop construct.

[1]

- (ii)** Suggest and justify a more appropriate loop construct that could be used.

Construct

Justification

[2]

11. O/N 25/P21/Q6/b

- 6 Students are learning about a simple check digit method for data validation. In this method, a single check digit is appended to the end of an original number to give a new number.

The students are studying a method which:

- calculates the sum of all the digits in the original number
- uses integer division to calculate the remainder when the sum is divided by 10
- uses the remainder as the check digit
- appends the check digit to the original number, creating the new number.

For example:

original number	4162
sum of all digits	$4 + 1 + 6 + 2 = 13$
remainder when the sum is divided by 10 using integer division	3
new number	41623

The method described can be used to detect single-digit errors. For example, if the new number is incorrectly input as 41633, then the check digit does not match and the input will be rejected.

- (b) A module `Generate()` will take an integer value representing an original number and return an integer value representing a new number which includes the check digit.

The original number is always at least three digits in length.

Write pseudocode for the module `Generate()`

Assume that the parameter is valid.

[7]

12. O/N 25/P22/Q1/c

- (b) Complete the table by giving the appropriate data type.

Variable	Example data value	Data type
Result	5.42	
MonthLetter	"JFMAMJJASOND"	
Birthday	15/11/2009	

[3]

- (c) Evaluate each expression in the table by using the data values shown in (b).

Write 'ERROR' if the expression contains an error.

Expression	Evaluates to
<code>INT(Result) + 1 > 6</code>	
<code>NUM_TO_STR(LENGTH(MonthLetter))</code>	
<code>NUM_TO_STR(Result + "3.2")</code>	
<code>MID(MonthLetter, MONTH(Birthday) - 2, 1)</code>	

[4]

13. O/N 25/P22/Q6

6 A string function `Compare()` will compare two strings.

The function will:

- take **four** parameters:
 - two strings, `String1` and `String2`
 - a character, `Position`, to indicate whether `String1` will be compared with the start ('s'), or the end ('e') of `String2`
 - a Boolean, `CaseMatters`, to indicate whether an upper case character and lower case character (for example, 'A' and 'a') are regarded as different (TRUE), or as the same (FALSE)
- return FALSE if `String2` has fewer characters than `String1`
- return TRUE if the comparison is true, otherwise return FALSE

For example:

Parameter				Return value
String1	String2	CaseMatters	Position	
"Cat"	"Catalogue"	TRUE	's'	TRUE
"CAT"	"Catalogue"	TRUE	's'	FALSE
"CAT"	"Catalogue"	FALSE	's'	TRUE
"Cat"	"Catalogue"	TRUE	'e'	FALSE
"GUE"	"Catalogue"	FALSE	'e'	TRUE
"Catalogue"	"Cat"	TRUE	's'	FALSE

Write pseudocode for the function `Compare()`

[7]

14. O/N 25/P23/Q1/a,b,c

1 A user requires a program that implements complex data encryption to a recognised international standard. A programmer has started the design of the program.

- (a) The programmer decides to use a library routine to provide part of the solution. One reason for this is that library routines may perform functions that the programmer is unable to program themselves.

State **three** other benefits of using library routines in the development of the program.

- 1
- 2
- 3

[3]

- (b) The programmer has identified a different part of the algorithm that would be appropriate to implement as a subroutine (a procedure or a function); no suitable library routine exists for this part.

(i) State **two** reasons why the programmer may decide to use a subroutine.

- 1
- 2

[2]

(ii) A function header in pseudocode is defined as:

```
FUNCTION Pass2 (Count : INTEGER) RETURNS BOOLEAN
```

Complete the table by describing the terms used in the function header.

The first row has been completed.

Term	Description
Pass2	the name of the function
Count	
BOOLEAN	

[2]

(c) Variables in the program have example values:

Variable	Example value
DoB	23/6/2011
Multiplier	2.5
AddressLine[1]	"35 Lincoln Avenue"

Complete the table by evaluating each expression using the example values:

Expression	Evaluates to
LENGTH (NUM_TO_STR (Multiplier))	
MONTH (DoB) > 4	
15 + STR_TO_NUM (MID (AddressLine[1], 2, 1))	

[3]

15. O/N 25/P23/Q5/b

- 5 A global 1D array `Num` of integers contains four elements. A program assigns values to these elements as shown:

```
Num[1] ← 1
```

```
Num[2] ← 2
```

```
Num[3] ← 5
```

```
Num[4] ← 3
```

A procedure `Process()` manipulates the values in the array.

The procedure is written in pseudocode:

```

PROCEDURE Process(Start : INTEGER)
  DECLARE CaseVar, Index, Count : INTEGER

  Index ← Start
  Count ← 0

  WHILE Count ≤ 20
    CaseVar ← Num[Index]
    CASE OF CaseVar
      1 : Num[Index] ← Num[Index] + Index //clause one
        Index ← Index + 1
        Count ← Count + 1

      2 : Num[Index] ← Num[Index] + Index //clause two
        Index ← Index + 2
        Count ← Count + 2

      3 : Num[Index] ← Num[Index] * 2      //clause three
        Index ← Index + 3
        Count ← Count - 1

      4 : Count ← Count + 4                //clause four
    OTHERWISE : Count ← 20
    ENDCASE

    Index ← (Index MOD 4) + 1

  ENDWHILE
ENDPROCEDURE

```

(b) As a reminder, the CASE structure in the pseudocode is:

```

CASE OF CaseVar
  1 : Num[Index] ← Num[Index] + Index //clause one
    Index ← Index + 1
    Count ← Count + 1

  2 : Num[Index] ← Num[Index] + Index //clause two
    Index ← Index + 2
    Count ← Count + 2

  3 : Num[Index] ← Num[Index] * 2      //clause three
    Index ← Index + 3
    Count ← Count - 1

  4 : Count ← Count + 4                //clause four
  OTHERWISE : Count ← 20
ENDCASE

```

The CASE structure could be optimised by combining two existing clauses.

(i) Identify the CaseVar values for these **two** existing clauses.

[1]

(ii) Write pseudocode for a single clause to replace the **two** clauses identified in **(b) (i)**.

[2]

16. O/N 25/P23/Q6/b

- 6** Students are learning about a simple check digit method for data validation. In this method, a single check digit is appended to the end of an original number to give a new number.

The students are studying a method which:

- calculates the sum of all the digits in the original number
- uses integer division to calculate the remainder when the sum is divided by 10
- uses the remainder as the check digit
- appends the check digit to the original number, creating the new number.

For example:

original number	4162
sum of all digits	$4 + 1 + 6 + 2 = 13$
remainder when the sum is divided by 10 using integer division	3
new number	41623

The original number is always at least three digits in length.

When the new number is input, the check digit is used to validate the new number.

- (b)** A function `CheckNumber()` will take an integer value and return the Boolean value `TRUE` if the check digit is correct, otherwise return `FALSE`

Write pseudocode for the function `CheckNumber()`

Assume that the parameter is valid.

[7]

17. M/J 25/P21/Q4/a**4** Study the algorithm:

```
DECLARE Chars : ARRAY[1:4] OF CHAR
DECLARE I, J : INTEGER
DECLARE Key : CHAR
I ← 2
Chars[1] ← 'D'
Chars[2] ← 'T'
Chars[3] ← 'H'
Chars[4] ← 'R'
WHILE I ≤ 4                                //Outer loop
    Key ← Chars[I]
    J ← I - 1
    WHILE J ≥ 0 AND Chars[J] > Key        //Inner loop
        Chars[J + 1] ← Chars[J]
        J ← J - 1
    ENDWHILE
    Chars[J + 1] ← Key
    I ← I + 1
ENDWHILE
```

(a) The outer loop structure used in the algorithm is **not** the most appropriate one to use.

State the type of loop structure that would be the most appropriate to use and justify why it is the most appropriate.

Loop structure

Justification

[2]

18. M/J 25/P21/Q7/a

- 7 A program is being developed to implement a customer loyalty scheme for a coffee shop.

The programmer has decided that the following data items need to be stored for each customer:

Data item	Description
customer ID	a unique six-digit string
points	an integer value that is increased by one for every cup of coffee the customer orders

When a customer visits the shop and orders coffee, the scheme operates as follows:

- The total number of points is increased by the number of coffees ordered.
- If just one cup of coffee is ordered and the number of points goes above 10, then:
 - the cup of coffee they have just ordered is given to them free of charge
 - the number of points is reduced by 11.
- If the order is for multiple coffees and the number of points goes above 10, then:
 - they get one coffee free of charge for every 11 points
 - the number of points is reduced by 11 for each free coffee.

For example, the:

- customer currently has 9 points
- customer orders 16 cups of coffee
- total number of points now becomes 25
- customer gets 2 free coffees and now has 3 points left.

The programmer has defined a program module that is called every time a customer places an order:

Module	Description
<code>CustomerOrder()</code>	• called with two integer parameters: ◦ the number of coffees ordered ◦ the current number of points • output a suitable message giving the number of free coffees • return a value for the new points total.

- (a) Write pseudocode for module `CustomerOrder()`

[6]

19. M/J 25/P23/Q1/a,b

Uses the insert (Appendix)

1 A program is being developed for the management of a sports centre.**(a)** The programmer developing the software decides to use modules (procedures or functions) and local variables. They also decide not to use global variables.**(i)** State **two** reasons why the programmer decides to use modules.

1

2

[2]

(ii) State **one** difference between local and global variables.

[1]

(iii) State **two** benefits of using local variables.

1

2

[2]

(b) The pseudocode design contains a number of expressions.

Complete each pseudocode expression so that it evaluates to the value shown.

Refer to the **insert** for the list of pseudocode functions and operators.

Expression	Evaluates to
(68)	' D '
..... (04 / 02 / 2025)	2
TRUE	FALSE
..... (..... ("Court1.4Upper" , ,))	1 . 4

[5]

20. M/J 25/P23/Q5

- 5 A module `Parity()` takes a string as a parameter. The parameter has the identifier `BitString` and it represents a binary value.

The module will concatenate a single character to the end of `BitString` by applying one of the two rules:

- '0' if `BitString` contains an even number of 1s
- '1' if `BitString` contains an odd number of 1s.

The modified value of `BitString` is then returned.

For example:

Parameter	String returned	Explanation
"0010010110"	"00100101100"	there are an even number of 1s in the string passed to <code>Parity()</code> so a '0' is concatenated to the end of <code>BitString</code>
"101010"	"1010101"	there are an odd number of 1s in the string passed to <code>Parity()</code> so a '1' is concatenated to the end of <code>BitString</code>

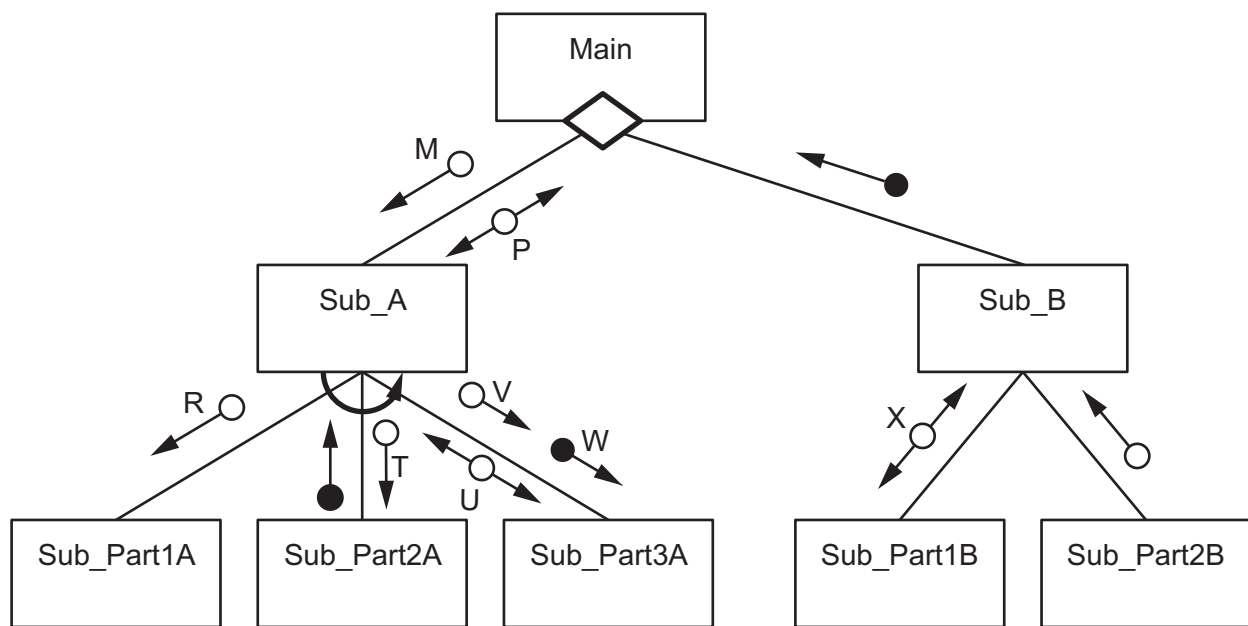
Write pseudocode for the module `Parity()`

Assume the parameter `BitString` can only contain the characters '0' and '1'

[8]

21. M/J 25/P23/Q6/a

6 Study the structure chart:



Some of the parameter data types are:

- R and V are of type `INTEGER`
- T is of type `REAL`
- U is of type `STRING`
- W is of type `BOOLEAN`.

Some of the modules in the structure chart are functions, others are procedures.

(a) Explain **one** difference between functions and procedures.

[1]

22. M/J 25/P23/Q7/a,b(ii)

7 A program is being developed to implement a customer loyalty scheme for a coffee shop.

The programmer has decided that the following data items need to be stored for each customer:

Data item	Description
customer ID	a unique six-digit string
loyalty points	an integer value that depends on how often the customer visits the coffee shop
last visit date	the date the customer last visited the coffee shop

The programmer has defined a program module:

Module	Description
UpdateVisit()	- called with two parameters: - loyalty points of type <code>INTEGER</code> - last visit date of type <code>DATE</code> - change the loyalty points: - increase by four if the last visit date and the current date are in the same month - otherwise increase by one - change the last visit date to the current date - the changed values must be available to the code that follows the call to <code>UpdateVisit()</code>

(a) (i) Write pseudocode for module `UpdateVisit()`

Assume the customer has made at least **one** previous visit.

[5]

(ii) The programmer decides to amend the `UpdateVisit()` module so that loyalty points are further increased if the customer visits the coffee shop the same day of the week as their last visit.

Write the pseudocode for this condition.

[1]

(b) A text file `Loyalty.txt` will be used to store the data items for the loyalty scheme.

The text file `Loyalty.txt` contains only one line of data for each customer.

Each data item is separated by a comma:

`<CustomerID>,<LoyaltyPointsString>,<LastVisitDateString>`

`LastVisitDateString` is always eight characters in length in the format: `DDMMYYYY`.

An example of how a line of data will be stored in the text file `Loyalty.txt` is:

`"100123,132,05032025"`

This example shows that the customer with an ID of 100123 had 132 loyalty points following their last visit to the coffee shop on 05/03/2025.

If a customer visits the coffee shop on a Monday, the value of their loyalty points is increased by 10.

The programmer has defined **two** more program modules:

Module	Description
<code>FindCustomer()</code> (is already written)	- called with a parameter of type <code>STRING</code> that represents a customer ID - returns the line of data read from the text file <code>Loyalty.txt</code> containing the data items for that customer
<code>MondayCheck()</code>	- called with a parameter of type <code>STRING</code> that represents the customer ID of a customer - calls <code>FindCustomer()</code> - extracts the loyalty points from the string returned by <code>FindCustomer()</code> - increases the loyalty points for the current customer by 10 if the day visited is a Monday - returns an integer value representing the loyalty points

(ii) A date is stored in `LastVisitDateString` in the format:

DDMMYYYY

DD is a 2-digit string

MM is a 2-digit string

YYYY is a 4-digit string.

For example, the date 03/09/2024 is stored as "03092024"

An algorithm is needed to convert the string stored in `LastVisitDateString` to data type `DATE` which is then stored in the variable `LastVisitDate`.

Complete the pseudocode for this algorithm:

Assume that `LastVisitDateString` has been declared and contains valid data.

```
DECLARE DayInt, MonthInt, YearInt : INTEGER
```

```
DECLARE LastVisitDate : DATE
```

[4]

23. O/N 24/P21/Q1/b,d

- 1 A program will calculate the tax payable based on the cost of an item.

Calculations will occur at many places in the program and these involve the use of one of three tax rates.

Tax rate values represent a percentage. For example, a tax rate value of 5.23 represents 5.23%. In this case, the tax payable on an item costing \$100 would be \$5.23.

Tax rate values are used at several places within the program. One example is given in pseudocode as follows:

```
HighRate ← FALSE
CASE OF ItemCost
    <= 50 : TaxRate ← 3.75      // tax rate of 3.75%
    <= 200 : TaxRate ← 5.23    // tax rate of 5.23%
    > 200 : TaxRate ← 6.25     // tax rate of 6.25%
        HighRate ← TRUE
ENDCASE
TaxPayable ← ItemCost * TaxRate // tax payable
```

- (b) During the design of the program, tax rate values have been used wherever they are needed as shown in the pseudocode example above. Tax rates do not change while the program runs.

- (i) Identify a more appropriate way of representing the tax rate values in the final program.

[1]

- (ii) Describe the benefits of your answer to part (b)(i) with reference to this program.

[3]

- (d) The final CASE condition (> 200) in the pseudocode example could be replaced with a keyword.

Give the keyword.

[1]

24. O/N 24/P21/Q2

- 2 A program uses three global integer variables HH, MM and SS to represent the current time in hours, minutes and seconds using the 24-hour clock notation.

Midnight would be represented as 00:00:00 (HH:MM:SS). If the variables HH, MM and SS contained the values 16, 30 and 10 respectively, then the time would be 16:30:10 or just after 4.30 in the afternoon.

A procedure Tick() will be called every second.

The procedure Tick() will:

- update the value in SS each time it is called
- update the values in HH and MM as appropriate
- call a procedure CheckAlarm() at the start of each minute
- call a procedure NewDay() whenever the time reaches midnight.

Complete the pseudocode for procedure Tick().

```
PROCEDURE Tick()
ENDPROCEDURE
```

[6]

25. O/N 24/P21/Q4/b,c

- 4 A program includes the following assignment statement:

```
Result ← STR_TO_NUM(x) / STR_TO_NUM(y)
```

When the program evaluates the expression in the statement, it performs a calculation.

Variable `Result` is of type real and variables `x` and `y` are of type string.

Two checks are required before the calculation is performed:

1. The two strings represent valid numeric values.
2. The numeric value of string `y` is not zero.

- (b) The designer considers implementing the **checks and calculation** as a module (a procedure or a function). One reason for this is that the same checks and calculations are performed at several places in the program.

Give **another** reason why this is a suitable approach **and** state what is avoided by this approach.

Reason

Avoided

[2]

- (c) The module to perform the checks and calculation will be implemented as a function. The function will need to return both a real **and** a Boolean value. To achieve this a record type is defined in pseudocode as follows:

```
TYPE Result
    DECLARE Done : BOOLEAN
    DECLARE Value : REAL
ENDTYPE
```

The function `Evaluate()` will:

- take two parameters of type string representing the two numeric values
- return a variable of type `Result` with the `Done` field set to `FALSE` if either of the following applies:
 - at least one of the strings does **not** represent a valid numeric value
 - the numeric value of the string representing value `y` is zero
- otherwise return a variable of type `Result` with the `Done` field set to `TRUE` and the `Value` field assigned the result of the formula (based on the numeric value of the two parameters).

Write pseudocode for the function `Evaluate()`.

[6]

26. O/N 24/P21/Q6/b

- 6 A factory produces food items. The items must be used within a certain number of days after their production date. The number of days is known as the shelf life. It is different for each type of item but is always a whole number in the range 1 to 21 (inclusive).

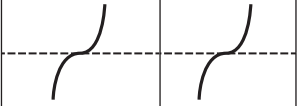
The latest date that an item can be used is called the 'use-by' date.

A program is needed to produce labels which show the 'use-by' date.

Part of the program is a function `GetDate()` which will:

- take two parameters: a production date and a value representing the shelf life
- return the corresponding 'use-by' date.

The program contains a global 1D array `DaysInMonth` of type integer which stores the number of days in each month (index 1 is January):

Index	Value
1	31
2	28
3	31
4	30
	
11	30
12	31

Note: Leap years are **not** considered

(b) Complete the pseudocode for the function `GetDate()`.

Date functions from the **insert** should be used in your solution.

```
FUNCTION GetDate(ProductionDate : DATE, ShelfLife : INTEGER) RETURNS DATE
ENDFUNCTION
```

[7]

Insert of this paper (its text differs from the Appendix copy):

An error will be generated if a function call is not properly formed or if the parameters are of an incorrect type or an incorrect value.

String and character functions

- A string of length 1 may be considered to be either of type `CHAR` or `STRING`
- A `CHAR` may be assigned to, or concatenated with, a `STRING`
- A `STRING` of length greater than 1 cannot be assigned to a `CHAR`

```
LEFT(ThisString : STRING, x : INTEGER) RETURNS STRING
```

returns leftmost x characters from `ThisString`

Example: `LEFT("ABCDEFGH", 3)` returns "ABC"

```
RIGHT(ThisString : STRING, x : INTEGER) RETURNS STRING
```

returns rightmost x characters from `ThisString`

Example: `RIGHT("ABCDEFGH", 3)` returns "FGH"

```
MID(ThisString : STRING, x : INTEGER, y : INTEGER) RETURNS STRING
```

returns a string of length y starting at position x from `ThisString`

Example: `MID("ABCDEFGH", 2, 3)` returns "BCD"

```
LENGTH(ThisString : STRING) RETURNS INTEGER
```

returns the integer value representing the length of `ThisString`

Example: `LENGTH("Happy Days")` returns 10

TO_UPPER(x : <datatype>) RETURNS <datatype>

<datatype> may be CHAR or STRING

returns an object of type <datatype> formed by converting all characters of x to upper case.

Examples:

- TO_UPPER("Error 803") returns "ERROR 803"
- TO_UPPER('a') returns 'A'

TO_LOWER(x : <datatype>) RETURNS <datatype>

<datatype> may be CHAR or STRING

returns an object of type <datatype> formed by converting all characters of x to lower case.

Examples:

- TO_LOWER("JIM 803") returns "jim 803"
- TO_LOWER('W') returns 'w'

NUM_TO_STR(x : <datatype1>) RETURNS <datatype2>

returns a string representation of a numeric value.

<datatype1> may be REAL or INTEGER, <datatype2> may be CHAR or STRING

Example: NUM_TO_STR(87.5) returns "87.5"

If x is a negative value, the returned value will be a string beginning with the '-' character.

STR_TO_NUM(x : <datatype1>) RETURNS <datatype2>

returns a numeric representation of a string.

<datatype1> may be CHAR or STRING, <datatype2> may be REAL or INTEGER

Example: STR_TO_NUM("23.45") returns 23.45

If the string begins with the minus character '-', the returned value will be negative.

IS_NUM(ThisString : <datatype>) RETURNS BOOLEAN

returns TRUE if ThisString represents a valid numeric value.

<datatype> may be CHAR or STRING

Example: IS_NUM("-12.36") returns TRUE

ASC(ThisChar : CHAR) RETURNS INTEGER

returns an integer value (the ASCII value) of ThisChar

Example: ASC('A') returns 65, ASC('B') returns 66

CHR(x : INTEGER) RETURNS CHAR

returns the character whose integer value (the ASCII value) is x

Example: CHR(65) returns 'A', CHR(66) returns 'B'

Numeric functions

INT(x : REAL) RETURNS INTEGER

returns the integer part of x

Example: INT(27.5415) returns 27

RAND(x : INTEGER) RETURNS REAL

returns a real number in the range 0 to x (not inclusive of x).

Example: RAND(87) could return 35.430729

Date functions

Date format is assumed to be DD/MM/YYYY unless otherwise stated.

`DAY(ThisDate : DATE) RETURNS INTEGER`

returns the day number from `ThisDate`

Example: `DAY(04/10/2003)` returns 4

`MONTH(ThisDate : DATE) RETURNS INTEGER`

returns the month number from `ThisDate`

Example: `MONTH(04/10/2003)` returns 10

`YEAR(ThisDate : DATE) RETURNS INTEGER`

returns the year number from `ThisDate`

Example: `YEAR(04/10/2003)` returns 2003

`DAYINDEX(ThisDate : DATE) RETURNS INTEGER`

returns the day index number from `ThisDate` where Sunday = 1, Monday = 2 etc.

Example: `DAYINDEX(07/11/2023)` returns 3

`SETDATE(Day, Month, Year : INTEGER) RETURNS DATE`

returns a value of type `DATE` with the value of `<Day>/<Month>/<Year>`

Example: `SETDATE(26, 10, 2003)` returns a date corresponding to 26/10/2003

`TODAY() RETURNS DATE`

returns a value of type `DATE` corresponding to the current date.

Text file functions

`EOF(FileName : STRING) RETURNS BOOLEAN`

returns `TRUE` if there are no more lines to be read from file `FileName`
will generate an error if the file is not already open in `READ` mode.

Operators

An error will be generated if an operator is used with a value or values of an incorrect type.

&	concatenates (joins) two strings. Example: "Summer" & " " & "Pudding" evaluates to "Summer Pudding" may also be used to concatenate a CHAR with a STRING
AND	performs a logical AND on two Boolean values. Example: TRUE AND FALSE evaluates to FALSE
OR	performs a logical OR on two Boolean values. Example: TRUE OR FALSE evaluates to TRUE
NOT	performs a logical NOT on a Boolean value. Example: NOT TRUE evaluates to FALSE
MOD	finds the remainder when one number is divided by another. Example: 10 MOD 3 evaluates to 1
DIV	finds the quotient when one number is divided by another. Example 10 DIV 3 evaluates to 3

Comparison operators

=	used to compare two items of the same type. evaluates to TRUE if the condition is true, otherwise evaluates to FALSE
>	Notes: - may be used to compare types REAL and INTEGER - may be used to compare types CHAR and STRING - case sensitive when used to compare types CHAR and/or STRING - cannot be used to compare two records
<	
>=	
<=	
<>	
	Examples: "Program" = "program" evaluates to FALSE - Count = 4 evaluates to TRUE when Count contains the value 4

27. O/N 24/P21/Q8/a

- 8 An exam paper has a maximum of 75 marks. One of five pass grades (A to E) is assigned, depending on the mark obtained. The lowest mark for a given grade is known as the grade boundary. For example, if the grade boundary for an A grade is 65 marks, then any candidate who achieves a mark of 65 or above will be awarded an A. A grade of U is awarded for marks below the E grade boundary.

The five grade boundaries are stored in a global 1D array `GradeBoundary` of type integer.

For example:

Element	Value	Comment
<code>GradeBoundary[1]</code>	65	The minimum mark for an A grade.
<code>GradeBoundary[2]</code>	57	The minimum mark for a B grade.
<code>GradeBoundary[3]</code>	43	The minimum mark for a C grade.
<code>GradeBoundary[4]</code>	35	The minimum mark for a D grade.
<code>GradeBoundary[5]</code>	27	The minimum mark for an E grade.

A global 2D array `Result` of type integer contains candidate marks for the exam. Each row relates to one candidate. Column 1 contains the candidate mark and column 2 contains the unique candidate ID.

For example, for the fourth and fifth candidates:

Element	Mark	Element	ID
<code>Result[4, 1]</code>	56	<code>Result[4, 2]</code>	1074832
<code>Result[5, 1]</code>	54	<code>Result[5, 2]</code>	2573839

There are more rows in the array than candidates who sit the exam. Any unused rows will be at the end of the array.

Candidate papers that are given a mark within two marks of any grade boundary must be checked.

For example, given the values in the example grade boundaries above, any paper that was awarded between 41 and 45 marks (inclusive) would need to be checked.

A program is being written to identify papers that need to be checked.

The programmer has defined the first program module as follows:

Module	Description
<code>CheckMark()</code>	- called with a parameter of type integer representing a candidate mark - returns <code>TRUE</code> if the mark is within 2 of any of the five grade boundaries, otherwise returns <code>FALSE</code>

- (a) Write pseudocode for module `CheckMark()`.

28. O/N 24/P22/Q2/a,b(ii)

2 A program is being developed to process bank card information.

When a card number is displayed, all the characters **except** the last four are replaced with the asterisk character ' * '.

Card numbers are stored as strings. The strings are between 10 and 20 characters in length.

The function `Conceal()` will take a string representing a card number and return a modified string.

Example strings:

Original string	Modified string
"1234567890"	"*****7890"
"1234567897652"	"*****7652"
"1234567890123456"	"*****3456"

(a) The function `Conceal()` will:

- take a numeric string as a parameter representing the card number
- return a string in which the asterisk character replaces all except the last four characters of the card number parameter.

Write pseudocode for the function `Conceal()`.

[6]

(b) The requirements have been changed. `Conceal()` will now be written as a procedure which will process 100 card numbers each time it is called.

The card numbers will be stored in a 2D array `CardNumber`. The original string will be stored in column one and the modified string in column two.

(ii) The new procedure `Conceal()` will write the modified string to the corresponding element in column two.

The array `CardNumber` is passed as a parameter to the new procedure `Conceal()`.

Identify how this parameter should be specified in the new procedure header.

[1]

29. O/N 24/P22/Q4

- 4 A global integer variable `Tick` is always incremented every millisecond (1000 times per second) regardless of the other programs running.

The value of `Tick` can be read by any program but the value should not be changed.

Assume that the value of `Tick` does not overflow.

As an example, the following pseudocode algorithm would output "Goodbye" 40 seconds after outputting "Hello".

```
DECLARE Start : INTEGER
OUTPUT "Hello"
Start ← Tick

REPEAT
    //do nothing
UNTIL Tick = Start + 40000

OUTPUT "Goodbye"
```

A program is needed to help a user to time an event such as boiling an egg.

The time taken for the event is known as the elapsed time.

The program contains a procedure `Timer()` which will:

- take two integer values representing an elapsed time in minutes and seconds
- use the value of variable `Tick` to calculate the elapsed time
- output a warning message 30 seconds before the elapsed time is up
- output a final message when the total time has elapsed.

For example, to set an alarm for 5 minutes and 45 seconds the program makes the following call:

```
CALL Timer(5, 45)
```

When 5 minutes and 15 seconds have elapsed, the program will output:

```
"30 seconds to go"
```

When 5 minutes and 45 seconds have elapsed, the program will output:

```
"The time is up!"
```

Write pseudocode for the procedure `Timer()`.

30. O/N 24/P22/Q5/c

- 5 A program contains a global 1D array `Data` with 100 elements of type `INTEGER`.

The program contains a function `Process()` expressed in pseudocode as follows:

```
FUNCTION Process(Number : INTEGER, Label : STRING) RETURNS STRING
  DECLARE Index, Count : INTEGER
  DECLARE ReturnValue : STRING

  Count ← INT(100 / Number)
  Index ← Data[Number]

  CASE OF (Index MOD 2)
    0 : ReturnValue ← TO_UPPER(RIGHT(Label, Count))
    1 : ReturnValue ← "*****"
  ENDCASE

  RETURN RetVal
ENDFUNCTION
```

- (c) The function `Process()` contains a selection construct using a `CASE` structure.

Write pseudocode using a single selection construct with the same functionality **without** using a `CASE` structure.

[2]

31. O/N 24/P23/Q1/c

- (b) Complete the table by giving the appropriate data type in each case.

Variable	Example data value	Data type
Available	TRUE	
Received	"18/04/2021"	
Index	100	

[3]

- (c) Evaluate each expression in the table by using the data values shown in part (b).

Write 'ERROR' if the expression contains an error.

Expression	Evaluates to
Available AND NOT(Index > 100)	
Index MOD 30	
NUM_TO_STR(Index + "33")	

[3]

32. O/N 24/P23/Q6

- 6 In some countries, on the third Sunday in March, daylight saving time begins when clocks move forward by one hour.

A module `AdjustClock()` will take an integer parameter representing a year. The module will return an integer value representing the number of the day in March on which the clocks move forward.

For example, the following line of pseudocode would assign `DayNumber` the value 20:

DayNumber $\leftarrow$ AdjustClock(2022)

Write pseudocode for the function AdjustClock().

Date functions from the **insert** should be used in your solution.

[7]

Insert of this paper (its text differs from the Appendix copy):

An error will be generated if a function call is not properly formed or if the parameters are of an incorrect type or an incorrect value.

String and character functions

- A string of length 1 may be considered to be either of type CHAR or STRING
- A CHAR may be assigned to, or concatenated with, a STRING
- A STRING of length greater than 1 cannot be assigned to a CHAR

LEFT(ThisString : STRING, x : INTEGER) RETURNS STRING

returns leftmost x characters from ThisString

Example: LEFT("ABCDEFGH", 3) returns "ABC"

RIGHT(ThisString : STRING, x : INTEGER) RETURNS STRING

returns rightmost x characters from ThisString

Example: RIGHT("ABCDEFGH", 3) returns "FGH"

MID(ThisString : STRING, x : INTEGER, y : INTEGER) RETURNS STRING

returns a string of length y starting at position x from ThisString

Example: MID("ABCDEFGH", 2, 3) returns "BCD"

LENGTH(ThisString : STRING) RETURNS INTEGER

returns the integer value representing the length of ThisString

Example: LENGTH("Happy Days") returns 10

TO_UPPER(x : <datatype>) RETURNS <datatype>

<datatype> may be CHAR or STRING

returns an object of type <datatype> formed by converting all characters of x to upper case.

Examples:

- TO_UPPER("Error 803") returns "ERROR 803"
- TO_UPPER('a') returns 'A'

TO_LOWER(x : <datatype>) RETURNS <datatype>

<datatype> may be CHAR or STRING

returns an object of type <datatype> formed by converting all characters of x to lower case.

Examples:

- TO_LOWER("JIM 803") returns "jim 803"
- TO_LOWER('W') returns 'w'

NUM_TO_STR(x : <datatype1>) RETURNS <datatype2>

returns a string representation of a numeric value.

<datatype1> may be REAL or INTEGER, <datatype2> may be CHAR or STRING

Example: NUM_TO_STR(87.5) returns "87.5"

If x is a negative value, the returned value will be a string beginning with the '-' character.

STR_TO_NUM(x : <datatype1>) RETURNS <datatype2>

returns a numeric representation of a string.

<datatype1> may be CHAR or STRING, <datatype2> may be REAL or INTEGER

Example: STR_TO_NUM("23.45") returns 23.45

If the string begins with the minus character '-', the returned value will be negative.

IS_NUM(ThisString : <datatype>) RETURNS BOOLEAN

returns TRUE if ThisString represents a valid numeric value.

<datatype> may be CHAR or STRING

Example: IS_NUM("-12.36") returns TRUE

ASC(ThisChar : CHAR) RETURNS INTEGER

returns an integer value (the ASCII value) of ThisChar

Example: ASC('A') returns 65, ASC('B') returns 66

CHR(x : INTEGER) RETURNS CHAR

returns the character whose integer value (the ASCII value) is x

Example: CHR(65) returns 'A', CHR(66) returns 'B'

Numeric functions

INT(x : REAL) RETURNS INTEGER

returns the integer part of x

Example: INT(27.5415) returns 27

RAND(x : INTEGER) RETURNS REAL

returns a real number in the range 0 to x (not inclusive of x).

Example: RAND(87) could return 35.430729

Date functions

Date format is assumed to be DD/MM/YYYY unless otherwise stated.

DAY(ThisDate : DATE) RETURNS INTEGER

returns the day number from ThisDate

Example: DAY(04/10/2003) returns 4

MONTH(ThisDate : DATE) RETURNS INTEGER

returns the month number from ThisDate

Example: MONTH(04/10/2003) returns 10

YEAR(ThisDate : DATE) RETURNS INTEGER

returns the year number from ThisDate

Example: YEAR(04/10/2003) returns 2003

`DAYINDEX(ThisDate : DATE) RETURNS INTEGER`

returns the day index number from `ThisDate` where Sunday = 1, Monday = 2 etc.

Example: `DAYINDEX(07/11/2023)` returns 3

`SETDATE(Day, Month, Year : INTEGER) RETURNS DATE`

returns a value of type `DATE` with the value of `<Day>/<Month>/<Year>`

Example: `SETDATE(26, 10, 2003)` returns a date corresponding to 26/10/2003

`TODAY() RETURNS DATE`

returns a value of type `DATE` corresponding to the current date.

Text file functions

`EOF(FileName : STRING) RETURNS BOOLEAN`

returns `TRUE` if there are no more lines to be read from file `FileName`
will generate an error if the file is not already open in `READ` mode.

Operators

An error will be generated if an operator is used with a value or values of an incorrect type.

<code>&</code>	concatenates (joins) two strings. Example: <code>"Summer" & " " & "Pudding"</code> evaluates to <code>"Summer Pudding"</code> may also be used to concatenate a <code>CHAR</code> with a <code>STRING</code>
<code>AND</code>	performs a logical <code>AND</code> on two Boolean values. Example: <code>TRUE AND FALSE</code> evaluates to <code>FALSE</code>
<code>OR</code>	performs a logical <code>OR</code> on two Boolean values. Example: <code>TRUE OR FALSE</code> evaluates to <code>TRUE</code>
<code>NOT</code>	performs a logical <code>NOT</code> on a Boolean value. Example: <code>NOT TRUE</code> evaluates to <code>FALSE</code>
<code>MOD</code>	finds the remainder when one number is divided by another. Example: <code>10 MOD 3</code> evaluates to 1
<code>DIV</code>	finds the quotient when one number is divided by another. Example <code>10 DIV 3</code> evaluates to 3

Comparison operators

<code>=</code>	used to compare two items of the same type. evaluates to <code>TRUE</code> if the condition is true, otherwise evaluates to <code>FALSE</code>
<code>></code>	Notes: - may be used to compare types <code>REAL</code> and <code>INTEGER</code> - may be used to compare types <code>CHAR</code> and <code>STRING</code> - case sensitive when used to compare types <code>CHAR</code> and/or <code>STRING</code> - cannot be used to compare two records
<code><</code>	
<code>>=</code>	
<code><=</code>	
<code><></code>	
	Examples: <code>"Program" = "program"</code> evaluates to <code>FALSE</code> <code>Count = 4</code> evaluates to <code>TRUE</code> when <code>Count</code> contains the value 4

33. M/J 24/P21/Q1/b

1 An algorithm is developed in pseudocode before being coded in a programming language.

(b) Other variables in the program have example values as shown:

Variable	Value
Sorted	False
Tries	9
ID	"ZGAC001"

Complete the table by evaluating each expression, using the example values.

Expression	Evaluates to
<code>Tries < 10 AND NOT Sorted</code>	
<code>Tries MOD 4</code>	
<code>TO_LOWER(MID(ID, 3, 1))</code>	
<code>LENGTH(ID & "xx") >= Tries</code>	

[4]

34. M/J 24/P21/Q2/b

2 An algorithm has three steps. It will:

1. repeatedly input a pair of numeric values A and B
2. count the number of pairs that are input until A has been greater than B 10 times
3. output the number of pairs that were input.

(b) Step 1 of the algorithm is changed.

A variable `ThisSequence` is used to enter a sequence of 10 pairs of numeric values, using a single input statement.

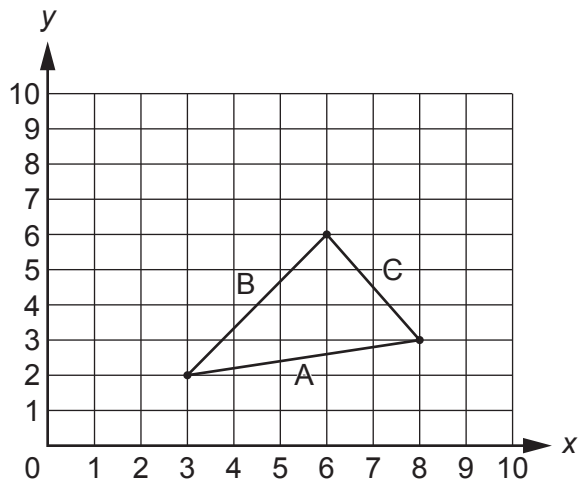
Following the input of `ThisSequence` the revised algorithm will extract the pairs of numbers.

Describe the variable `ThisSequence` and how the numbers are extracted.

[2]

35. M/J 24/P21/Q6/a

6 Three points on a grid form a triangle with sides of length A, B and C as shown in the example:



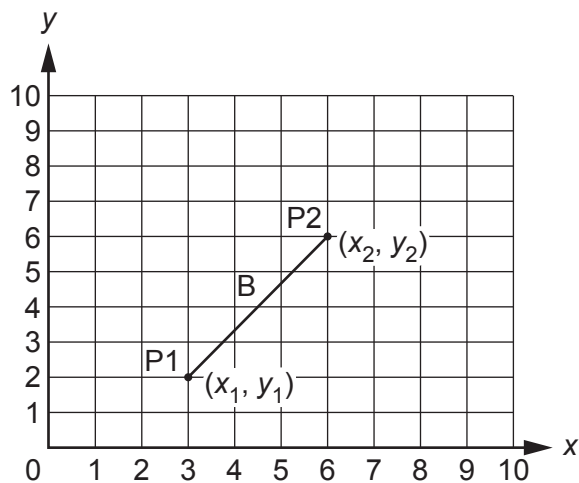
A triangle is said to be right-angled if the following test is true (where A is the length of the longest side):

$$A^2 = B^2 + C^2$$

A^2 means A multiplied by A, for example 3^2 means 3×3 which evaluates to 9

You can calculate A^2 , B^2 and C^2 by using the coordinates of the endpoints of each line.

For example, B^2 is calculated as follows:



The endpoints, P1 and P2, have the coordinates (3, 2) and (6, 6).

The value B^2 is given by the formula:

$$B^2 = (x_1 - x_2)^2 + (y_1 - y_2)^2$$

In this example:

$$B^2 = (3 - 6)^2 + (2 - 6)^2$$

$$B^2 = (-3)^2 + (-4)^2$$

$$B^2 = 9 + 16$$

$$B^2 = 25$$

(a) A function `IsRA()` will:

- take three sets of integers as parameters representing the coordinates of the three endpoints that form a triangle
- return `TRUE` if the endpoints form a right-angled triangle, otherwise return `FALSE`.

In pseudocode, the operator ‘^’ represents an exponent, which is the number of times a value is multiplied by itself. For example, the expression Value^2 may be written in pseudocode as `Value ^ 2`

Complete the pseudocode for the function `IsRA()`.

```
FUNCTION IsRA(x1, y1, x2, y2, x3, y3 : INTEGER) RETURNS BOOLEAN
    ENDFUNCTION
```

[6]

36. M/J 24/P21/Q8/a

- 8 A teacher is designing a program to process pseudocode projects written by her students.

Each student project is stored in a text file.

The process is split into a number of stages. Each stage performs a different task and creates a new file named as shown:

File name	Comment
MichaelAday_src.txt	student project file produced by student Michael Aday
MichaelAday_S1.txt	file produced by stage 1
MichaelAday_S2.txt	file produced by stage 2

The teacher has defined the first program module as follows:

Module	Description
<code>DeleteComment()</code>	• called with a parameter of type string representing a line of pseudocode from a student's project file • returns the line after removing any comments Note on comments: A comment starts with two forward slash characters and includes all the remaining characters on the line. The following example shows a string before and after the comment has been removed: Before: <code>IF X2 > 13 THEN //check if limit exceeded</code> After: <code>IF X2 > 13 THEN</code>

- (a) Complete the pseudocode for module `DeleteComment()`.

```
FUNCTION DeleteComment(Line : STRING) RETURNS STRING
    ENDFUNCTION
```

[8]

37. M/J 24/P22/Q1/b

Uses the insert (Appendix)

(b) Program variables have data types as follows:

Variable	Data type
MyChar	CHAR
MyString	STRING
MyInt	INTEGER

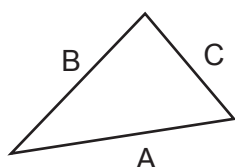
Complete the table by filling in each gap with a function (from the **insert**) so that each expression is valid.

Expression
MyInt ← (3.1415926)
MyChar ← ("Elwood", 3, 1)
MyString ← (..... (27.509))
MyInt ← (..... ("ABC123", 3))

[4]

38. M/J 24/P22/Q4

4 A triangle has sides of length A, B and C.



In this example, A is the length of the longest side.

This triangle is said to be right-angled if the following equation is true:

$$A \times A = (B \times B) + (C \times C)$$

A procedure will be written to check whether three lengths represent a right-angled triangle. The lengths will be input in any sequence.

The procedure `IsRA()` will:

- prompt and input three integer values representing the three lengths
- test whether the three lengths correspond to the sides of a right-angled triangle
- output a suitable message.

The length of the longest side may **not** be the first value input.

Write pseudocode for the procedure `IsRA()`.

[5]

39. M/J 24/P22/Q5/a(ii)

5 A program is being designed in pseudocode.

The program contains a global 1D array `Data` of type string containing 200 elements.

The first element has the index value 1.

A procedure `Process()` is written to initialise the values in the array:

```
PROCEDURE Process(Label : STRING)
  DECLARE Index : INTEGER
  Index ← 0
  INPUT Data[Index]
  WHILE Index < 200
    Index ← Index + 1
    CASE OF (Index MOD 2)
      0 : Data[Index] ← TO_UPPER(Label)
      1 : Data[Index] ← TO_LOWER(Label)
      OTHERWISE : OUTPUT "Alarm 1201"
    ENDCASE
  NEXT Index
  OUTPUT "Completed " & Index & " times"
ENDPROCEDURE
```

(ii) The procedure contains a statement that is **not** needed.

Identify the pseudocode statement **and** explain why it is **not** needed.

Statement

Explanation

[2]

40. M/J 24/P22/Q6/a

- 6** A music player stores music in a digital form and has a display which shows the track being played.

(a) Up to 16 characters can be displayed. Track titles longer than 16 characters will need to be trimmed as follows:

- Words must be removed from the end of the track title until the resulting title is less than 14 characters.
- When a word is removed, the space in front of that word is also removed.
- Three dots are added to the end of the last word displayed when one or more words have been removed.

The table below shows some examples:

Original title	Display string															
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Bat out of Hull	B	a	t		o	u	t		o	f		H	u	l	l	
Bohemian Symphony	B	o	h	e	m	i	a	n	.	.	.					
Paperbook Writer	P	a	p	e	r	b	o	o	k		W	r	i	t	e	r
Chris Sings the Blues	C	h	r	i	s		S	i	n	g	s	.	.	.		
Green Home Alabama	G	r	e	e	n		H	o	m	e	.	.	.			

A function `Trim()` will:

- take a string representing the original title
- return the string to be displayed.

Assume:

- Words in the original title are separated by a single space character.
- There are no spaces before the first word or after the last word of the original title.
- The first word of the original title is less than 14 characters.

Write pseudocode for the function `Trim()`.

41. M/J 24/P22/Q8/b

- 8 A teacher is designing a program to process pseudocode projects written by her students.

Each student project is stored in a text file.

The process is split into a number of stages. Each stage performs a different task and creates a new file.

For example:

File name	Comment
MichaelAday_src.txt	Student project file produced by student Michael Aday
MichaelAday_S1.txt	File produced by stage 1
MichaelAday_S2.txt	File produced by stage 2

- (b) The teacher has defined the first program module as follows:

Module	Description
DeleteSpaces()	- called with a parameter of type string representing a line of pseudocode from a student's project file - returns the line after removing any leading space characters The following example shows a string before and after the leading spaces have been removed: Before: " IF X2 > 13 THEN" After: "IF X2 > 13 THEN"

Complete the pseudocode for module DeleteSpaces().

```
FUNCTION DeleteSpaces(Line : STRING) RETURNS STRING
ENDFUNCTION
```

[6]

42. M/J 24/P23/Q1/a,b

Uses the insert (Appendix)

- 1** A program uses many complex algorithms.

One algorithm is repeated in several places. The code for the algorithm is the same wherever it is used, but the calculations within the algorithm may operate on different data.

The result of each calculation is used by the code that follows it.

It is decided to modify the program and implement the algorithm as a separate module.

- (a) (i)** State **two** benefits of this modification to the existing program.

1

2

[2]

- (ii)** Describe how the modification would be implemented.

[3]



- (b)** Four of the expressions used in the program are represented by pseudocode in the table.

Complete each pseudocode expression with a function or operator so that it evaluates to the value shown.

Any functions and operators used must be defined in the **insert**.

Pseudocode expression	Evaluates to
..... ("Random", 2, 3)	"and"
5 + (10/11/2023)	15
..... ("45000")	TRUE
(20 3) + 1	3

[4]



43. M/J 24/P23/Q4/a

- 4 A procedure `TwoParts()` will input a sequence of real values, one at a time.

The procedure will:

- process the sequence in two parts
- form a first total by adding the values until the first zero
- form a second total by adding the values after the first zero until the second zero
- output the average of the two totals, together with a suitable message.

Values input in the first part are totalled using global variable `TotalA` and those input in the second part are totalled using global variable `TotalB`.

- (a) Write pseudocode for the procedure `TwoParts()`.

[6]

**44. M/J 24/P23/Q5/a,b**

- 5 A program is being designed in pseudocode.

The program contains the following declaration:

```
DECLARE Data : ARRAY[1:1000] OF STRING
```

A procedure `ArrayInitialise()` is written to initialise the values in the array:

```
PROCEDURE ArrayInitialise(Label : STRING)
  DECLARE Index : INTEGER
  Index ← 1
  WHILE Index ≤ 1000
    CASE OF (Index MOD 2)
      0 : Data[Index] ← FormatA(Label)
          Index ← Index + 1
      1 : Data[Index] ← FormatB(Label)
          Index ← Index + 1
    ENDCASE
  ENDWHILE
ENDPROCEDURE
```

Functions `FormatA()` and `FormatB()` apply fixed format case changes to the parameter string.

- (a) The design of the procedure does **not** use the most appropriate loop construct.

Suggest a **more appropriate** construct that could be used **and** explain your choice.

Construct

Explanation

[2]

- (b) The algorithm calls one of the functions `FormatA()` and `FormatB()` each time within the loop.

Explain why this is **not** efficient **and** suggest a more efficient solution.

[4]



45. M/J 24/P23/Q6/a,b(i)

- 6 A program displays a progress bar to inform the user of the progress of tasks that take a significant time to complete, such as those involving file transfer operations.

Task progress is divided into 11 steps. Each step represents the amount of progress as a percentage. An image is associated with each step and each image is stored in a different file.

Different progress bar images may be selected. For a given image, files all have the same filename root, with a different suffix.

The table illustrates the process for using the image with filename root `BargraphA`

Step	Percentage progress	Image filename	Image
1	< 10	BargraphA-1.bmp	
2	>= 10 and < 20	BargraphA-2.bmp	
3	>= 20 and < 30	BargraphA-3.bmp	
			
9	>= 80 and < 90	BargraphA-9.bmp	
10	>= 90 and < 100	BargraphA-10.bmp	
11	100	BargraphA-11.bmp	

A procedure `Progress()` will:

- be called with two parameters:
 - an integer representing the percentage progress (0 to 100 inclusive)
 - a string representing the image filename root
- generate the full image filename
- call a procedure `Display()` using the full image filename as the parameter.



- (a) Write pseudocode for procedure `Progress()`.

[6]

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52

- (b)** The definition of procedure `Progress()` is provided here for reference:

A procedure `Progress()` will:

- be called with two parameters:
 - an integer representing the percentage progress (0 to 100 inclusive)
 - a string representing the image filename root
- generate the full image filename
- call a procedure `Display()` using the full image filename as the parameter.

`Progress()` will be rewritten and a new module `Progress2()` produced with these requirements:

- an additional parameter of type integer will specify the total number of steps
- the image filename will be returned (procedure `Display()` will **not** be called from within `Progress2()`).

- (i) Write pseudocode for the new module header.

[2]

46. M/J 24/P23/Q7/a(ii)

- 7 Seven program modules form part of a program. A description of the relationship between the modules is summarised below. Any return values are stated in the description.

Module name	Description
Mod-A	calls Mod-B followed by Mod-C
Mod-B	- called with parameters Par1 and Par2 - calls either Mod-D or Mod-E, determined when the program runs - returns a Boolean value
Mod-C	- called with parameters Par1 and Par3 - Par3 is passed by reference - repeatedly calls Mod-F followed by Mod-G
Mod-D	called with parameter Par2
Mod-E	- called with parameter Par3 - returns an integer value
Mod-F	called with parameter Par3
Mod-G	- called with parameter Par3 - Par3 is passed by reference

Parameters in the table are as follows:

- Par1 and Par3 are of type string.
- Par2 is of type integer.

- (ii) Modules Mod-F and Mod-G are both called with Par3 as a parameter.
 In the case of Mod-F, the parameter is passed by value.
 In the case of Mod-G, the parameter is passed by reference.

Explain the effect of the **two** different ways of passing the parameter Par3.

[2]



48. O/N 23/P21/Q1/a,b,c

1 The following pseudocode represents part of the algorithm for a program:

```

CASE OF ThisValue
    < 30      : Level ← "Low"           // less than 30
               Check ← 1
    < 20      : Level ← "Very Low"      // less than 20
               Check ← ThisValue / 2
    30 TO 40  : Level ← "Medium"        // between 30 and 40
               Check ← ThisValue / 3
               Data[ThisValue] ← Data[ThisValue] + 1
    > 40      : Level ← "High"
ENDCASE

```

(a) Complete the table by writing the answer for each row:

	Answer
The value assigned to <code>Level</code> when <code>ThisValue</code> is 40	
The value assigned to <code>Check</code> when <code>ThisValue</code> is 36	
The value assigned to <code>Level</code> when <code>ThisValue</code> is 18	
The number of elements in array <code>Data</code> that may be incremented	

[4]

(b) The pseudocode contains four assignments to variable `Level`. One of these assignments will never be performed.

Identify this assignment **and** explain why this is the case.

[3]

(c) The following line is added immediately before the `ENDCASE` statement:

```
OTHERWISE : Level ← "Undefined"
```

State why this assignment is never performed.

[1]

49. O/N 23/P21/Q6/a,b

6 The pseudocode `OUTPUT` command starts each output on a new line.

- (a) A new procedure `MyOutput()` will take a string and a Boolean parameter. `MyOutput()` may be called repeatedly and will use concatenation to build a string using a global variable `MyString`, up to a maximum length of 255 characters.

`MyString` will be output in either of these two cases:

1. The Boolean parameter value is `TRUE`
2. The resulting string (after concatenation) would be longer than 255 characters.

If `MyString` is not output, the string is concatenated with `MyString`.

For example, the calls to `MyOutput()` given below would result in the output as shown:

```
MyOutput("Hello ", FALSE)
MyOutput("ginger ", FALSE)
MyOutput("cat", TRUE)
MyOutput("How are you?", TRUE)
```

Resulting output:

```
Hello ginger cat
How are you?
```

Notes:

- `MyString` is initialised to an empty string before `MyOutput()` is called for the first time.
- No string passed to `MyOutput()` will be longer than 255 characters.

Write pseudocode for `MyOutput()`.

[7]

- (b) The design of the procedure given in **part (a)** is modified and `MyString` is changed from a global to a local variable declared in `MyOutput()`.

When the modified procedure is converted into program code, it does not work as expected.

Explain why it does not work as expected.

[2]

50. O/N 23/P21/Q8/d

- 8 A class of students are developing a program to send data between computers. Many computers are connected together to form a wired network. Serial ports are used to connect one computer to another.

Each computer:

- is assigned a unique three-digit ID
- has three ports, each identified by an integer value
- is connected to between one and three other computers.

Messages are sent between computers as a string of characters organised into fields as shown:

<STX><DestinationID><SourceID><Data><ETX>

Field number	Field name	Description
n/a	STX	a single character marking the start of the message (ASCII value 02)
1	DestinationID	three numeric characters that identify the destination computer
2	SourceID	three numeric characters that identify the source computer
3	Data	a variable length string containing the data being sent (Minimum length is 1 character)
n/a	ETX	a single character marking the end of the message (ASCII value 03)

For example, the following message contains the data "Hello Kevin" being sent from computer "101" to computer "232":

<STX>"232101Hello Kevin"<ETX>

Each computer will run a copy of the same program. Each program will contain a global variable, `MyID` of type string, that contains the unique ID of the computer in which the program is running.

The programmer has defined the first two program modules as follows:

Module	Description
<code>Transmit()</code> (already written)	• takes two parameters: ◦ a string containing a message ◦ an integer containing a port number • transmits the message using the given port
<code>SendFile()</code>	• takes three parameters: ◦ a string containing a text file name ◦ a string containing a Destination ID ◦ an integer containing a Port number • transmits the file one line at a time • transmits a final message with data string "*****"

(d) A new module has been defined:

Module	Description
<code>GetField()</code>	• takes two parameters: ◦ a string containing a message ◦ an integer containing a field number • If the field number is valid (in the range 1 to 3, inclusive), it returns a string containing the required field, otherwise it returns an empty string.

As a reminder, a message is defined as follows:

<STX><DestinationID><SourceID><Data><ETX>

Field number	Field name	Description
Not applicable	STX	a single character marking the start of the message (ASCII value 02)
1	DestinationID	three numeric characters that identify the destination computer
2	SourceID	three numeric characters that identify the source computer
3	Data	a variable length string containing the data being sent (Minimum length is 1 character)
Not applicable	ETX	a single character marking the end of the message (ASCII value 03)

Write pseudocode for module `GetField()`.

[6]

51. O/N 23/P22/Q1/b

- 1 A shop sells car accessories. A customer order is created if an item cannot be supplied from current stock. A program is being developed to create and manage the customer orders.

(b) Other variables in the program have example values as shown:

Variable	Example value
Total	124.00
DepRate	2.00
Description	"AB12345:Cleaning Brush (small)"

Complete the table by evaluating each expression using the example values.

Expression	Evaluates to
<code>(Total * DepRate) + 1.5</code>	
<code>RIGHT(Description, 7)</code>	
<code>(LENGTH(Description) - 8) > 16</code>	
<code>NUM_TO_STR(INT(DepRate * 10)) & '%'</code>	

[4]

52. O/N 23/P22/Q2/b

2 An algorithm will:

1. input a sequence of integer values, one at a time
2. ignore all values until the value 27 is input, then sum the remaining values in the sequence
3. stop summing values when the value 0 is input and then output the sum of the values.

(b) The solution to the algorithm includes iteration.

Give the name of a suitable loop structure that could be used.

Justify your answer.

Name

Justification

[2]

53. O/N 23/P22/Q4/a

4 A procedure `Count()` will:

1. input a value (all values will be positive integers)
2. count the number of odd values and count the number of even values
3. repeat from step 1 until the value input is 99
4. output the two count values, with a suitable message.

The value 99 must not be counted.

(a) Write pseudocode for the procedure `Count()`.

[6]

54. O/N 23/P22/Q6/b(i,ii)

6 (a) A procedure `CreateFiles()` will take two parameters:

- a string representing a file name
- an integer representing the number of files to be created.

The procedure will create the number of text files specified.

Each file is given a different name. Each file name is formed by concatenating the file name with a suffix based on the file number. The suffix is always three characters.

For example, the call `CreateFiles("TestData", 3)` would result in the creation of the three files, `TestData.001`, `TestData.002` and `TestData.003`.

Each file will contain a single line. For example, file `TestData.002` would contain the string:

```
This is File TestData.002
```

Write pseudocode for `CreateFiles()`.

Assume both parameters are valid and that the integer value is between 1 and 999, inclusive.

[6]

(b) A module `CheckFiles()` will count the number of files produced by `CreateFiles()` in part **(a)**.

`CheckFiles()` will take a string representing a file name and return the number of files found.

(i) Identify the type of module that should be used for `CheckFiles()`.

[1]

(ii) Write the module header for `CheckFiles()`.

[1]

55. O/N 23/P22/Q8/b

8 A class of students are developing a program to send data between computers. Many computers are connected together to form a wired network. Serial ports are used to connect one computer to another.

Each computer:

- is assigned a unique three-digit ID
- has three ports, each identified by an integer value
- is connected to between one and three other computers.

Data is sent as individual message strings.

Each string contains the destination ID (the ID of the computer that is to receive the message) followed by the data:

```
<DestinationID><Data>
```

Messages may pass through several computers on the way to their destination.

When a message arrives at a computer, that is **not** the destination, the program needs to forward it on to another computer using one of its serial ports.

The port to use is obtained from information that is stored in an array `RouteTable`.

`RouteTable` is a global 2D array of integers. It is declared in pseudocode as follows:

```
DECLARE RouteTable : ARRAY[1:6,1:3] OF INTEGER
```

The values in the first two columns of `RouteTable` define a range of ID values.

Column 3 gives the corresponding port number to use when forwarding the message to a computer with an ID within this range.

For example, the contents of `RouteTable` could be:

	Column 1	Column 2	Column 3
Row 1	100	199	1
Row 2	200	259	2
Row 3	-1	<undefined>	<undefined>
Row 4	260	399	2
Row 5	400	599	3
Row 6	600	999	1

In this example, a message that arrives with a `DestinationID` of "283" will be forwarded using port 2.

Row 3 in the example shows an unused row. These may occur anywhere. Unused rows have the column 1 element set to -1. The value of unused elements in the other two columns is undefined.

The programmer has defined the first program module as follows:

Module	Description
<code>GetPort()</code>	- takes a <code>DestinationID</code> as a parameter of type string - searches for the range corresponding to the <code>DestinationID</code> in the array - returns the port number, or returns -1 if no corresponding range is found

- (b) Copies of the same program will run on each computer. The program contains a global variable `MyID` of type string, which contains the unique ID of the computer in which the program is running.

When messages are received, they are placed on one of two stacks. Stack 1 is used for messages that have reached their destination and stack 2 is used for messages that will be forwarded on to another computer.

Additional modules are defined:

Module	Description
<code>StackMsg()</code> (already written)	• takes two parameters: ◦ a string representing a message ◦ an integer representing the stack number • adds the message to the required stack • returns <code>TRUE</code> if the message is added to the required stack, otherwise returns <code>FALSE</code>
<code>ProcessMsg()</code>	• takes a message as a parameter of type string • ignores any message with a zero-length data field • extract the <code>DestinationID</code> from the message • checks whether the <code>DestinationID</code> is this computer or whether the message is to be forwarded • uses <code>StackMsg()</code> to add the message to the appropriate stack • outputs an error if the message could not be added to the stack

Write pseudocode for module `ProcessMsg()`.

Module `StackMsg()` must be used.

56. O/N 23/P23/Q1/b,c

- 1 A program is being developed in pseudocode before being converted into a programming language.

(b) Other variables in the program have example values as shown:

Variable	Value
Active	TRUE
Fraction	0.2
Code	"Ab12345"

Complete the table by evaluating each expression using the example values.

Expression	Evaluates to
Fraction >= 0.2 AND NOT Active	
INT((Fraction * 100) + 13.3)	
STR_TO_NUM(MID(Code, 4, 2)) + 5	
LENGTH("TRUE" & Code)	

[4]

- (c) The program makes use of complex statistical functions. The required functions are not built-in to the programming language and are too complicated for the programmer to write.

One solution would be to employ another programmer who has experience of writing these functions, as there is no time to train the existing programmer.

State **one other** way that these functions may be provided for inclusion in the program.

[1]

57. O/N 23/P23/Q4/a,b

- 4 A procedure `RandList()` will output a sequence of 25 random integers, where each integer is larger than the previous one.

(a) Write pseudocode for procedure `RandList()`.

[6]

- (b) Procedure `RandList()` is modified so that the random numbers are also written into a 1D array `Result`.

A new module is written to confirm that the numbers in the array are in ascending order.

This module contains an `IF` statement that performs a comparison between elements:

```
IF (Result[x + 1] = Result[x]) OR (Result[x] > Result[x + 1]) THEN
    Sequence ← FALSE
ENDIF
```

Write a simplified version of the conditional clause.

[1]

58. O/N 23/P23/Q6

- 6 A function `TestNum()` will take a six-digit string as a parameter.

The function will test whether the string meets certain conditions and will return an integer value as follows:

Return value	Condition	Example
1	The last three digits are the same but non-zero.	"253444"
2	The last three digits are zero.	"253000"
3	The first three and last three digits are the same.	"410410"

The function will return the highest possible value for the given string.

If the string does not meet any of the conditions, zero is returned.

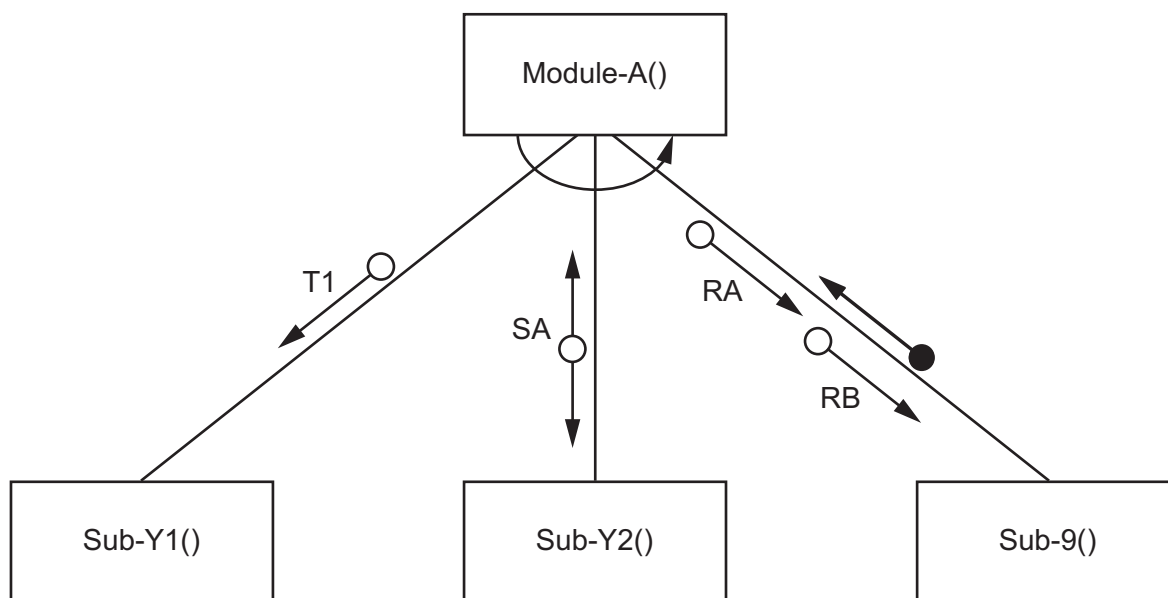
Write pseudocode for function `TestNum()`.

Assume that the parameter is valid.

[6]

59. O/N 23/P23/Q7/b(ii)

7 A structure chart shows the modular structure of a program:



(b) The structure chart shows that Sub-9() is a function.

A Boolean value is returned by Sub-9() for processing by Module-A().

The original parameter RA is of type integer and RB is of type string.

A record type `MyType` will be defined with three fields to store the values passed between the two modules.

(ii) The design is modified and Sub-9() is changed to a procedure.

The procedure will be called with a single parameter of type `MyType`.

Write the pseudocode header for procedure `Sub-9()`.

[2]

60. O/N 23/P23/Q8/c

8 A class of students are developing a program to send data between computers. Many computers are connected together to form a wired network. Serial ports are used to connect one computer to another.

Each computer:

- is assigned a unique three-digit ID
- has three ports, each identified by an integer value
- is connected to between one and three other computers.

Messages are sent between computers as a string of characters organised into fields as shown:

```
<STX><DestinationID><SourceID><Data><ETX>
```

Field name	Description
STX	a single character marking the start of the message (ASCII value 02)
DestinationID	three numeric characters identifying the destination computer
SourceID	three numeric characters identifying the source computer
Data	a variable length string containing the data being sent (Minimum length is 1 character)
ETX	a single character marking the end of the message (ASCII value 03)

For example, the following message contains the data "Hello Jack" being sent from computer "202" to computer "454":

```
<STX>"454202Hello Jack"<ETX>
```

Each computer will run a copy of the same program. Each program will contain a global variable `MyID` of type string which contains the unique ID of the computer in which the program is running.

The first two program modules are defined as follows:

Module	Description
<code>GetData()</code> (already written)	- returns the data field from a message that has been received - If no message is available, the module waits until one has been received.
<code>ReceiveFile()</code>	- takes a file name as a parameter of type string - creates a text file with the given file name (no checking required) - writes the data field returned by <code>GetData()</code> to the file - repeats until the data field is "*****", which is not written to the file - outputs a final message giving the total number of characters written to the file, for example: 132456 characters were written to newfile.txt

(c) Two new modules are defined, which will allow two users to exchange messages.

Module	Description
<code>Transmit()</code> (already written)	- takes two parameters: - a string representing a message - an integer representing a port number - transmits the message using the given port
<code>Chat()</code>	- takes two parameters: - a string representing a Destination ID - an integer representing a port number - extracts data from a received message using <code>GetData()</code> and outputs it - forms a message using data input by the user and sends it using <code>Transmit()</code> - repeats until either the output string or the sent string is "Bye"

Reminders:

- Each program contains a global variable `MyID` of type string which contains the unique ID of the computer in which the program is running.
- Messages are sent between computers as a string of characters organised into fields as shown:

```
<STX><DestinationID><SourceID><Data><ETX>
```

Write pseudocode for module `Chat()`.

Modules `GetData()` and `Transmit()` must be used.

61. M/J 23/P21/Q1/b,c

- 1 A programmer has written a program which includes the function `Calculate()`. When the program is run, the function returns an unexpected value.

(b) The algorithm for function `Calculate()` contains the three pseudocode statements shown.

Describe the error in each statement or write 'no error' if the statement contains no error.

Assume any variables used are of the correct type for the given function.

Statement 1: `Index ← STR_TO_NUM("27") + 2)`

Error

Statement 2: `Index ← STR_TO_NUM(MID("CPE1704TKS", 4, 2))`

Error

Statement 3: `IF MONTH(ThisDate) > '6' THEN`

Error

[3]

(c) The program contains variables with values as follows:

Variable	Value
Active	TRUE
Points	75
Exempt	FALSE

(i) Complete the table by evaluating each expression.

	Expression	Evaluation
1	<code>(Points > 99) OR Active</code>	
2	<code>(Points MOD 2 = 0) OR Exempt</code>	
3	<code>(Points <= 75) AND (Active OR Exempt)</code>	
4	<code>(Active OR NOT Active) AND NOT Exempt</code>	

[2]

(ii) Write expression 4 from the table in part (c)(i) in its simplest form.

[1]

62. M/J 23/P21/Q2/b

- 2 A program contains an algorithm to output a string of a specified length containing identical characters.
- (b) A different part of the program uses the variable `StartDate`.

Write pseudocode statements to declare `StartDate` **and** assign to it the date corresponding to 15/11/2005.

Declaration

Assignment

[3]

63. M/J 23/P21/Q4

- 4 Function `Replace()` will:
1. take three parameters:
 - a string (the original string)
 - a char (the original character)
 - a char (the new character)
 2. form a new string from the original string where all instances of the original character are replaced by the new character
 3. return the new string.

Write pseudocode for function `Replace()`.

[6]

64. M/J 23/P21/Q8/a

- 8 A computer shop assembles computers using items bought from several suppliers. A text file `Stock.txt` contains information about each item.

Information for each item is stored as a single line in the `Stock.txt` file in the format:

`<ItemNum><SupplierCode><Description>`

Item information is as follows:

	Format	Comment
ItemNum	4 numeric characters	unique for each item in the range "0001" to "5999" inclusive
SupplierCode	5 alphabetic characters	to identify the supplier of the item
Description	a string	a minimum of 12 characters

The file is organised in ascending order of `ItemNum` and does **not** contain all possible values in the range.

A programmer has started to define program modules as follows:

Module	Description
<code>SuppExists()</code> (already written)	- called with a parameter of type string representing a supplier code - returns <code>TRUE</code> if the supplier code is already in use, otherwise returns <code>FALSE</code>
<code>IsNewSupp()</code>	- called with a parameter of type string representing a new supplier code - returns <code>TRUE</code> if the string only contains alphabetic characters (either upper or lower case) and the supplier code is not already in use, otherwise returns <code>FALSE</code>

- (a) Write pseudocode for module `IsNewSupp()`.

Module `SuppExists()` has already been written and should be used as part of your solution. Module `SuppExists()` will generate a run-time error if the given parameter is not 5 characters in length.

65. M/J 23/P22/Q1/a

- 1 A program calculates the postal cost based on the weight of the item and its destination. Calculations occur at various points in the program and these result in the choice of several possible postal costs. The programmer has built these postal costs into the program.

For example, the postal cost of \$3.75 is used in the following lines of pseudocode:

```
IF Weight < 250 AND ValidAddress = TRUE THEN
    ItemPostalCost ← 3.75      // set postal cost for item to $3.75
    ItemStatus ← "Valid"      // item can be sent
ENDIF
```

- (a) (i) Identify a more appropriate way of representing the postal costs.

[1]

- (ii) Describe the advantages of your answer to **part (a)(i)** with reference to this program.

[3]

66. M/J 23/P22/Q2/a,b

- 2 A program stores a user's date of birth using a variable `MyDOB` of type `DATE`.

- (a) Write a pseudocode statement, using a function from the **insert**, to assign the value corresponding to 17/11/2007 to `MyDOB`.

[1]

- (b) `MyDOB` has been assigned a valid value representing the user's date of birth.

Write a pseudocode statement to calculate the number of months from the month of the user's birth until the end of the year and to assign this to the variable `NumMonths`.

For example, if `MyDOB` contains a value representing 02/07/2008, the value 5 would be assigned to `NumMonths`.

[2]

Insert of this paper (its text differs from the Appendix copy):

An error will be generated if a function call is not properly formed or if the parameters are of an incorrect type or an incorrect value.

String and Character Functions

- A string of length 1 may be either of type `CHAR` or `STRING`
- A `CHAR` may be assigned to, or concatenated with, a `STRING`
- A `STRING` of length greater than 1 cannot be assigned to a `CHAR`

<code>LEFT(ThisString : STRING, x : INTEGER)</code> RETURNS STRING returns leftmost x characters from <code>ThisString</code> Example: <code>LEFT("ABCDEFGH", 3)</code> returns "ABC"
<code>RIGHT(ThisString : STRING, x : INTEGER)</code> RETURNS STRING returns rightmost x characters from <code>ThisString</code> Example: <code>RIGHT("ABCDEFGH", 3)</code> returns "FGH"
<code>MID(ThisString : STRING, x : INTEGER, y : INTEGER)</code> RETURNS STRING returns a string of length y starting at position x from <code>ThisString</code> Example: <code>MID("ABCDEFGH", 2, 3)</code> returns string "BCD"
<code>LENGTH(ThisString : STRING)</code> RETURNS INTEGER returns the integer value representing the length of <code>ThisString</code> Example: <code>LENGTH("Happy Days")</code> returns 10
<code>TO_UPPER(x : <datatype>)</code> RETURNS <datatype> <datatype> may be CHAR or STRING returns an object of type <datatype> formed by converting all characters of x to upper case. Examples: <code>TO_UPPER("Error 803")</code> returns "ERROR 803" <code>TO_UPPER('a')</code> returns 'A'
<code>TO_LOWER(x : <datatype>)</code> RETURNS <datatype> <datatype> may be CHAR or STRING returns an object of type <datatype> formed by converting all characters of x to lower case. Examples: <code>TO_LOWER("JIM 803")</code> returns "jim 803" <code>TO_LOWER('W')</code> returns 'w'
<code>NUM_TO_STR(x : <datatype1>)</code> RETURNS <datatype2> returns a string representation of a numeric value. <datatype1> may be REAL or INTEGER, <datatype2> may be CHAR or STRING Example: <code>NUM_TO_STR(87.5)</code> returns "87.5"
<code>STR_TO_NUM(x : <datatype1>)</code> RETURNS <datatype2> returns a numeric representation of a string. <datatype1> may be CHAR or STRING, <datatype2> may be REAL or INTEGER Example: <code>STR_TO_NUM("23.45")</code> returns 23.45
<code>IS_NUM(ThisString : <datatype>)</code> RETURNS BOOLEAN returns TRUE if <code>ThisString</code> represents a valid numeric value. <datatype> may be CHAR or STRING Example: <code>IS_NUM("-12.36")</code> returns TRUE

ASC(ThisChar : CHAR) RETURNS INTEGER

returns an integer value (the ASCII value) of character ThisChar

Example: ASC('A') returns 65, ASC('B') returns 66, etc.

CHR(x : INTEGER) RETURNS CHAR

returns the character whose integer value (the ASCII value) is x

Example: CHR(65) returns 'A', CHR(66) returns 'B', etc.

Numeric Functions

INT(x : REAL) RETURNS INTEGER

returns the integer part of x

Example: INT(27.5415) returns 27

RAND(x : INTEGER) RETURNS REAL

returns a real number in the range 0 to x (**not** inclusive of x).

Example: RAND(87) may return 35.43

Date Functions

Date format is assumed to be DD/MM/YYYY unless otherwise stated.

DAY(ThisDate : DATE) RETURNS INTEGER

returns the current day number from ThisDate

Example: DAY(04/10/2003) returns 4

MONTH(ThisDate : DATE) RETURNS INTEGER

returns the current month number from ThisDate

Example: MONTH(04/10/2003) returns 10

YEAR(ThisDate : DATE) RETURNS INTEGER

returns the current year number from ThisDate

Example: YEAR(04/10/2003) returns 2003

DAYINDEX(ThisDate : DATE) RETURNS INTEGER

returns the day index number from ThisDate where Sunday = 1, Monday = 2 etc.

Example: DAYINDEX(09/05/2023) returns 3

SETDATE(Day, Month, Year : INTEGER) RETURNS DATE

returns a value of type DATE with the value of <Day>/<Month>/<Year>

Example: SETDATE(26, 10, 2003) returns a date corresponding to 26/10/2003

TODAY() RETURNS DATE

returns a value of type DATE corresponding to the current date.

Text File Functions

EOF(FileName : STRING) RETURNS BOOLEAN

returns TRUE if there are no more lines to be read from file FileName

The function will generate an error if the file is not already open in READ mode.

Operators

An error will be generated if an operator is used with a value or values of an incorrect type.

&	concatenates (joins) two strings. Example: "Summer" & " " & "Pudding" evaluates to "Summer Pudding" The operator may also be used to concatenate a CHAR with a STRING
AND	performs a logical AND on two Boolean values. Example: TRUE AND FALSE evaluates to FALSE
OR	performs a logical OR on two Boolean values. Example: TRUE OR FALSE evaluates to TRUE
NOT	performs a logical NOT on a Boolean value. Example: NOT TRUE evaluates to FALSE
MOD	finds the remainder when one number is divided by another. Example: 10 MOD 3 evaluates to 1
DIV	finds the quotient when one number is divided by another. Example 10 DIV 3 evaluates to 3

Comparison Operators

=	used to compare two items of the same type. evaluates to TRUE if the condition is true, otherwise evaluates to FALSE
>	Notes:
<	- may be used to compare types REAL and INTEGER - may be used to compare types CHAR and STRING - case sensitive when used to compare types CHAR or STRING - cannot be used to compare two records
>=	
<=	Examples:
<>	"Program" = "program" evaluates to FALSE - Count = 4 evaluates to TRUE when variable Count contains the value 4

67. M/J 23/P22/Q4

4 A function `GetNum()` will:

- take two parameters: a string and a character
- count the number of times that the character occurs in the string
- return the count.

Any comparison between characters needs to be case sensitive. For example, character 'a' and character 'A' are not identical.

Write pseudocode for function `GetNum()`.

68. M/J 23/P22/Q6

- 6 A procedure `Square()` will take an integer value in the range 1 to 9 as a parameter and output a number square.

The boundary of a number square is made up of the character representing the parameter value. The inside of the number square is made up of the asterisk character (*).

Parameter value	1	2	3	4	...	9
Output	1	22 22	333 3*3 333	4444 4**4 4**4 4444	...	999999999 9*****9 9*****9 9*****9 9*****9 9*****9 9*****9 9*****9 9*****9 999999999

The pseudocode `OUTPUT` command starts each output on a new line. For example, the following three `OUTPUT` statements would result in the outputs as shown:

```
OUTPUT "Hello"
OUTPUT "ginger"
OUTPUT "cat"
```

Resulting output:

```
Hello
ginger
cat
```

Write pseudocode for procedure `Square()`.

Parameter validation is not required.

69. M/J 23/P22/Q8/a

- 8 A computer shop assembles computers using items bought from several suppliers. A text file `Stock.txt` contains information about each item.

Information for each item is stored as a single line in the `Stock.txt` file in the format:

`<ItemNum><SupplierCode><Description>`

Valid item information is as follows:

	Format	Comment
ItemNum	4 numeric characters	unique number for each item in the range "0001" to "5999" inclusive
SupplierCode	3 alphabetic characters	to identify the supplier of the item
Description	a string	a minimum of 12 characters

The file is organised in ascending order of `ItemNum` and does **not** contain all possible values in the range.

A programmer has started to define program modules as follows:

Module	Description
<code>OnlyAlpha()</code> (already written)	- called with a parameter of type string - returns <code>TRUE</code> if the string contains only alphabetic characters, otherwise returns <code>FALSE</code>
<code>CheckInfo()</code>	- called with a parameter of type string representing a line of item information - checks to see whether the item information in the string is valid - returns <code>TRUE</code> if the item information is valid, otherwise returns <code>FALSE</code>

- (a) Write pseudocode for module `CheckInfo()`.

Module `OnlyAlpha()` should be used as part of your solution.

[7]

70. M/J 23/P23/Q2/a

- 2 A program stores a date of birth for a student using a variable, `MyDOB`, of type `DATE`.

- (a) `MyDOB` has been assigned a valid value corresponding to Kevin's date of birth.

Complete the pseudocode statement to test whether Kevin was born on a Thursday.

IF

THEN [2]

71. M/J 23/P23/Q4

4 A function `MakeString()` will:

1. take two parameters:
 - a count as an integer
 - a character
2. generate a string of length equal to the count, made up of the character
3. return the string generated, or return "ERROR" if the count is less than 1.

For example, the function call:

`MakeString(3, 'Z')` will return the string "ZZZ"

Write pseudocode for function `MakeString()`.

[6]

72. M/J 23/P23/Q6/a,b

6 A procedure `Select()` will:

1. take two integer values as parameters representing start and end values where both values are greater than 9 and the end value is greater than the start value
2. output **each** integer value between the start and the end value (**not** including the start and end values), where the sum of the last two digits is 6, for example, 142.

(a) Write pseudocode for procedure `Select()`.

Parameter validation is **not** required.

[7]

(b) The check performed by procedure `Select()` on the last two digits is needed at several places in the program and will be implemented using a new function.

The new function `CheckNum()` will:

- allow the required sum to be specified (not just 6)
- check one number
- return an appropriate value.

Describe the function interface **and two** advantages of this modular approach.

Interface

Advantage 1

Advantage 2

[4]

73. O/N 22/P21/Q1/a,c

- 1 (a) An algorithm includes a number of complex calculations. A programmer is writing a program to implement the algorithm and decides to use library routines to provide part of the solution.

State **three** possible benefits of using library routines in the development of the program.

1

2

3

[3]

- (c) The pseudocode statements in the following table may contain errors.

State the error in each case or write 'NO ERROR' if the statement contains no error.

Assume that any variables used are of the correct type for the given function.

Statement	Error
IF EMPTY ← "" THEN	
Status ← IS_NUM(-23.4)	
X ← STR_TO_NUM("37") + 5	
Y ← STR_TO_NUM("37" + "5")	

[4]

74. O/N 22/P21/Q6/a

- 6** The following pseudocode algorithm attempts to check whether a string is a valid email address.

```

FUNCTION IsValid(InString : STRING) RETURNS BOOLEAN
  DECLARE Index, Dots, Ats, Others : INTEGER
  DECLARE NextChar : CHAR
  DECLARE Valid : BOOLEAN

  Index ← 1
  Dots ← 0
  Ats ← 0
  Others ← 0
  Valid ← TRUE

  REPEAT
    NextChar ← MID(InString, Index, 1)
    CASE OF NextChar
      '.' : Dots ← Dots + 1
      '@' : Ats ← Ats + 1
            IF Ats > 1 THEN
              Valid ← FALSE
            ENDIF
      OTHERWISE : Others ← Others + 1
    ENDCASE

    IF Dots > 1 AND Ats = 0 THEN
      Valid ← FALSE
    ELSE
      Index ← Index + 1
    ENDIF

  UNTIL Index > LENGTH(InString) OR Valid = FALSE

  IF NOT (Dots >= 1 AND Ats = 1 AND Others > 8) THEN
    Valid ← FALSE
  ENDIF

  RETURN Valid

ENDFUNCTION

```

- (a)** Part of the validation is implemented by the line:

```
IF NOT (Dots >= 1 AND Ats = 1 AND Others > 8) THEN
```

State the values that would result in the condition evaluating to TRUE.

75. O/N 22/P21/Q7/a

7 A simple arithmetic expression is stored as a string in the format:

`<Value1><Operator><Value2>`

An operator character is one of the following: `'+'` `'-'` `'*'` `'/'`

Example arithmetic expression strings:

`"803+1904"`

`"34/7"`

(a) A procedure `Calculate()` will:

- take an arithmetic expression string as a parameter
- evaluate the expression
- output the result.

Assume:

- the string contains only numeric digits and a single operator character
- `Value1` and `Value2` represent integer values
- `Value1` and `Value2` are unsigned (they will not be preceded by `'+'` or `'-'`).

(i) Write pseudocode for the procedure `Calculate()`.

[7]

(ii) `Calculate()` is changed to a function that returns the value of the evaluated expression.

Write the header for the function in pseudocode.

[1]

76. O/N 22/P21/Q8/b

- 8 A teacher is designing a program to perform simple syntax checks on programs written by students. Student programs are submitted as text files, which are known as project files.

A project file may contain blank lines.

The teacher has defined the first program module as follows:

Module	Description
<code>CheckFile()</code>	• takes the name of an existing project file as a parameter of type string • returns <code>TRUE</code> if the file is valid (it contains at least 10 non-blank lines), otherwise returns <code>FALSE</code>

- (a) Write pseudocode for module `CheckFile()`.

[7]

Further modules are defined as follows:

Module	Description
<code>CheckLine()</code>	• takes a line from a project file as a parameter of type string • returns zero if the line is blank or contains no syntax error, otherwise returns an error number as an integer
<code>CountErrors()</code>	• takes two parameters: ○ the name of a project file as a string ○ the maximum number of errors as an integer • uses <code>CheckFile()</code> to test the project file. Outputs an error message and ends if the project file is not valid • calls <code>CheckLine()</code> for each line in the project file • counts the number of errors • outputs the number of errors or a warning message if the maximum number of errors is exceeded

- (b) `CountErrors()` is called to check the project file `Jim01Prog.txt` and to stop if more than 20 errors are found.

Write the pseudocode statement for this call.

[2]

77. O/N 22/P22/Q1/a,c

- 1 (a) A programmer is developing an algorithm to solve a problem. Part of the algorithm would be appropriate to implement as a subroutine (a procedure or a function).

(i) State **two** reasons why the programmer may decide to use a subroutine.

1

2

[2]

(ii) A procedure header is shown in pseudocode:

```
PROCEDURE MyProc(Count : INTEGER, Message : STRING)
```

Give the correct term for the identifiers **Count** and **Message** **and** explain their use.

Term

Use

[2]

(c) Part of an identifier table is shown:

Variable	Type	Example value
FlagDay	DATE	23/04/2004
CharList	STRING	"ABCDEF"
Count	INTEGER	29

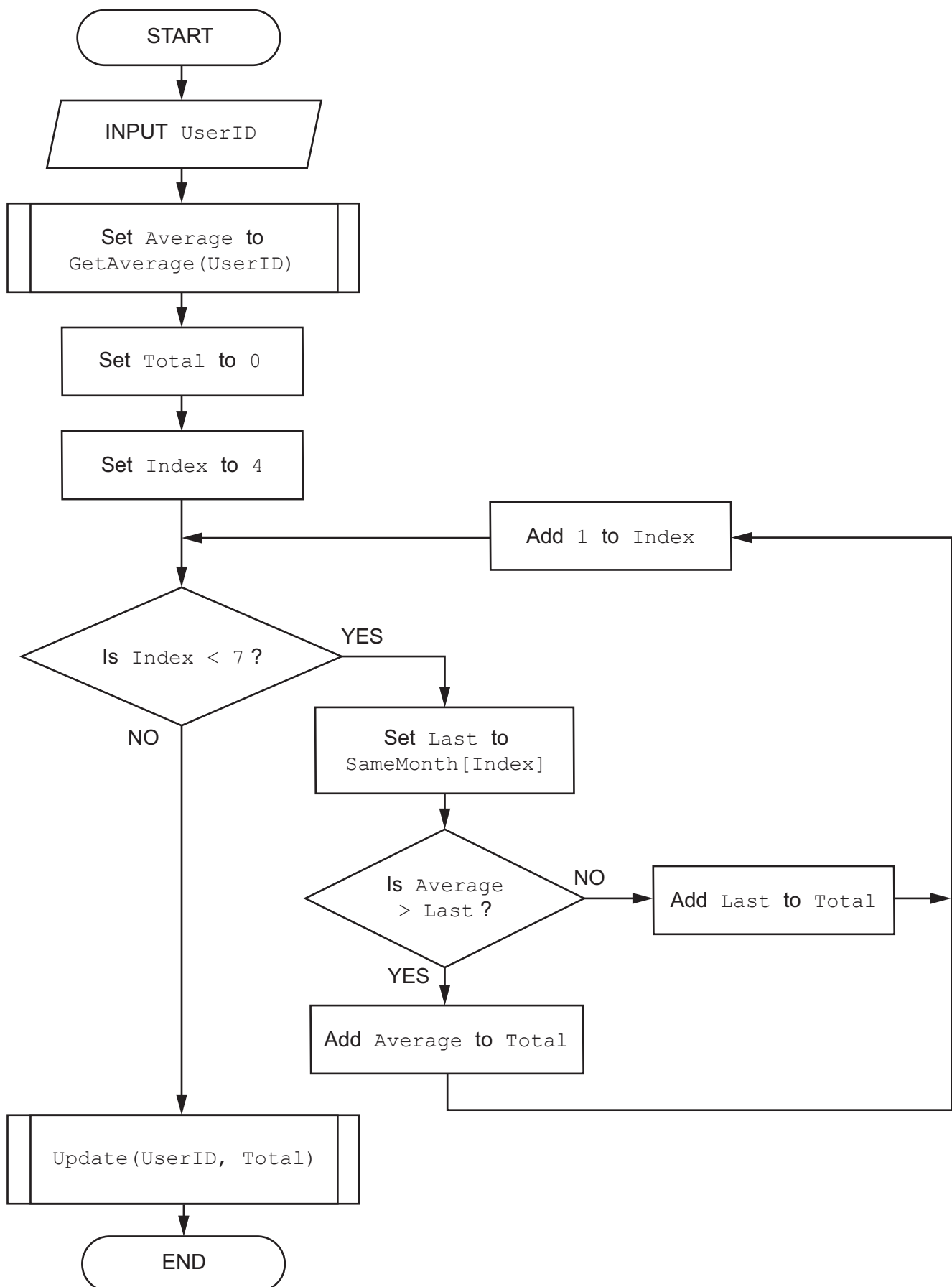
Complete the table by evaluating each expression using the example values.

Expression	Evaluation
MID(CharList, MONTH(FlagDay), 1)	
INT(Count / LENGTH(CharList))	
(Count >= 29) AND (DAY(FlagDay) > 23)	

[3]

78. O/N 22/P22/Q4/b

- 4 The program flowchart represents a simple algorithm.



(b) Give the name of the iterative construct in the flowchart.

79. O/N 22/P22/Q6/a

- 6 (a)** The factorial of an integer number is the product of all the integers from that number down to 1.

In general, the factorial of n is $n \times (n-1) \times \dots \times 2 \times 1$

For example, the factorial of 5 is $5 \times 4 \times 3 \times 2 \times 1 = 120$

In this question, n will be referred to as the `BaseNumber`.

A function `FindBaseNumber()` will:

- be called with a positive, non-zero integer value as a parameter
- return `BaseNumber` if the parameter value is the factorial of the `BaseNumber`
- return `-1` if the parameter value **is not** a factorial.

For example:

Parameter value	Value returned
120	5
12	-1
6	3
1	1

`FindBaseNumber(12)` will return `-1` because 12 is not a factorial.

You may use the rest of this page for rough working.

Write pseudocode for the function `FindBaseNumber()`.

80. O/N 22/P23/Q5/a,b(ii)

- 5 (a) A text string contains three data items concatenated as shown:

<StockID><Description><Cost>

Item lengths are:

Item	Length
StockID	5
Description	32
Cost	the remainder of the string

A procedure `Unpack()` takes four parameters of type string. One parameter is the original text string. The other three parameters are used to represent the three data items shown in the table and are assigned values within the procedure. These values will be used by the calling program after the procedure ends.

- (i) Write pseudocode for the procedure `Unpack()`.

[6]

- (ii) Explain the term **procedure interface** with reference to procedure `Unpack()`.

[2]

- (b) The design changes and a record structure is defined to store the three data items.

A user-defined data type `StockItem` is created as shown:

```
TYPE StockItem
  DECLARE StockID : STRING
  DECLARE Description : STRING
  DECLARE Cost : REAL
ENDTYPE
```

- (ii) Procedure `Unpack()` is modified and converted to a function which takes the original text string as the only parameter.

Explain the other changes that need to be made to convert the procedure into a function.

[2]

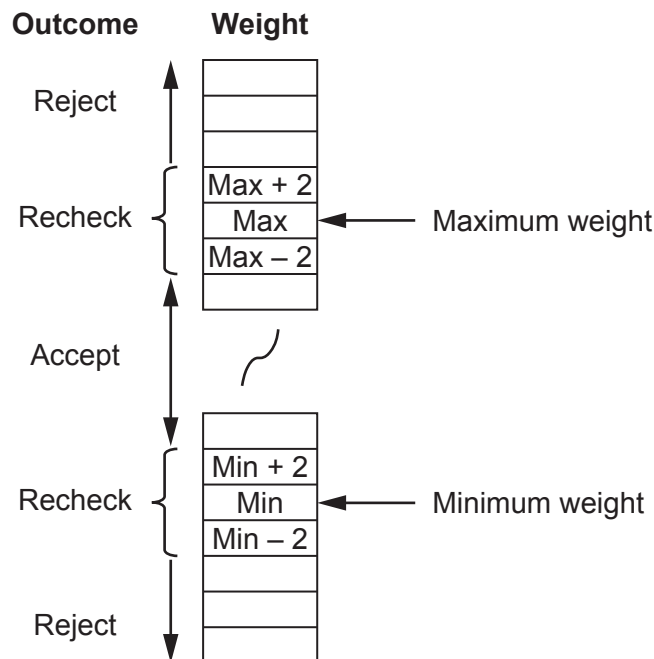
81. O/N 22/P23/Q6/b

- 6 Components are weighed during manufacture. Weights are measured to the nearest whole gram.

Components that weigh at least 3 grams more than the maximum weight, or at least 3 grams less than the minimum weight, are rejected.

A component is rechecked if it weighs within 2 grams of either the maximum or minimum weight.

The final outcome of weighing each component is shown below:



A function `Status()` will be called with three parameters. These are integers representing the weight of an individual component together with the minimum and maximum weights.

The value returned from the function will be as follows:

Outcome	Return value
Accept	'A'
Reject	'R'
Recheck	'C'

- (b) Write pseudocode for `Status()`.

[6]

82. O/N 21/P21/Q6/c

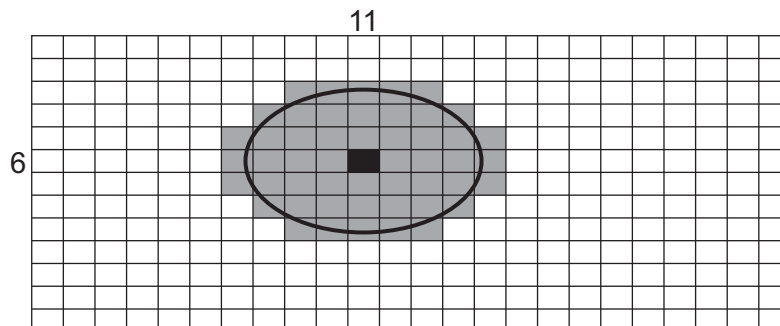
- 6 A mobile phone has a touchscreen. The screen is represented by a grid, divided into 800 rows and 1280 columns.

The grid is represented by a 2D array `Screen` of type `INTEGER`. An array element will be set to 0 unless the user touches that part of the screen.

Many array elements are set to 1 by a single touch of a finger or a stylus.

The following diagram shows a simplified touchscreen. The dark line represents a touch to the screen. All grid elements that are wholly or partly inside the outline will be set to 1. These elements are shaded.

The element shaded in black represents the centre point.



A program is needed to find the coordinates (the row and column) of the centre point. The centre point on the diagram above is row 6, column 11.

Assume:

- the user may only touch one area at a time
- screen rotation does not affect the touchscreen.

The programmer has started to define program modules as follows:

Module	Description
SetRow() (generates test data)	• Called with three parameters of type <code>INTEGER</code>: ◦ a row number ◦ the number of pixels to be skipped starting from column 1 ◦ the number of pixels that should be set to 1 • Sets the required number of pixels to 1 For example, <code>SetRow(3, 8, 5)</code> will give row 3 as in the diagram shown.
SearchInRow()	• Takes two parameters of type <code>INTEGER</code>: ◦ a row number ◦ a start column (1 or 1280) • Searches the given row from the start column (either left to right or right to left) for the first column that contains an element set to 1 • Returns the column number of the first element in the given row that is set to 1 • Returns -1 if no element is set to 1
SearchInCol()	• Takes two parameters of type <code>INTEGER</code>: ◦ a column number ◦ a start row (1 or 800) • Searches the given column from the start row (either up or down) for the first row that contains an element set to 1 • Returns the row number of the first element in the given column that is set to 1 • Returns -1 if no element is set to 1

(c) The following new module is introduced:

Module	Description
GetCentreCol()	- Called with a row number as an <code>INTEGER</code> - Uses <code>SearchInRow()</code> to find the first and last columns in the given row which have an array element set to 1 - Returns the index of the column midway between the first and last columns - Returns -1 if no element is set to 1

Write pseudocode to implement the module `GetCentreCol()`.

[6]

83. O/N 21/P22/Q1/c,d

(c) Complete the pseudocode expressions so that they evaluate to the values shown.

Any functions and operators used must be defined in the **insert**.

Expression	Evaluates to
('C')	67
2 * ("27")	54
(27 /)	13
"Sub" & ("Abstraction" , ,)	"Subtract"

[4]

(d) Evaluate the expressions given in the following table. The variables have been assigned values as follows:

```
PumpOn ← TRUE
PressureOK ← TRUE
HiFlow ← FALSE
```

Expression	Evaluates to
PressureOK AND HiFlow	
PumpOn OR PressureOK	
NOT PumpOn OR (PressureOK AND NOT HiFlow)	
NOT (PumpOn OR PressureOK) AND NOT HiFlow	

[2]

Insert of this paper (its text differs from the Appendix copy):

Note: An error occurs if a function call is not properly formed, or if the parameters are incorrect.

STRING Functions

`LEFT(ThisString : STRING, x : INTEGER)` RETURNS STRING
returns leftmost x characters from ThisString

Example: `LEFT("ABCDEFGH", 3)` returns "ABC"

`RIGHT(ThisString: STRING, x : INTEGER)` RETURNS STRING
returns rightmost x characters from ThisString

Example: `RIGHT("ABCDEFGH", 3)` returns "FGH"

`MID(ThisString : STRING, x : INTEGER, y : INTEGER)` RETURNS STRING
returns a string of length y starting at position x from ThisString

Example: `MID("ABCDEFGH", 2, 3)` returns "BCD"

`LENGTH(ThisString : STRING)` RETURNS INTEGER
returns the integer value representing the length of ThisString

Example: `LENGTH("Happy Days")` returns 10

`LCASE(ThisChar : CHAR)` RETURNS CHAR
returns the character value representing the lower case equivalent of ThisChar
Alphabetic characters that are not upper case are unchanged.

Example: `LCASE('W')` returns 'w'

`UCASE(ThisChar : CHAR)` RETURNS CHAR
returns the character value representing the upper case equivalent of ThisChar
Alphabetic characters that are not lower case are unchanged.

Example: `UCASE('a')` returns 'A'

`TO_UPPER(ThisString : STRING)` RETURNS STRING
returns a string formed by converting all characters of ThisString to upper case.

Example: `TO_UPPER("Error 803")` returns "ERROR 803"

`TO_LOWER(ThisString : STRING)` RETURNS STRING
returns a string formed by converting all characters of ThisString to lower case.

Example: `TO_LOWER("JIM 803")` returns "jim 803"

`NUM_TO_STR(x : <data type1>) RETURNS <data type2>`
returns a string representation of a numeric value.

Note: <data type1> may be `REAL` or `INTEGER`
<data type2> may be `CHAR` or `STRING`

Example: `NUM_TO_STR(87.5)` returns `"87.5"`

`STR_TO_NUM(x : <data type1>) RETURNS <data type2>`
returns a numeric representation of a string.

Note: <data type1> may be `CHAR` or `STRING`
<data type2> may be `REAL` or `INTEGER`

Example: `STR_TO_NUM("23.45")` returns `23.45`

`IS_NUM(ThisString : STRING) RETURNS BOOLEAN`
returns the value `TRUE` if `ThisString` represents a valid numeric value.

Example 1: `IS_NUM("12.36")` returns `TRUE`

Example 2: `IS_NUM("-12.36")` returns `TRUE`

Example 3: `IS_NUM("12.3a")` returns `FALSE`

`ASC(ThisChar : CHAR) RETURNS INTEGER`
returns an integer value (the ASCII value) of `ThisChar`

Example: `ASC('A')` returns `65`

`CHR(x : INTEGER) RETURNS CHAR`
returns the character whose integer value (the ASCII value) is `x`

Example: `CHR(87)` returns `'W'`

NUMERIC Functions

`INT(x : REAL) RETURNS INTEGER`
returns the integer part of `x`

Example: `INT(27.5415)` returns `27`

`RAND(x : INTEGER) RETURNS REAL`
returns a real number in the range 0 to `x` (**not** inclusive of `x`).

Example: `RAND(87)` could return `35.43`

OTHER Functions

EOF(FileName : STRING) RETURNS BOOLEAN
returns TRUE if there are no more lines to be read from file FileName

Note: The function will generate an error if the file is not already open in READ mode.

Note: An error occurs if an operator with a value of an incorrect type is used.

OPERATORS

&	Concatenates (joins) two strings Example: "Summer" & " " & "Pudding" evaluates to "Summer Pudding"
AND	Performs a logical AND on two Boolean values Example: TRUE AND FALSE evaluates to FALSE
OR	Performs a logical OR on two Boolean values Example: TRUE OR FALSE evaluates to TRUE
NOT	Performs a logical NOT on a Boolean value Example: NOT TRUE evaluates to FALSE
MOD	Finds the remainder when one number is divided by another Example: 10 MOD 3 evaluates to 1
DIV	Finds the quotient when one number is divided by another Example: 10 DIV 3 evaluates to 3

84. O/N 21/P22/Q4/a,c

4 The following is a procedure design in pseudocode.

Line numbers are given for reference only.

```

10  PROCEDURE Check(InString : STRING)
11      DECLARE Odds, Evens, Index : INTEGER
12
13      Odds ← 0
14      Evens ← 0
15      Index ← 1
16
17      WHILE Index <= LENGTH(InString)
18          IF STR_TO_NUM(MID(InString, Index, 1)) MOD 2 <> 0 THEN
19              Odds ← Odds + 1
20          ELSE
21              Evens ← Evens + 1
22          ENDIF
23          Index ← Index + 1
24      ENDWHILE
25
26      CALL Result(Odds, Evens)
27  ENDPROCEDURE

```

(a) Complete the following table by giving the answers, using the given pseudocode.

Answer

A line number containing a variable being incremented	
The type of loop structure	
The number of functions used	
The number of parameters passed to STR_TO_NUM()	
The name of a procedure other than Check()	

[5]

(c) (i) The loop structure used in the pseudocode is not the most appropriate.

State a more appropriate loop structure **and** justify your choice.

Loop structure

Justification

[2]

(ii) The appropriate loop structure is now used. Two lines of pseudocode are changed and two lines are removed.

Write the line numbers of the two lines that are removed.

[1]

85. O/N 21/P22/Q6/b,c(ii),d

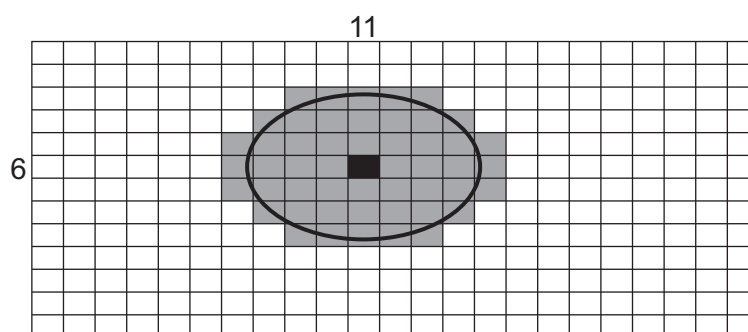
- 6** A mobile phone has a touchscreen. The screen is represented by a grid, divided into 800 rows and 1280 columns.

The grid is represented by a 2D array `Screen` of type `INTEGER`. An array element will be set to 0 unless the user touches that part of the screen.

Many array elements will set to 1 by a single touch of a finger or a stylus.

The following diagram shows a simplified touchscreen. The dark line represents a touch on the screen. All grid elements that are wholly or partly inside the outline will be set to 1. These elements are shaded.

The element shaded in black represents the centre point of the touch.



A program is needed to find the coordinates (the row and column) of the centre point. The centre point on the diagram shown is row 6, column 11.

Assume:

- the user may only touch one area at a time
- screen rotation does not affect the touchscreen.

The programmer has decided to use global values `CentreRow` and `CentreCol` as coordinate values for the centre point.

The programmer has started to define program modules as follows:

Module	Description
<code>FirstRowSet()</code>	- Searches for the first row that has an array element set to 1 - Returns the index of that row (1 is the first row) - Returns -1 if there are no elements set to 1
<code>LastRowSet()</code>	- Searches for the last row that has an array element set to 1 - Returns the index of that row - Returns -1 if there are no elements set to 1
<code>FirstColSet()</code>	- Searches for the first column that has an array element set to 1 - Returns the index of that column (1 is the first column) - Returns -1 if there are no elements set to 1
<code>LastColSet()</code>	- Searches for the last column that has an array element set to 1 - Returns the index of that column - Returns -1 if there are no elements set to 1

- (a)** Write efficient pseudocode for the module `FirstRowSet()`.

- (b) Describe a feature of your solution to **part (a)** that indicates the pseudocode represents an efficient algorithm.

[2]

- (c) The programmer decides to produce a **single** search module `FindSet()`, which will be able to perform each of the individual searches performed by the first four modules in the table.

- (ii) Give one possible advantage **and** one possible disadvantage of combining the four searches into a single module.

Advantage

Disadvantage

[2]

- (d) An additional module `GetCentre()` is defined as follows:

Module	Description
<code>GetCentre()</code>	- Calculates the coordinates of the centre point - Uses the four modules: <code>FirstRowSet()</code>, <code>LastRowSet()</code>, <code>FirstColSet()</code>, <code>LastColSet()</code> - Assigns values to <code>CentreRow</code> and <code>CentreCol</code> - Sets <code>CentreRow</code> to <code>-1</code> if no screen touch is detected

Write pseudocode for the module `GetCentre()`.

[6]

86. O/N 21/P23/Q6/c

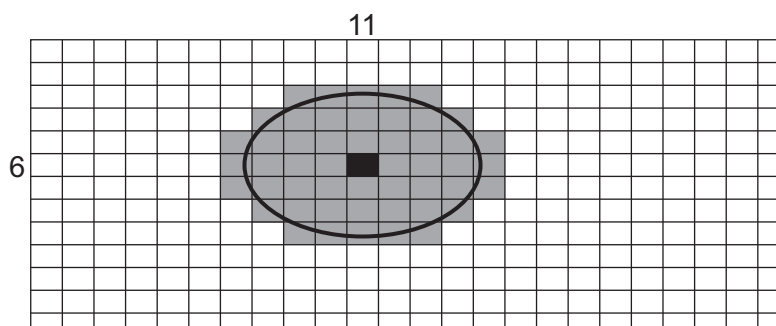
- 6 A mobile phone has a touchscreen. The screen is represented by a grid, divided into 800 rows and 1280 columns.

The grid is represented by a 2D array `Screen` of type `INTEGER`. An array element will be set to 0 unless the user touches that part of the screen.

Many array elements are set to 1 by a single touch of a finger or a stylus.

The following diagram shows a simplified touchscreen. The dark line represents a touch to the screen. All grid elements that are wholly or partly inside the outline will be set to 1. These elements are shaded.

The element shaded in black represents the centre point.



A program is needed to find the coordinates (the row and column) of the centre point. The centre point on the diagram above is row 6, column 11.

Assume:

- the user may only touch one area at a time
- screen rotation does not affect the touchscreen.

The programmer has started to define program modules as follows:

Module	Description
<code>SetRow()</code> (generates test data)	• Called with three parameters of type <code>INTEGER</code>: ◦ a row number ◦ the number of pixels to be skipped starting from column 1 ◦ the number of pixels that should be set to 1 • Sets the required number of pixels to 1 For example, <code>SetRow(3, 8, 5)</code> will give row 3 as in the diagram shown.
<code>SearchInRow()</code>	• Takes two parameters of type <code>INTEGER</code>: ◦ a row number ◦ a start column (1 or 1280) • Searches the given row from the start column (either left to right or right to left) for the first column that contains an element set to 1 • Returns the column number of the first element in the given row that is set to 1 • Returns -1 if no element is set to 1
<code>SearchInCol()</code>	• Takes two parameters of type <code>INTEGER</code>: ◦ a column number ◦ a start row (1 or 800) • Searches the given column from the start row (either up or down) for the first row that contains an element set to 1 • Returns the row number of the first element in the given column that is set to 1 • Returns -1 if no element is set to 1

(c) The following new module is introduced:

Module	Description
<code>GetCentreCol()</code>	• Called with a row number as an <code>INTEGER</code> • Uses <code>SearchInRow()</code> to find the first and last columns in the given row which have an array element set to 1 • Returns the index of the column midway between the first and last columns • Returns -1 if no element is set to 1

Write pseudocode to implement the module `GetCentreCol()`.

87. M/J 21/P21/Q1/b

- (b) Each pseudocode statement in the following table may contain an error due to the incorrect use of the function or operator.

Describe the error in each case, or write 'NO ERROR' if the statement contains no error.

You can assume that none of the variables referenced are of an incorrect type.

Statement	Error
Result $\leftarrow$ 2 & 4	
SubString $\leftarrow$ MID("pseudocode", 4, 1)	
IF x = 3 OR 4 THEN	
Result $\leftarrow$ Status AND INT(x/2)	
Message $\leftarrow$ "Done" + LENGTH(MyString)	

[5]

88. M/J 21/P21/Q2/b

- 2 (a) Four program modules form part of a program for a library.

A description of the relationship between the modules is summarised as follows:

Module name	Description
UpdateLoan()	• Calls either LoanExtend() or LoanReturn()
LoanExtend()	• Called with parameters LoanID and BookID • Calls CheckReserve() to see whether the book has been reserved for another library user • Returns TRUE if the loan has been extended, otherwise returns FALSE
CheckReserve()	• Called with BookID • Returns TRUE if the book has been reserved, otherwise returns FALSE
LoanReturn()	• Called with parameters LoanID and BookID • Returns a REAL (which is the value of the fine to be paid in the case of an overdue loan)

Draw a structure chart to show the relationship between the four modules and the parameters passed between them.

[5]

(b) The definition for module `LoanReturn()` is amended as follows:

Module name	Description
<code>LoanReturn()</code>	Called with parameters <code>LoanID</code> , <code>BookID</code> and <code>Fine</code> The module code checks whether the book has been returned on time and then assigns a new value to <code>Fine</code>

- `LoanID` and `BookID` are of type `STRING`
- `Fine` is of type `REAL`

Write the pseudocode header for the **amended** module `LoanReturn()`.

[2]

89. M/J 21/P21/Q4/b

- 4 Study the following pseudocode. Line numbers are for reference only.

```

10 FUNCTION Convert(Name : STRING) RETURNS STRING
11
12   DECLARE Flag: BOOLEAN
13   DECLARE Index : INTEGER
14   DECLARE ThisChar : CHAR
15   DECLARE NewName : STRING
16
17   CONSTANT SPACECHAR = ' '
18
19   Flag ← TRUE
20   Index ← 1
21   NewName ← ""           // formatted name string
22
23   WHILE Index <= LENGTH(Name)
24     ThisChar ← MID(Name, Index, 1)
25     IF Flag = TRUE THEN
26       NewName ← NewName & UCASE(ThisChar)
27       IF ThisChar <> SPACECHAR THEN
28         Flag ← FALSE
29       ENDIF
30     ELSE
31       NewName ← NewName & ThisChar
32     ENDIF
33     IF ThisChar = SPACECHAR THEN
34       Flag ← TRUE
35     ENDIF
36     Index ← Index + 1
37   ENDWHILE
38
39   RETURN NewName
40
41 ENDFUNCTION

```

(b) The pseudocode for `Convert()` contains a conditional loop.

State a more appropriate loop structure.

Justify your answer.

Loop structure

Justification

[2]

Insert of this paper (its text differs from the Appendix copy):

Note: An error occurs if a function call is not properly formed, or if the parameters are incorrect.

STRING Functions

`LEFT(ThisString : STRING, x : INTEGER) RETURNS STRING`
 returns leftmost x characters from `ThisString`

Example: `LEFT("ABCDEFGH", 3)` returns "ABC"

`RIGHT(ThisString : STRING, x : INTEGER) RETURNS STRING`
returns rightmost `x` characters from `ThisString`

Example: `RIGHT("ABCDEFGH", 3)` returns "FGH"

`MID(ThisString : STRING, x : INTEGER, y : INTEGER) RETURNS STRING`
returns a string of length `y` starting at position `x` from `ThisString`

Example: `MID("ABCDEFGH", 2, 3)` returns "BCD"

`LENGTH(ThisString : STRING) RETURNS INTEGER`
returns the integer value representing the length of `ThisString`

Example: `LENGTH("Happy Days")` returns 10

`LCASE(ThisChar : CHAR) RETURNS CHAR`
returns the character value representing the lower case equivalent of `ThisChar`
Characters that are not upper case alphabetic are returned unchanged

Example: `LCASE('W')` returns 'w'

`UCASE(ThisChar : CHAR) RETURNS CHAR`
returns the character value representing the upper case equivalent of `ThisChar`
Characters that are not lower case alphabetic are returned unchanged

Example: `UCASE('a')` returns 'A'

`TO_UPPER(ThisString : STRING) RETURNS STRING`
returns a string formed by converting all characters of `ThisString` to upper case

Example: `TO_UPPER("Error 803")` returns "ERROR 803"

`TO_LOWER(ThisString : STRING) RETURNS STRING`
returns a string formed by converting all characters of `ThisString` to lower case

Example: `TO_LOWER("JIM 803")` returns "jim 803"

`NUM_TO_STR(x : <data type>) RETURNS STRING`
returns a string representation of a numeric value
Note: <data type> may be REAL or INTEGER

Example: `NUM_TO_STR(87.5)` returns "87.5"

90. M/J 21/P21/Q7/a,c

- 7 A program is needed to take a string containing a full name and produce a new string of initials.

Some words in the full name will be ignored. For example, "the", "and", "of", "for" and "to" may all be ignored.

Each letter of the abbreviated string must be upper case.

For example:

Full name	Initials
Integrated Development Environment	IDE
The American Standard Code for Information Interchange	ASCII

The programmer has decided to use a global variable `FNString` of type `STRING` to store the full name.

It is assumed that:

- words in the full name string are separated by a single space character
- space characters will not occur at the beginning or the end of the full name string
- the full name string contains at least one word.

The programmer has started to define program modules as follows:

Module	Description
<code>GetStart()</code>	• Called with an <code>INTEGER</code> as a parameter, representing the number of a word in <code>FNString</code>. • Returns the character start position of that word in <code>FNString</code> or returns <code>-1</code> if that word does not exist • For example: if <code>FNString</code> contains the string "hot and cold", <code>GetStart(3)</code> returns 9
<code>GetWord()</code>	• Called with a parameter representing the position of the first character of a word in <code>FNString</code> • Returns the word from <code>FNString</code> • For example: if <code>FNString</code> contains the string "hot and cold", <code>GetWord(9)</code> returns "cold"

(a) Write pseudocode for the module `GetStart()`.

[7]

(c) As a reminder, the module description of `GetWord()` is repeated:

Module	Description
<code>GetWord()</code>	• Called with a parameter representing the position of the first character of a word in <code>FNString</code> • Returns the word from <code>FNString</code> • For example: if <code>FNString</code> contains the string "hot and cold", <code>GetWord(9)</code> returns "cold"

Write pseudocode for the module `GetWord()`.

[5]

91. M/J 21/P22/Q5/b

(b) The following is a pseudocode function.

Line numbers are given for reference only.

```

01  FUNCTION StringClean(InString : STRING) RETURNS STRING
02
03      DECLARE NextChar : CHAR
04      DECLARE OutString : STRING
05      DECLARE Counter : INTEGER
06
07      OutString ← ""
08
09      FOR Counter ← 1 TO LENGTH(InString)
10          NextChar ← MID(InString, Counter, 1)
11          NextChar ← LCASE(NextChar)
12          IF NOT((NextChar < 'a') OR (NextChar > 'z')) THEN
13              OutString ← OutString & NextChar
14          ENDIF
15      NEXT Counter
16
17      RETURN OutString
18
19  ENDFUNCTION

```

(i) Examine the pseudocode and complete the following table.

Answer

Give a line number containing an example of an initialisation statement.	
Give a line number containing the start of a repeating block of code.	
Give a line number containing a logic operation.	
Give the number of parameters to the function MID().	

[4]

(ii) Write a simplified version of the statement in line 12.

[2]

Insert of this paper (its text differs from the Appendix copy):

Note: An error occurs if a function call is not properly formed, or if the parameters are incorrect.

STRING Functions

LEFT(ThisString : STRING, x : INTEGER) RETURNS STRING
returns leftmost x characters from ThisString

Example: LEFT("ABCDEFGH", 3) returns "ABC"

RIGHT(ThisString : STRING, x : INTEGER) RETURNS STRING
returns rightmost x characters from ThisString

Example: RIGHT("ABCDEFGH", 3) returns "FGH"

`MID(ThisString : STRING, x : INTEGER, y : INTEGER)` RETURNS STRING
returns a string of length `y` starting at position `x` from `ThisString`

Example: `MID("ABCDEFGH", 2, 3)` returns "BCD"

`LENGTH(ThisString : STRING)` RETURNS INTEGER
returns the integer value representing the length of `ThisString`

Example: `LENGTH("Happy Days")` returns 10

`LCASE(ThisChar : CHAR)` RETURNS CHAR
returns the character value representing the lower case equivalent of `ThisChar`
Characters that are not upper case alphabetic are returned unchanged

Example: `LCASE('W')` returns 'w'

`UCASE(ThisChar : CHAR)` RETURNS CHAR
returns the character value representing the upper case equivalent of `ThisChar`
Characters that are not lower case alphabetic are returned unchanged

Example: `UCASE('a')` returns 'A'

`TO_UPPER(ThisString : STRING)` RETURNS STRING
returns a string formed by converting all characters of `ThisString` to upper case

Example: `TO_UPPER("Error 803")` returns "ERROR 803"

`TO_LOWER(ThisString : STRING)` RETURNS STRING
returns a string formed by converting all characters of `ThisString` to lower case

Example: `TO_LOWER("JIM 803")` returns "jim 803"

`NUM_TO_STR(x : <data type>)` RETURNS STRING
returns a string representation of a numeric value
Note: <data type> may be REAL or INTEGER

Example: `NUM_TO_STR(87.5)` returns "87.5"

92. M/J 21/P22/Q8/b

- 8 A program is needed to take a string containing a full name and to produce a new string of initials.

Some words in the full name will be ignored. For example, “the”, “and”, “of”, “for” and “to” may all be ignored.

Each letter of the new string must be upper case.

For example:

Full name	Initials
Integrated Development Environment	IDE
The American Standard Code for Information Interchange	ASCII

The programmer has decided to use the following global variables:

- a ten element 1D array `IgnoreList` of type `STRING` to store the ignored words
- a string `FNString` to store the full name string.

Assume that:

- each alphabetic character in the full name string may be either upper or lower case
- the full name string contains at least one word.

The programmer has started to define program modules as follows:

Module	Description
<code>GetStart()</code>	• Called with an <code>INTEGER</code> as its parameter, representing the number of a word in <code>FNString</code> • Returns the character start position of that word in <code>FNString</code> or returns <code>-1</code> if that word does not exist • For example: <code>GetStart(3)</code> applied to "hot and cold" returns 9
<code>GetWord()</code>	• Called with the position of the first character of a word in <code>FNString</code> as its parameter • Returns the word from <code>FNString</code> • For example: if <code>FNString</code> contains the string "hot and cold", <code>GetWord(9)</code> returns "cold"
<code>IgnoreWord()</code>	• Called with a <code>STRING</code> parameter representing a word • Searches for the word in the <code>IgnoreList</code> array • Returns <code>TRUE</code> if the word is found, otherwise returns <code>FALSE</code>
<code>GetInitials()</code>	• Processes the sequence of words in the full name one word at a time • Calls <code>GetStart()</code>, <code>GetWord()</code> and <code>IgnoreWord()</code> to process each word to form the new string • Outputs the new string

- (b) Write pseudocode for the module `GetInitials()`.

93. M/J 21/P23/Q1/b

- (b) Each pseudocode statement in the following table may contain an error due to the incorrect use of the function or operator.

Describe the error in each case, or write 'NO ERROR' if the statement contains no error.

You can assume that none of the variables referenced are of an incorrect type.

Statement	Error
Result $\leftarrow$ 2 & 4	
SubString $\leftarrow$ MID("pseudocode", 4, 1)	
IF x = 3 OR 4 THEN	
Result $\leftarrow$ Status AND INT(x/2)	
Message $\leftarrow$ "Done" + LENGTH(MyString)	

[5]

94. M/J 21/P23/Q2/b

- 2 (a) Four program modules form part of a program for a library.

A description of the relationship between the modules is summarised as follows:

Module name	Description
UpdateLoan()	• Calls either LoanExtend() or LoanReturn()
LoanExtend()	• Called with parameters LoanID and BookID • Calls CheckReserve() to see whether the book has been reserved for another library user • Returns TRUE if the loan has been extended, otherwise returns FALSE
CheckReserve()	• Called with BookID • Returns TRUE if the book has been reserved, otherwise returns FALSE
LoanReturn()	• Called with parameters LoanID and BookID • Returns a REAL (which is the value of the fine to be paid in the case of an overdue loan)

Draw a structure chart to show the relationship between the four modules and the parameters passed between them.

[5]

(b) The definition for module `LoanReturn()` is amended as follows:

Module name	Description
<code>LoanReturn()</code>	Called with parameters <code>LoanID</code> , <code>BookID</code> and <code>Fine</code> The module code checks whether the book has been returned on time and then assigns a new value to <code>Fine</code>

- `LoanID` and `BookID` are of type `STRING`
- `Fine` is of type `REAL`

Write the pseudocode header for the **amended** module `LoanReturn()`.

[2]

95. M/J 21/P23/Q4/b

- 4 Study the following pseudocode. Line numbers are for reference only.

```

10 FUNCTION Convert(Name : STRING) RETURNS STRING
11
12   DECLARE Flag: BOOLEAN
13   DECLARE Index : INTEGER
14   DECLARE ThisChar : CHAR
15   DECLARE NewName : STRING
16
17   CONSTANT SPACECHAR = ' '
18
19   Flag ← TRUE
20   Index ← 1
21   NewName ← ""           // formatted name string
22
23   WHILE Index <= LENGTH(Name)
24     ThisChar ← MID(Name, Index, 1)
25     IF Flag = TRUE THEN
26       NewName ← NewName & UCASE(ThisChar)
27       IF ThisChar <> SPACECHAR THEN
28         Flag ← FALSE
29       ENDIF
30     ELSE
31       NewName ← NewName & ThisChar
32     ENDIF
33     IF ThisChar = SPACECHAR THEN
34       Flag ← TRUE
35     ENDIF
36     Index ← Index + 1
37   ENDWHILE
38
39   RETURN NewName
40
41 ENDFUNCTION

```

(b) The pseudocode for `Convert()` contains a conditional loop.

State a more appropriate loop structure.

Justify your answer.

Loop structure

Justification

[2]

Insert of this paper (its text differs from the Appendix copy):

Note: An error occurs if a function call is not properly formed, or if the parameters are incorrect.

STRING Functions

`LEFT(ThisString : STRING, x : INTEGER) RETURNS STRING`
 returns leftmost x characters from `ThisString`

Example: `LEFT("ABCDEFGH", 3)` returns "ABC"

`RIGHT(ThisString : STRING, x : INTEGER) RETURNS STRING`
returns rightmost `x` characters from `ThisString`

Example: `RIGHT("ABCDEFGH", 3)` returns "FGH"

`MID(ThisString : STRING, x : INTEGER, y : INTEGER) RETURNS STRING`
returns a string of length `y` starting at position `x` from `ThisString`

Example: `MID("ABCDEFGH", 2, 3)` returns "BCD"

`LENGTH(ThisString : STRING) RETURNS INTEGER`
returns the integer value representing the length of `ThisString`

Example: `LENGTH("Happy Days")` returns 10

`LCASE(ThisChar : CHAR) RETURNS CHAR`
returns the character value representing the lower case equivalent of `ThisChar`
Characters that are not upper case alphabetic are returned unchanged

Example: `LCASE('W')` returns 'w'

`UCASE(ThisChar : CHAR) RETURNS CHAR`
returns the character value representing the upper case equivalent of `ThisChar`
Characters that are not lower case alphabetic are returned unchanged

Example: `UCASE('a')` returns 'A'

`TO_UPPER(ThisString : STRING) RETURNS STRING`
returns a string formed by converting all characters of `ThisString` to upper case

Example: `TO_UPPER("Error 803")` returns "ERROR 803"

`TO_LOWER(ThisString : STRING) RETURNS STRING`
returns a string formed by converting all characters of `ThisString` to lower case

Example: `TO_LOWER("JIM 803")` returns "jim 803"

`NUM_TO_STR(x : <data type>) RETURNS STRING`
returns a string representation of a numeric value
Note: <data type> may be REAL or INTEGER

Example: `NUM_TO_STR(87.5)` returns "87.5"

96. M/J 21/P23/Q7/a,c

- 7 A program is needed to take a string containing a full name and produce a new string of initials.

Some words in the full name will be ignored. For example, "the", "and", "of", "for" and "to" may all be ignored.

Each letter of the abbreviated string must be upper case.

For example:

Full name	Initials
Integrated Development Environment	IDE
The American Standard Code for Information Interchange	ASCII

The programmer has decided to use a global variable `FNString` of type `STRING` to store the full name.

It is assumed that:

- words in the full name string are separated by a single space character
- space characters will not occur at the beginning or the end of the full name string
- the full name string contains at least one word.

The programmer has started to define program modules as follows:

Module	Description
<code>GetStart()</code>	• Called with an <code>INTEGER</code> as a parameter, representing the number of a word in <code>FNString</code>. • Returns the character start position of that word in <code>FNString</code> or returns <code>-1</code> if that word does not exist • For example: if <code>FNString</code> contains the string "hot and cold", <code>GetStart(3)</code> returns 9
<code>GetWord()</code>	• Called with a parameter representing the position of the first character of a word in <code>FNString</code> • Returns the word from <code>FNString</code> • For example: if <code>FNString</code> contains the string "hot and cold", <code>GetWord(9)</code> returns "cold"

(a) Write pseudocode for the module `GetStart()`.

[7]

(c) As a reminder, the module description of `GetWord()` is repeated:

Module	Description
<code>GetWord()</code>	• Called with a parameter representing the position of the first character of a word in <code>FNString</code> • Returns the word from <code>FNString</code> • For example: if <code>FNString</code> contains the string "hot and cold", <code>GetWord(9)</code> returns "cold"

Write pseudocode for the module `GetWord()`.

[5]

Answers Section

1. M/J 26/P21/Q1/a,b

1(a)	<u>Count-control(led) loop // Count-control(led) iteration</u>	1
1(b)	4	1

2. M/J 26/P21/Q5/a,b

5(a)	<i>Example solution:</i> <pre> FUNCTION BulbCost(Quantity : INTEGER) RETURNS REAL DECLARE Cost : REAL DECLARE Twenty : INTEGER DECLARE Other : INTEGER Twenty ← Quantity DIV 20 Other ← Quantity MOD 20 Cost ← (Twenty * 30) + (Other * 1.75) RETURN Cost ENDFUNCTION </pre> <i>Mark as follows:</i> MP1 Function header and single parameter and returns REAL and end MP2 Declaration of required local variable(s) MP3 Correct calculation of 'multiples of 20' and 'other' amounts using MOD and DIV // alternative method MP4 Cost is returned	4
5(b)	TotalCost ← BulbCost (NumberPurchased)	1

3. M/J 26/P21/Q6/a,b

6(a)	MP1 LEFT(DataPacket, 3) // MID(DataPacket, 1, 3) MP2 MID(DataPacket, 5, 1)	2
------	--	----------

6(b)	<i>Example solution:</i> <pre> FUNCTION RejectDescription(ThisReject : STRING) RETURNS STRING DECLARE Description : STRING Description ← <u>MID(ThisReject, 7,</u> <u>LENGTH(ThisReject) - 6)</u> Part 1 Part 2 OR: Description ← RIGHT(ThisReject, LENGTH(ThisReject) - 6) Part 1 Part 2 RETURN Description ENDFUNCTION </pre> <i>Mark as follows:</i> MP1 Function header and end function (and string parameter) MP2 Calculation for description expression (<i>part 1</i>) using MID/RIGHT MP3 Calculation for description expression (<i>part 2</i>) using the LENGTH function MP4 Return of a calculated value and RETURN STRING in the header	4
------	--	----------

4. M/J 26/P22/Q1/a(ii,iii)

1(a)(ii)	<pre> IF LENGTH(Pass) < 6 OR LENGTH(Pass) > 8 / >= 9 THEN </pre> Mark as follows: 1 Mark: Use of LENGTH function twice 1 Mark: Number 8 used in second condition	2	
----------	--	----------	--

1(a)(iii)	Mark as follows: 1 DECLARE LetterFlag, DigitFlag: <u>BOOLEAN</u> 1 mark for BOOLEAN 2 LetterFlag ← <u>FALSE</u> 1 mark for FALSE 3 FOR Index ← 1 TO <u>LENGTH</u>(Pass) 1 mark for LENGTH 4 Character ← <u>MID</u>(Pass, <u>Index</u>, <u>1</u>) 1 mark for MID, Index and 1 5 IF <u>IS_NUM</u>(Character) = TRUE THEN 1 mark for IS_NUM 6 DigitFlag ← <u>TRUE</u> ELSE LetterFlag ← <u>TRUE</u> 1 mark for both variables being set to TRUE 7 IF <u>NOT</u>(LetterFlag AND DigitFlag) THEN 1 mark for NOT	7	
-----------	---	---	--

5. M/J 26/P22/Q5/c

5(c)(i)	1 mark: The parameters / Num1 and Num2 are passed by value / BYVAL // The parameters / Num1 and Num2 are not passed / should be passed by reference / BYREF 1 mark: (The swap made inside) the procedure does not change the arguments used /parameters (passed) / the values stored in the Numbers elements passed // The procedure does not swap / change the values stored in the array	2	
5(c)(ii)	Solution <pre>PROCEDURE Swap (BYREF Num1, Num2 : INTEGER) DECLARE Temp : INTEGER Temp ← Num1 Num1 ← Num2 Num2 ← Temp ENDPROCEDURE</pre> 2 marks: code as above 1 mark: just the procedure header as above	2	NE for header mark If ByRef is mentioned in the header but syntax is not correct but allow FT for second mark

6. M/J 26/P22/Q6/a

6(a)	Mark Scheme One – loop used to form return string (See MS Two for solutions using LEFT () /MID () to form return string) Example solution <pre> FUNCTION AddPadding() RETURNS STRING CONSTANT Space = ' ' DECLARE Plaintext : STRING DECLARE LenOfPlaintext : INTEGER DECLARE Index : INTEGER REPEAT OUTPUT "Enter plaintext" INPUT Plaintext LenOfPlaintext ← LENGTH(Plaintext) UNTIL LenOfPlaintext >= 1 AND LenOfPlaintext <= 25 FOR Index ← LenOfPlaintext + 1 TO 25 Plaintext ← Plaintext & Space NEXT Index // Alternative way to append spaces using while loop WHILE LENGTH(Plaintext) <> 25 Plaintext ← Plaintext & Space ENDWHILE //Alternative way to append spaces DECLARE SpaceString : STRING SpaceString ← "" FOR Index ← LenOfPlaintext + 1 TO 25 SpaceString ← SpaceString & Space NEXT Index Plaintext ← Plaintext & SpaceString RETURN Plaintext ENDFUNCTION </pre> Mark points 1 Function heading, return type and ending 2 Prompt for and input of plaintext 3 Conditional loop that checks one of the boundary values 4 Conditional loop that correctly checks both boundaries of plaintext 5 ...Re-input of plaintext in a loop 6 Loop that iterates for number of spaces to add to end of plaintext 7 ...Spaces appended to plaintext // Spaces appended to an empty string 8 Correctly padded plaintext returned following a reasonable attempt at MP7	8	MP 4 Must be a conditional loop If no loop used then MP3, MP4 and MP5 can not be given
------	---	---	---

6(a)	Mark Scheme Two - No Loop used to form return string – use of LEFT () /MID () Function Example Solution: <pre> FUNCTION AddPadding() RETURNS STRING CONSTANT Fill = " " //24 Spaces DECLARE Plaintext : STRING DECLARE LenOfPlaintext : INTEGER DECLARE Index : INTEGER REPEAT OUTPUT "Enter plaintext" INPUT Plaintext LenOfPlaintext ← LENGTH(Plaintext) UNTIL LenOfPlaintext >= 1 AND LenOfPlaintext <= 25 Plaintext ← LEFT(Plaintext & Fill, 25) RETURN Plaintext ENDFUNCTION </pre> Mark points 1 Function heading, return type and ending 2 Prompt for and input of plaintext 3 Conditional loop that checks one of the boundary values Conditional loop that correctly checks for length of plaintext 4 ...Re-input of plaintext in a loop 5 Use of string of at least 24 spaces Correct use of LEFT () / MID () function and & to append padding characters 6 Correctly padded plaintext returned following a reasonable attempt at MP 6 and MP 7		Do not accept use of language specific string manipulation which is not the compatible with the pseudocode used for the syllabus
------	---	--	---

7. M/J 26/P23/Q1/a,b

1(a)	<u>Post-condition(al) loop // post-condition(al) iteration</u>	1	
1(b)	5	1	

8. M/J 26/P23/Q5/a

5(a)	<pre> FUNCTION InterviewSummary(ThisApplicant : STRING) RETURNS STRING DECLARE Summary : STRING Summary ← MID(ThisApplicant, 8, <u>MP2</u> LENGTH(ThisApplicant) - 7) <u>MP3</u> Or: Summary ← RIGHT(ThisApplicant, LENGTH(ThisApplicant) - 7) <u>MP2</u> <u>MP3</u> RETURN Summary ENDFUNCTION </pre> Mark as follows: MP1 Function identifier and function ending and parameter MP2 Calculation for summary expression (part 1) – using MID or RIGHT MP3 Calculation for summary expression (part 2) – using LENGTH MP4 Return of the calculated STRING value (earlier declaration optional) and RETURNS STRING in the header –	4	Do not accept a final missing close bracket (for the MID or RIGHT function).
------	---	----------	---

9. O/N 25/P21/Q1/a

1(a)			4
	Expression		
	MyString	← 'X' & MyString	
	MyChar	← MID ("ABCD", 1 / 2 / 3 / 4 , 1)	
	MyString	← NUM_TO_STR (DAY / MONTH / YEAR / DAYINDEX (MyDOB))	
	MyInt	← INT (LENGTH (MyString) / 2)	
One mark per row			

10. O/N 25/P21/Q5/b

5(b)(i)	The loop continues after the (first instance) of the search value / string / matching element has been found Comparisons/tests continue to be made even if the search value / string / matching element has been found	1
---------	--	----------

5(b)(ii)	One mark per point Construct: A conditional loop Justification: The loop / search can be terminated as soon as the (first occurrence of) SearchString / required value / required string / matching element is found	2
----------	---	----------

11. O/N 25/P21/Q6/b

6(b)	Example solution – conversion of Number to a string <pre> FUNCTION Generate (Number : INTEGER) RETURNS INTEGER DECLARE Total, Index, CheckDigit : INTEGER DECLARE NumString : STRING Total ← 0 NumString ← NUM_TO_STR (Number) FOR Index ← 1 TO LENGTH (NumString) Total ← Total + STR_TO_NUM (MID (NumString, Index, 1)) NEXT Index CheckDigit ← Total MOD 10 Number ← Number * 10 + CheckDigit // NumString ← NumString & NUM_TO_STR (CheckDigit) RETURN Number // STR_TO_NUM (NumString) ENDFUNCTION </pre> Mark as follows: 1 Function heading, parameters return type and ending 2 Initialise Total 3 Convert Number to a string and loop for length of converted Number 4 attempt to form sum of digits in a loop 5 completely correct sum of digits in a loop 6 Calculate CheckDigit 7 Add CheckDigit to original number multiplied by 10 and return number // Convert CheckDigit to a character and appended to NumString and then convert to a number and return	7
------	---	----------

6(b)	Example Solution – no conversion to string <pre> FUNCTION Generate (Number : INTEGER) RETURNS INTEGER DECLARE Total, CheckDigit, Remainder, Num: INTEGER Num ← Number Total ← 0 WHILE Num > 0 Remainder ← Num MOD 10 Num ← Num DIV 10 Total ← Total + Remainder ENDWHILE CheckDigit ← Total MOD 10 Number ← Number * 10 + CheckDigit RETURN Number ENDFUNCTION </pre> Mark as follows: 1 Function heading, parameters return type and ending 2 Initialise Total 3 Loop while Num > 0 4 attempt to use MOD and DIV to sum successive digits in Number 5 completely correct sum of digits 6 Calculate CheckDigit 7 Add CheckDigit to original number and return number	
------	--	--

12. O/N 25/P22/Q1/c

1(c)	<table><tr><th>Expression</th><th>Evaluates to</th></tr><tr><td>INT(Result) + 1 > 6</td><td>FALSE</td></tr><tr><td>NUM_TO_STR(LENGTH(MonthLetter))</td><td>"12"</td></tr><tr><td>NUM_TO_STR(Result + "3.2")</td><td>ERROR</td></tr><tr><td>MID(MonthLetter, MONTH(Birthday) - 2, 1)</td><td>"S"</td></tr></table> One mark per row	Expression	Evaluates to	INT(Result) + 1 > 6	FALSE	NUM_TO_STR(LENGTH(MonthLetter))	"12"	NUM_TO_STR(Result + "3.2")	ERROR	MID(MonthLetter, MONTH(Birthday) - 2, 1)	"S"	4
Expression	Evaluates to											
INT(Result) + 1 > 6	FALSE											
NUM_TO_STR(LENGTH(MonthLetter))	"12"											
NUM_TO_STR(Result + "3.2")	ERROR											
MID(MonthLetter, MONTH(Birthday) - 2, 1)	"S"											

1(b)	<table><tr><th>Variable</th><th>Example data value</th><th>Data type</th></tr><tr><td>Result</td><td>5.42</td><td>REAL</td></tr><tr><td>MonthLetter</td><td>"JFMAMJJASOND"</td><td>STRING</td></tr><tr><td>Birthday</td><td>15/11/2009</td><td>DATE</td></tr></table> One mark per row	Variable	Example data value	Data type	Result	5.42	REAL	MonthLetter	"JFMAMJJASOND"	STRING	Birthday	15/11/2009	DATE	3
Variable	Example data value	Data type												
Result	5.42	REAL												
MonthLetter	"JFMAMJJASOND"	STRING												
Birthday	15/11/2009	DATE												

13. O/N 25/P22/Q6

6	Example solution: <pre> FUNCTION Compare(String1, String2 : STRING, __ CaseMatters : BOOLEAN, Position : CHAR) RETURNS BOOLEAN DECLARE S1Length : INTEGER S1Length ← LENGTH(String1) IF LENGTH(String2) < S1Length THEN RETURN FALSE // String2 is shorter than String1 ENDIF IF CaseMatters = FALSE THEN String1 ← TO_UPPER(String1) String2 ← TO_UPPER(String2) ENDIF CASE Position 'e' : String2 ← RIGHT(String2, S1Length) 's' : String2 ← LEFT(String2, S1Length) ENDCASE IF String1 = String2 THEN RETURN TRUE ELSE RETURN FALSE ENDIF ENDFUNCTION </pre> Mark as follows: MP1 Function heading and parameters and ending and return type MP2 Calculate length of String1 MP3 if String2 is shorter than String1, return FALSE MP4 Compare (modified) strings MP5 Test CaseMatters - if FALSE and convert two string(s) to same case MP6 Test for the value of 's'/'e' and form modified string(s) in one or both cases MP7 Return appropriate BOOLEAN value in all cases	7
---	---	---

14. O/N 25/P23/Q1/a,b,c

1(a)	Answers include: 1 They are tried and tested // Free from errors 2 They are readily available // Speed up development time 3 They will be updated automatically when improvements are made // No maintenance of routine is needed (as automatically updated) 4 The algorithm will be compliant with the international standard One mark per point Max 3	3								
1(b)(i)	1 When a part of the algorithm performs a specific task 2 Part is repeated // performed in several places 3 Testing / debugging / maintenance is easier One mark for each point Max 2	2								
1(b)(ii)	<table><thead><tr><th>Term</th><th>Description</th></tr></thead><tbody><tr><td>Pass2</td><td>the name of the function</td></tr><tr><td>Count</td><td>The parameter / value passed to the function</td></tr><tr><td>BOOLEAN</td><td>The (data) type <u>returned</u> (by the function)</td></tr></tbody></table> One mark per description (excluding first row)	Term	Description	Pass2	the name of the function	Count	The parameter / value passed to the function	BOOLEAN	The (data) type <u>returned</u> (by the function)	2
Term	Description									
Pass2	the name of the function									
Count	The parameter / value passed to the function									
BOOLEAN	The (data) type <u>returned</u> (by the function)									
1(c)	<table><thead><tr><th>Expression</th><th>Evaluates to</th></tr></thead><tbody><tr><td>LENGTH(NUM_TO_STR(Multiplier))</td><td>3</td></tr><tr><td>MONTH (DoB) > 4</td><td>TRUE</td></tr><tr><td>15 + STR_TO_NUM(MID(AddressLine[1], 2, 1))</td><td>20</td></tr></tbody></table> One mark per row	Expression	Evaluates to	LENGTH(NUM_TO_STR(Multiplier))	3	MONTH (DoB) > 4	TRUE	15 + STR_TO_NUM(MID(AddressLine[1], 2, 1))	20	3
Expression	Evaluates to									
LENGTH(NUM_TO_STR(Multiplier))	3									
MONTH (DoB) > 4	TRUE									
15 + STR_TO_NUM(MID(AddressLine[1], 2, 1))	20									

15. O/N 25/P23/Q5/b

5(b)(i)	1 and 2	1
---------	---------	----------

5(b)(ii)	<pre> 1 TO 2 : Num[Index] ← Num[Index] + Index Index ← Index + CaseVar Count ← Count + CaseVar </pre> One mark per point: • use of variable <code>CaseVar</code> rather than literal values • completely correct clause	2
----------	---	----------

16. O/N 25/P23/Q6/b

6(b)	Note 2 different solutions with different mark scheme Example solution – conversion to string and using a loop <pre> FUNCTION CheckNumber(Number : INTEGER) RETURNS BOOLEAN DECLARE Total, Index, CheckDigit : INTEGER DECLARE NumString : STRING Total ← 0 NumString ← NUM_TO_STR(Number) CheckDigit ← STR_TO_NUM(RIGHT(NumString, 1)) // CheckDigit ← Number MOD 10 FOR Index ← 1 TO LENGTH(NumString) - 1 // 1 TO LENGTH(NUM_TO_STR(Number DIV 10)) Total ← Total + STR_TO_NUM(MID(NumString, Index, 1)) ENDFOR IF Total MOD 10 <> CheckDigit THEN RETURN FALSE ENDIF RETURN TRUE ENDFUNCTION </pre> Mark as follows: 1 Function Header including Identifier, parameters and return type 2 Use of <code>Num_TO_STR</code> to convert parameter // Use of <code>Num_TO_STR</code> to convert parameter excluding least significant digit to a string 3 Extract <code>CheckDigit</code> from <code>NumString</code> // Extract least significant digit from parameter 4 Loop for length of <code>NumString</code> - 1 following correct conversion to string 5 attempt to form sum of digits in a loop 6 completely correct sum of digits in a loop and of sum initialised before loop 7 Calculate check digit from sum 8 Compare sum with final digit of parameter and return appropriate Boolean value Max 7	7
------	---	----------

6(b)	Example solution – numeric version <pre> FUNCTION CheckNumber(Number : INTEGER) RETURNS BOOLEAN DECLARE Sum, CheckDigit, Remainder: INTEGER Remainder ← Number MOD 10 Number ← Number DIV 10 CheckDigit ← Remainder Sum ← 0 WHILE Number > 0 Remainder ← Number MOD 10 Number ← Number DIV 10 Total ← Total + Remainder ENDWHILE IF Total MOD 10 <> CheckDigit THEN RETURN FALSE ENDIF RETURN TRUE ENDFUNCTION </pre> 1 Function Header including Identifier, parameters and return type 2 Initialise Sum 3 Set CheckDigit to least significant digit in Number 4 Loop while Number > 0 5 attempt to use MOD and DIV to sum successive digits in Number in loop 6 completely correct sum of all digits in Number apart from least significant digit in loop 7 Calculate check digit from sum 8 Compare with CheckDigit and return appropriate Boolean value Max 7	
------	--	--

17. M/J 25/P21/Q4/a

4(a)	MP1 Count-controlled MP2 The number of iterations required is known	2
------	---	----------

18. M/J 25/P21/Q7/a

7(a)	Example solution: <pre> FUNCTION CustomerOrder(Number, Points : INTEGER) RETURNS INTEGER DECLARE NewPoints, NumberFree: INTEGER NewPoints ← Points + Number NumberFree ← 0 WHILE NewPoints >= 11 AND Number > 0 NumberFree ← NumberFree + 1 NewPoints ← NewPoints -11 Number ← Number - 1 END WHILE OUTPUT "Number of free coffees is: ", NumberFree RETURN NewPoints ENDFUNCTION </pre> MP1 Function header and ending and parameters and return type MP2 Number of coffees ordered added to points MP3 Correct calculation of one free coffee MP4 Attempted calculation of multiple free coffees and reduced <code>NewPoints</code> MP5 Output number of free drinks with suitable message MP6 Return of <code>NewPoints</code>	6
------	---	----------

19. M/J 25/P23/Q1/a,b

1(a)(i)	MP1 When a task/module is repeated / reused / called // performed in several places MP2 A specific task can be identified /coded as a module / subroutine / procedure / function. MP3 Reduces complexity of <u>program / code</u> // Program / code is simplified MP4 Module / subroutine / procedure / function already available. MP5 <u>Testing</u> / <u>debugging</u> / <u>maintenance</u> is easier Max 2	2
1(a)(ii)	MP1 Local variables are used within a module // are accessible only within the module (in which they are declared) MP2 Global variables are accessible to all parts of the program / MP3 Local variables use memory only while the module is executing // Global variables use memory for the whole time the program is running Max 1	1

1(a)(iii)	MP1 The same variable name can be reused in other parts of the program MP2 Using local variables makes modules self-contained // cannot accidentally change the same identifier outside of the module MP3 Using local variables aids modularisation. MP4 Memory allocated to local variables can be reused when module not in use Max 2	2																		
1(b)	Mark as follows: <table border="1"> <thead> <tr> <th>Expression</th><th>Evaluates to</th><th>Mark</th></tr> </thead> <tbody> <tr> <td>CHR(68)</td><td>'D'</td><td>1</td></tr> <tr> <td>MONTH(04/02/2025)</td><td>2</td><td>1</td></tr> <tr> <td>// -2 + DAY(04/02/2025) // -2023 + YEAR(04/02/2025)</td><td></td><td></td></tr> <tr> <td>NOT TRUE // FALSE = TRUE</td><td>FALSE</td><td>1</td></tr> <tr> <td>STR_TO_NUM(MID("Court1.4Upper",6,3))</td><td>1.4</td><td>2</td></tr> </tbody> </table> Mark Row 4 as follows: STR_TO_NUM 1 mark MID("Court1.4Upper",6,3) 1 mark	Expression	Evaluates to	Mark	CHR (68)	'D'	1	MONTH (04/02/2025)	2	1	// -2 + DAY (04/02/2025) // -2023 + YEAR (04/02/2025)			NOT TRUE // FALSE = TRUE	FALSE	1	STR_TO_NUM (MID ("Court1.4Upper",6,3))	1.4	2	5
Expression	Evaluates to	Mark																		
CHR (68)	'D'	1																		
MONTH (04/02/2025)	2	1																		
// -2 + DAY (04/02/2025) // -2023 + YEAR (04/02/2025)																				
NOT TRUE // FALSE = TRUE	FALSE	1																		
STR_TO_NUM (MID ("Court1.4Upper",6,3))	1.4	2																		

5	Example solution: <pre> FUNCTION Parity(BitString : STRING) RETURNS STRING DECLARE Index, Count: INTEGER Count ← 0 FOR Index ← 1 TO LENGTH(BitString) IF MID(BitString, Index, 1) = '1' THEN Count ← Count + 1 ENDIF NEXT Index IF Count MOD 2 = 1 THEN BitString ← BitString & '1' ELSE BitString ← BitString & '0' ENDIF RETURN BitString ENDFUNCTION </pre> MP1 Function heading and ending and parameter (STRING) and return type STRING MP2 Loop for length of BitString MP3 Extract current character in a loop MP4 Compare current character to '1' or '0' in a loop MP5 Increment count and was Initialised earlier in a loop MP6 Check if even/odd number of 1s MP7 Append '1' or '0' as appropriate MP8 Return amended BitString	8
---	--	---

21. M/J 25/P23/Q6/a

6(a)	MP1 A function returns a (single) <u>value</u> and a procedure does not MP2 A procedure can pass parameters by ByRef Max 1	1
------	---	---

22. M/J 25/P23/Q7/a,b(ii)

7(a)(i)	Example solution: <pre> PROCEDURE UpdateVisit(BYREF LastVisitDate : DATE, __ BYREF LoyaltyPoints : INTEGER) IF MONTH(TODAY()) = MONTH(LastVisitDate) THEN LoyaltyPoints ← LoyaltyPoints + 4 ELSE LoyaltyPoints ← LoyaltyPoints + 1 ENDIF LastVisitDate ← TODAY() ENDPROCEDURE </pre> MP1 Procedure heading and ending and two parameters of correct type MP2 ... both passed BYREF MP3 Use of TODAY () function to obtain current date MP4 Use MONTH function (×2) and one parameter is the header parameter and 2nd parameter is TODAY () / assigned variable MP5 If visit is same month, increase LoyaltyPoints by 4 otherwise increase by 1 MP6 LastVisitDate set to TODAY () / 'current date' variable (assigned or unassigned) Max 5	5
7(a)(ii)	DAYINDEX(LastVisitDate) = DAYINDEX(TODAY())	1
7(b)(ii)	Example solution: <pre> DayInt ← STR_TO_NUM(LEFT(LastVisitDateString, 2)) MonthInt ← STR_TO_NUM(MID(LastVisitDateString, 3, 2)) YearInt ← STR_TO_NUM(RIGHT(LastVisitDateString, 4)) LastVisitDate ← SETDATE(DayInt, MonthInt, YearInt) </pre> Mark as follows: MP1 Use of one substring function MP2 DayInt, MonthInt and YearInt all correctly extracted MP3 Use of STR_TO_NUM x 3 MP4 SETDATE () function used to convert to a date type and assigned to LastVisitDate	4

23. O/N 24/P21/Q1/b,d

1(b)(i)	Use constants (to represent the tax rate values)	1
1(b)(ii)	One mark per bullet point (or equivalent to max 3): Tax rates are entered once only - Avoids / Minimise (input) error(s) / changing the Tax rates accidentally // avoids different values for tax rates at different points in the program - When required, the constant value (representing a tax rate) is changed (once) // Easier to maintain / update the program (when the tax rates change) - Makes the program / code easier to understand	3
1(d)	OTHERWISE	1

24. O/N 24/P21/Q2

2	Example solution: <pre> PROCEDURE Tick() SS ← SS + 1 IF SS = 60 THEN SS ← 0 MM ← MM + 1 IF MM = 60 THEN MM ← 0 HH ← HH + 1 IF HH = 24 THEN HH ← 0 CALL NewDay() ENDIF ENDIF CALL CheckAlarm() ENDIF ENDPROCEDURE </pre> Mark as follows: - Increment SS anywhere in the code - Call <code>CheckAlarm()</code> when <u>SS is set to zero</u> - Test if <code>SS = 60</code> anywhere in the code ... if so: - set SS to 0 - increment MM - When MM is set to 60 set MM to 0 and increment HH - When HH is set to 24 set HH to 0 and call <code>NewDay()</code> Max 6 marks	6
---	---	----------

25. O/N 24/P21/Q4/b,c

4(b)	Answers include: Reason: • This part of the algorithm performs a specific task // if the check or calculation is changed it is changed only once • A (similar) subroutine is already available // Library routine is already available • The program is simplified / easier to understand / easier to design / code / test / debug / maintain Max 1 marks for 'Reason' Avoided: • Unnecessary code duplication • Errors caused by differences where several copies of the check and calculation exist Max 1 marks for 'Avoided'	2
------	--	----------

4(c)	Example solution: <pre> FUNCTION Evaluate (NumStr1, NumStr2 : STRING) RETURNS Result DECLARE RetVal : Result DECLARE Num1, Num2 : REAL RetVal.Done ← FALSE IF IS_NUM(NumStr1) = TRUE AND IS_NUM(NumStr2) = TRUE THEN Num1 ← STR_TO_NUM(NumStr1) Num2 ← STR_TO_NUM(NumStr2) IF Num2 <> 0 THEN RetVal.Value ← (Num1 / Num2) RetVal.Done ← TRUE ENDIF ENDIF RETURN RetVal ENDFUNCTION </pre> Note that order of parameters is not specified so divisor could be either parameter. Mark as follows: 1 Function heading, parameters, ending 2 ... Correct return type of <code>Result</code> 3 Local declaration of type <code>Result</code> 4 Attempt at using <code>IS_NUM()</code> to check both parameters/strings 5 Attempt at using of <code>STR_TO_NUM()</code> to convert both parameters/strings 6 Check if the divisor, is non-zero ... 7 ... and if so correct calculation and assignment to <code>Value</code> field 8 Return variable of type <code>Result</code> having assigned <u>both</u> fields Max 6 marks	6
------	---	----------

26. O/N 24/P21/Q6/b

6(b)	Example Solution <pre> FUNCTION GetDate(ProductionDate : DATE, ShelfLife : INTEGER) RETURNS DATE DECLARE NewDate : DATE DECLARE DD, MM, YY, LastDay : INTEGER DD ← DAY(ProductionDate) MM ← MONTH(ProductionDate) YY ← YEAR(ProductionDate) DD ← DD + ShelfLife LastDay ← DaysInMonth[MM] IF DD > LastDay THEN MM ← MM + 1 DD ← DD - LastDay IF MM = 13 THEN MM ← 1 YY ← YY + 1 ENDIF ENDIF NewDate ← SETDATE(DD, MM, YY) RETURN NewDate ENDFUNCTION </pre> Mark as follows: 1 Declare three variables of TYPE INTEGER to store DD, MM and YY 2 Correct use of date functions from Insert to extract three integer values representing DD, MM and YY and use/assign to a variable 3 Add parameter ShelfLife to DD and use / assign result to a variable 4 Extract LastDay from DaysInMonth using MM as Index 5 Test for new DD after end of month / DaysInMonth[MM] ... and if DD > DaysInMonth[MM]: 6 MM is incremented by 1 and DD is set to DD - DaysInMonth[MM] 7 If MM = 13 / MM > 12 then increment YY and set MM to 1 8 Use of SETDATE to assign value to NewDate, NewDate must have been declared as a type DATE 9 Return date Max 7 marks	7
------	--	----------

27. O/N 24/P21/Q8/a

8(a)	Loop example solution <pre> FUNCTION CheckMark(Mark : INTEGER) RETURNS BOOLEAN DECLARE Index, Lower, Upper : INTEGER FOR Index ← 1 TO 5 Lower ← GradeBoundary[Index] - 2 Upper ← GradeBoundary[Index] + 2 IF Mark >= Lower AND Mark <= Upper THEN RETURN TRUE ENDIF NEXT Index RETURN FALSE ENDFUNCTION </pre> Mark as follows: 1 Loop through all elements in <i>GradeBoundary</i> array 2 Attempt to calculate range, both <i>Lower</i> and <i>Upper</i>, in a loop 3 Completely correct range calculation in a loop 4 Test if given mark / parameter, is within range in a loop 5 Set variable / immediate RETURN if recheck required in a loop 6 Return Boolean in both cases following a reasonable attempt Selection example solution <pre> FUNCTION CheckMark(Mark : INTEGER) RETURNS BOOLEAN CASE OF Mark GradeBoundary[1] - 2 TO GradeBoundary[1] + 2 : RETURN TRUE GradeBoundary[2] - 2 TO GradeBoundary[2] + 2 : RETURN TRUE GradeBoundary[3] - 2 TO GradeBoundary[3] + 2 : RETURN TRUE GradeBoundary[4] - 2 TO GradeBoundary[4] + 2 : RETURN TRUE GradeBoundary[5] - 2 TO GradeBoundary[5] + 2 : RETURN TRUE OTHERWISE : RETURN FALSE ENDCASE ENDFUNCTION </pre> Mark as follows: 1 Correct syntax used for selection structure(s) 2 Correct checks for two ranges 3 Correct checks for three ranges 4 Correct checks for four ranges 5 Correct checks for all five ranges 6 Return Boolean in both cases following a reasonable attempt	6
------	--	----------

28. O/N 24/P22/Q2/a,b(ii)

2(a)	Example 'loop solution': <pre> FUNCTION Conceal(CardNumber : STRING) RETURNS STRING DECLARE MaskedString : STRING DECLARE Count : INTEGER CONSTANT Asterisk = '*' MaskedString ← RIGHT(CardNumber, 4) FOR Count ← 1 TO LENGTH(CardNumber) - 4 MaskedString ← Asterisk & MaskedString NEXT Count RETURN MaskedString ENDFUNCTION </pre> Mark as follows: MP1 Function heading and parameter and ending and return type MP2 Declaration of all local variables used - including the loop counter MP3 Calculate number of digits to mask / number of asterisks required MP4 use of a 'relevant' Loop MP5 Correct number of iterations MP6 Concatenate asterisk to start/end of MaskedString in a loop MP7 Assign last four characters to MaskedString // Concatenate the retained original last four digits MP8 Return masked string Max 6 marks ALTERNATIVE 'non loop' solution: <pre> FUNCTION Conceal(CardNumber : STRING) RETURNS STRING DECLARE MaskedString : STRING DECLARE Count : INTEGER CONSTANT Asterisks = "*****" //20 asterisks Count ← LENGTH(CardNumber) - 4 MaskedString ← LEFT(Asterisks, Count) & RIGHT(CardNumber, 4) RETURN MaskedString ENDFUNCTION </pre> Mark as follows: MP1 Function heading and parameter and ending and return type MP2 Declaration of all local variables used MP3 Calculate number of digits to mask / number of asterisks required	6
------	--	----------

2(a)	MP4 Trim asterisk string to number calculated in MP3 MP5 Extract the last four characters MP6 Concatenate trimmed asterisk string with last four characters of CardNumber MP7 Return masked string Max 6 marks	
2(b)(ii)	Any reference to BYREF // 'by reference'	1

29. O/N 24/P22/Q4

4	Example solution: <pre> PROCEDURE Timer(Mins, Secs : INTEGER) DECLARE WarningTick, EndTick : INTEGER EndTick ← Tick + 1000 * ((Mins * 60) + Secs) WarningTick ← EndTick - (30 * 1000) REPEAT //do nothing UNTIL Tick = WarningTick OUTPUT "30 seconds to go" REPEAT //do nothing UNTIL Tick = EndTick OUTPUT "The time is up!" ENDPROCEDURE </pre> Mark as follows: MP1 Procedure heading and parameters and ending MP2 'Attempt' to calculate 'total time'/EndTick // 'elapsed time' // WarningTick MP3 Correct calculation of EndTick and WarningTick MP4 (Design mark) • Two separate loops – checking warning time then the final time, OR ... • Single loop checking the final time with an IF statement to check for warning time, OR ... • Single loop with two IF statements checking the warning time and final time MP5 Completely correct MP4 MP6 Output both messages (must be meaningful and follow successful MP4)	6
---	--	----------

30. O/N 24/P22/Q5/c

5(c)	Example solution: <pre>IF Index Mod 2 = 0 THEN ReturnValue ← TO_UPPER(RIGHT(Label, Count)) ELSE ReturnValue ← "*****" ENDIF</pre> Mark as follows: MP1 IF...THEN...ELSE...ENDIF MP2 Both correct assignments and the correct test/logic	2
------	--	----------

31. O/N 24/P23/Q1/c

1(c)	Expression		Evaluates to	3
	Available AND NOT(Index > 100)		TRUE	
	Index MOD 30		10	
	NUM_TO_STR(Index + "33")		ERROR	
	One mark per row			

1(b)	Variable	Example data value	Data type	3
	Available	TRUE	BOOLEAN	
	Received	"18/04/2021"	STRING	
	Index	100	INTEGER	
	One mark per row			

32. O/N 24/P23/Q6

6	Loop Solution Example solution: <pre> FUNCTION AdjustClock(ThisYear : INTEGER) RETURNS INTEGER DECLARE ThisDayNumber, SundayCount : INTEGER DECLARE ThisDate : DATE ThisDayNumber ← 0 SundayCount ← 0 REPEAT ThisDayNumber ← ThisDayNumber + 1 ThisDate ← SETDATE(ThisDayNumber, 3, ThisYear) IF DAYINDEX(ThisDate) = 1 THEN SundayCount ← SundayCount + 1 ENDIF UNTIL SundayCount = 3 RETURN ThisDayNumber ENDFUNCTION </pre> Mark as follows: MP1 Function heading, parameter, ending and return type MP2 Declare local integer variable that is used to create a date MP3 Loop until 3rd Sunday found MP4 Attempt to use both SETDATE() and DAYINDEX() in a loop MP5 Correctly generate value of type DATE using SETDATE() in a loop MP6 Test if value represents a Sunday using DAYINDEX() in a loop MP7 Increment Sunday count in a loop and initialised correctly before loop MP8 Return day number of third Sunday Max 7 marks Non-loop solution Example solution: <pre> FUNCTION AdjustClock(ThisYear : INTEGER) RETURNS INTEGER DECLARE FirstDayIndex: INTEGER DECLARE ThisDate : DATE ThisDate ← SETDATE(1, 3, ThisYear) FirstDayIndex ← DAYINDEX(ThisDate) IF FirstDayIndex = 1 THEN ThirdSunday ← FirstDayIndex + 14 //First day is Sunday ELSE ThirdSunday ← 23 - FirstDayIndex // Other days ENDIF RETURN ThirdSunday ENDFUNCTION </pre>	7
---	---	---

6	Mark as follows: MP1 Function heading, parameter, ending and return type MP2 Declare local integer variable that is used to hold a day number MP4 Attempt to use both <code>SETDATE()</code> and <code>DAYINDEX()</code> MP5 Correctly generate value of type <code>DATE</code> using <code>SETDATE()</code> for first of March / specific day in March MP6 Test if date represents a Sunday / specific day using <code>DAYINDEX()</code> and calculate third Sunday // Correctly calculate third Sunday for a value returned by <code>DAYINDEX()</code> for one day MP7 Calculate third Sunday for other six days MP8 Return day number of third Sunday Max 7 marks	
---	---	--

33. M/J 24/P21/Q1/b

1(b)			4
	Expression	Evaluates to	
	Tries < 10 AND NOT Sorted	TRUE	
	Tries MOD 4	1	
	TO_LOWER(MID(ID, 3, 1))	'a' // "a"	
	LENGTH(ID & "xx") >= Tries	TRUE	
One mark per row			

34. M/J 24/P21/Q2/b

2(b)	One mark per point: 1 A (variable of type) string will be input // by example e.g. "67,72" 2 A special / identified character would need to be used to separate each numeric value // all numbers are fixed length	2
------	---	---

35. M/J 24/P21/Q6/a

6(a)	<pre> FUNCTION IsRA(x1, y1, x2, y2, x3, y3 : INTEGER) RETURNS BOOLEAN DECLARE Len1, Len2, Len3 : INTEGER Len1 ← (x1 - x2) ^ 2 + (y1 - y2) ^ 2 Len2 ← (x1 - x3) ^ 2 + (y1 - y3) ^ 2 Len3 ← (x2 - x3) ^ 2 + (y2 - y3) ^ 2 IF (Len1 = Len2 + Len3) OR__ (Len2 = Len1 + Len3) OR__ (Len3 = Len1 + Len2) THEN RETURN TRUE ELSE RETURN FALSE ENDIF ENDFUNCTION </pre> Mark as follows: 1 Calculate the square of the length of at least one side 2 Calculation of all three lengths squared correctly 3 One correct comparison of square of three lengths 4 All three comparisons... 5 combined using logical operators / nested IF // completely correct selection 6 Return result correctly in both cases	6
------	--	---

8(a)	<pre> FUNCTION DeleteComment(Line : STRING) RETURNS STRING DECLARE NewLine, TwoChars : STRING DECLARE Count, TrimTo : INTEGER CONSTANT Comment = "/" NewLine ← Line TrimTo ← 0 Count ← 1 WHILE Count < LENGTH(Line) AND TrimTo = 0 TwoChars ← MID(Line, Count, 2) // extract 2 chars IF TwoChars = Comment THEN TrimTo ← Count ENDIF Count ← Count + 1 ENDWHILE IF TrimTo <> 0 THEN NewLine ← LEFT(Line, TrimTo - 1) ENDIF RETURN NewLine ENDFUNCTION </pre> Mark as follows: 1 Loop to length of <code>Line</code> (parameter) // length of <code>Line</code> -1 2 Terminate loop on first double slash 3 Attempt to extract one/two characters in a loop 4 Check for <code>"/"</code> after attempt at extraction in a loop 5 Record start position of comment // Calculate amount of trim required 6 Attempt to trim <code>Line</code> from start of comment 7 Completely correct trimmed <code>Line</code> from start of comment 8 Return <code>Line</code> after a reasonable overall attempt	8
------	---	---

1(b)																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																													
------	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

38. M/J 24/P22/Q4

4	Example solution: <pre> PROCEDURE IsRA() DECLARE a, b, c : INTEGER OUTPUT "Input length of the first side" INPUT a OUTPUT "Input length of the second side" INPUT b OUTPUT "Input length of the third side" INPUT c IF (a * a = (b * b) + (c * c)) OR__ (b * b = (a * a) + (c * c)) OR__ (c * c = (a * a) + (b * b)) THEN OUTPUT "It is right-angled" ELSE OUTPUT "Not right-angled" ENDIF ENDPROCEDURE </pre> Mark as follows: 1. Procedure heading and ending and declaration of all variables used 2. Appropriate prompt and input for each length 3. One correct length test 4. All three length tests // selection of which test is required 5. Output one of two messages following a reasonable attempt at MP3	5
---	---	---

39. M/J 24/P22/Q5/a(ii)

5(a)(ii)	One mark per point: Statement: • The OTHERWISE statement Explanation: • The result of MOD 2 can only be 0 or 1	2
----------	---	---

40. M/J 24/P22/Q6/a

6(a)	Example solution: <pre> Function Trim(Name : STRING) RETURNS STRING CONSTANT Dots = "... " CONSTANT Space = " " IF LENGTH(Name) <= 16 THEN RETURN Name ENDIF // Otherwise it has to be trimmed WHILE LENGTH(Name) > 13 REPEAT Name ← LEFT(Name, LENGTH(Name) - 1) // strip last char UNTIL RIGHT(Name, 1) = Space // back to SPACE ENDWHILE Name ← LEFT(Name, LENGTH(Name) - 1) // remove the space Name ← Name & Dots RETURN Name ENDFUNCTION </pre> Mark as follows: 1. Function heading, ending, parameter and return type 2. If length of original string <= 16 then return original string 3. Any Conditional loop... 4. until string is short enough 5. Inner conditional loop / second condition to identify word(s) to remove from the end of string // Check to identify word(s) to remove from the end of string 6. Attempt to strip characters back to space 7. Correct removal of word/words from end of string and remove final space 8. Concatenate <code>Dots</code> and return result Max 7 marks	7
------	---	----------

41. M/J 24/P22/Q8/b

8(b)	Example algorithm based on finding position of first non-space character and then using substring function: <pre> FUNCTION DeleteSpaces(Line : STRING) RETURNS STRING DECLARE NewLine : STRING DECLARE EndOfLeading : BOOLEAN DECLARE Count, NumSpaces : INTEGER DECLARE NextChar : CHAR CONSTANT Space = " " NumSpaces ← 0 EndOfLeading ← FALSE FOR Count ← 1 TO LENGTH(Line) NextChar ← MID(Line, Count, 1) IF NextChar <> Space AND EndOfLeading = FALSE THEN NumSpaces ← Count - 1 // the number to trim EndOfLeading = TRUE ENDIF NEXT Count NewLine ← RIGHT(Line, LENGTH(Line) - NumSpaces) RETURN NewLine ENDFUNCTION </pre> Mark as follows: 1 Loop to length of parameter // Loop until first non-space character in Line 2 Extract a character in a loop 3 Identify <u>first</u> non-space character in a loop 4 Attempt at removing leading spaces in Line 5 Leading spaces removed from Line // Create new string without leading space 6 Return a string following a reasonable attempt at removing leading spaces in Line	6
------	--	----------

42. M/J 24/P23/Q1/a,b

1(a)(i)	Two marks for the benefits: 1. The code can be called when needed 2. Any subsequent change to the algorithm / calculation needs to be made once only // Easier to manage / maintain (the program) 3. Less code / no code duplication 4. The algorithm / code / calculation can be designed / coded / tested once Note: Max 2 marks	2
---------	---	----------

1(a)(ii)	One mark per point: 1. Create a module / function / procedure / subroutine using the (old) code / algorithm 2. Replace the old code / algorithm wherever it appears with a call to the module / function / procedure / subroutine 3. Pass the data as parameter(s) // Use of <u>global variable(s)</u> 4. <u>Return</u> the result (of the calculation) // Assign result to <u>global variable(s)</u> / <u>BYREF parameter(s)</u> 5. Create <u>local</u> variables as needed (to perform the calculation) Note: Max 3 marks	3										
1(b)	<table><thead><tr><th>Pseudocode expression</th><th>Evaluates to</th></tr></thead><tbody><tr><td>MID ("Random", 2, 3)</td><td>"and"</td></tr><tr><td>5 + DAY (10/11/2023)</td><td>15</td></tr><tr><td>IS_NUM ("45000")</td><td>TRUE</td></tr><tr><td>(20 MOD 3) + 1</td><td>3</td></tr></tbody></table> One mark per row	Pseudocode expression	Evaluates to	MID ("Random", 2, 3)	"and"	5 + DAY (10/11/2023)	15	IS_NUM ("45000")	TRUE	(20 MOD 3) + 1	3	4
Pseudocode expression	Evaluates to											
MID ("Random", 2, 3)	"and"											
5 + DAY (10/11/2023)	15											
IS_NUM ("45000")	TRUE											
(20 MOD 3) + 1	3											

43. M/J 24/P23/Q4/a

4(a)	Example solution: <pre> PROCEDURE TwoParts() DECLARE NextNum, Average : REAL TotalA ← 0.0 // 0 TotalB ← 0.0 // 0 REPEAT INPUT NextNum TotalA ← TotalA + NextNum UNTIL NextNum = 0 REPEAT INPUT NextNum TotalB ← TotalB + NextNum UNTIL NextNum = 0 Average ← (TotalA + TotalB) / 2 OUTPUT "The average is ", Average ENDPROCEDURE </pre> Mark as follows: 1. Procedure heading and ending 2. Declare all local variables 3. Initialise <code>TotalA</code> and <code>TotalB</code> 4. First conditional loop until zero entered, summing <code>TotalA</code> // Loop until both parts (sequences) have been entered 5. Second conditional loop until zero entered, summing <code>TotalB</code> // Loop summing appropriate Totals 6. Calculation of average and output with a message // Calculation of the average for the values making up the two totals and both output with a suitable message	6
------	--	----------

44. M/J 24/P23/Q5/a,b

5(a)	One mark per point • Count-controlled loop • the number of iterations is known	2
5(b)	Two mark for Statement of Problem: 1 The functions will return the same value every time they are called 2 ... because <code>Label</code> / the parameter value does not change within the loop Two marks for Solution: 3 Assign <code>FormatA(Label)</code> and <code>FormatB(Label)</code> to two (local) variables <u>before</u> the loop 4 Use the (new) variables in place of the function calls / in the loop // Replace the references to <code>FormatA()</code> and <code>FormatB()</code> in the CASE clauses with the new variable names	4

45. M/J 24/P23/Q6/a,b(i)

6(a)	Example Solutions: <pre> PROCEDURE Progress(Percent : INTEGER, Root : STRING) DECLARE StepValue : INTEGER DECLARE Filename : STRING StepValue ← (Percent DIV 10) + 1 // INT(Percent / 10) + 1 FileName ← Root & "-" & NUM_TO_STR(StepValue) & ".bmp" CALL Display(Filename) ENDPROCEDURE </pre> Alternative: <pre> PROCEDURE Progress(Percent : INTEGER, Root : STRING) DECLARE StepValue : INTEGER DECLARE Filename : STRING DECLARE Found : BOOLEAN StepValue ← 1 Found ← FALSE REPEAT IF Percent < StepValue * 10 THEN Found ← TRUE ENDIF StepValue ← StepValue + 1 UNTIL Found Filename ← Root & "-" & NUM_TO_STR(StepValue - 1) & ".bmp" CALL Display(Filename) ENDPROCEDURE </pre> Mark as follows for use of DIV/INT or loop solution: 1 Procedure heading, parameters and ending 2 Calculate StepValue as integer value // Attempt at calculating file number 3 Add 1 to obtain file number // Completely correct file number calculation 4 Use of NUM_TO_STR() to convert file number to string and use 5 Concatenate Root, hyphen, file number and ".bmp" suffix 6 Call Display() with filename as parameter following a reasonable attempt	6
------	--	----------

6(a)	Example Selection Solution: <pre> PROCEDURE Progress(Percent : INTEGER, Root : STRING) DECLARE Filename, StepValue : STRING CASE OF Percent < 10 : StepValue ← "1" < 20 : StepValue ← "2" < 30 : StepValue ← "3" < 40 : StepValue ← "4" < 50 : StepValue ← "5" < 60 : StepValue ← "6" < 70 : StepValue ← "7" < 80 : StepValue ← "8" < 90 : StepValue ← "9" < 100 : StepValue ← "10" OTHERWISE : StepValue ← "11" ENDCASE Filename ← Root & "-" & StepValue & ".bmp" CALL Display(Filename) ENDPROCEDURE </pre> Mark as follows for loop solution: 1 Procedure heading, parameters and ending 2 Correct selection construct(s) structure 3 Use of selection statement to obtain two file numbers 4 Use of selection statement to obtain all the file numbers 5 Concatenate Root, hyphen, file number and ".bmp" suffix 6 Call Display() once with filename as parameter following a reasonable attempt	
6(b)(i)	Example answer: <pre> FUNCTION Progress2(Percent, Steps : INTEGER, Root : STRING) RETURNS STRING </pre> One mark for each: • Keyword <code>FUNCTION Progress2</code> and three parameters of correct type • Keyword <code>RETURNS</code> and type string	2

46. M/J 24/P23/Q7/a(ii)

7(a)(ii)	Points required: 1 any change made to the parameter value / Par3 within Mod-G() is reflected in the (subsequent) value in the calling module / Mod-C() (after Mod-G() terminates) 2 any change made to the parameter value / Par3 within Mod-F() is NOT reflected in the (subsequent) value in the calling module / Mod-C() (after Mod-F() terminates) Mark as follows: 1 mark for a reasonable attempt to explain 2 marks for full explanation including context	2
----------	---	----------

47. M/J 24/P23/Q8/a

8(a)	Example: <pre> FUNCTION Header(Line : STRING) RETURNS STRING IF LENGTH(Line) >= 13 THEN IF TO_UPPER(LEFT(Line, 9)) = "FUNCTION " THEN RETURN "F" & RIGHT(Line, LENGTH(Line) - 9) ENDIF IF TO_UPPER(LEFT(Line, 10)) = "PROCEDURE " THEN RETURN "P" & RIGHT(Line, LENGTH(Line) - 10) ENDIF ENDIF RETURN "" ENDFUNCTION </pre> Mark as follows: 1 Function heading, parameter, return type and ending 2 Check that the line is at least 13 characters long before attempting to extract and return empty string 3 Attempt at: Extract characters, 9 or 10 characters, corresponding to keyword plus space and compare with appropriate keyword plus space 4 Completely correct MP3 5 Use of type case conversion to allow for 'any case' 6 Calculation of 'rest of line' and concatenation with 'P or 'F' 7 Return string	7
------	---	----------

48. O/N 23/P21/Q1/a,b,c

1(a)	One mark per row:		4
		Answer	
	The value assigned to Level when ThisValue is 40	"Medium"	
	The value assigned to Check when ThisValue is 36	12	
	The value assigned to Level when ThisValue is 18	"Low"	
	The number of elements in array Data that may be incremented	11	

1(b)	One mark for identifying assignment: MP1 <code>Level ← "Very Low"</code> // the level is assigned value "very low" Explanation points: MP2 because CASE clauses are checked in sequence // because of the order of the clauses MP3 a value < 30 satisfies the first clause // Clause '<u>< 20</u>' will never be tested	3
1(c)	MP1 all of the possible values are addressed via all / four / three / the other clauses // there are no other possible values to map to OTHERWISE	1

49. O/N 23/P21/Q6/a,b

6(a)	<pre> PROCEDURE MyOutput(NewString : STRING, EOL : BOOLEAN) IF LENGTH(MyString) + LENGTH(NewString) > 255 THEN OUTPUT MyString // Resulting string would be too long MyString ← NewString ELSE MyString ← MyString & NewString // Concat with MyString IF EOL = TRUE THEN OUTPUT MyString MyString ← "" ENDIF ENDIF ENDPROCEDURE </pre> MP1 Procedure heading, including parameters, and ending MP2 Produce concatenated string MP3 ... Check whether resulting string would be too long MP4 If so, then output old MyString MP5 ... and assign NewString to MyString MP6 Else concatenate NewString to MyString MP7 (test for length < 255) Test EOL – If TRUE then Output MP8 ... and reset MyString to empty string	7
6(b)	MP1 A new (instance of) variable MyString is created each time the procedure is called / executed MP2 So the previous contents are lost	2

50. O/N 23/P21/Q8/d

8(d)	<pre> FUNCTION GetField(Msg : STRING, FieldNo : INTEGER) RETURNS STRING DECLARE RetString : STRING CASE OF FieldNo 1 : RetString ← MID(Msg, 2, 3) 2 : RetString ← MID(Msg, 5, 3) 3 : RetString ← MID(Msg, 8, LENGTH(Msg) - 8) OTHERWISE : RetString ← "" ENDCASE RETURN RetString ENDFUNCTION </pre> MP1 Use of CASE ... ENDCASE or IF ... THEN ... ENDIF MP2 Field 1 and Field 2 extracted correctly MP3 Calculate a length of field 3 MP4 Field 3 extracted <u>correctly</u> MP5 Return empty string in case of invalid parameter (via OTHERWISE or initialisation) MP6 Final RETURN, after a reasonable attempt	6
------	--	----------

51. O/N 23/P22/Q1/b

1(b)	One mark per row <table><thead><tr><th>Expression</th><th>Evaluates to</th></tr></thead><tbody><tr><td>(Total * DepRate) + 1.5</td><td>249.50</td></tr><tr><td>RIGHT(Description, 7)</td><td>"(small)"</td></tr><tr><td>(LENGTH(Description) - 8) > 16</td><td>TRUE</td></tr><tr><td>NUM_TO_STR(INT(DepRate * 10)) & '%'</td><td>"20%"</td></tr></tbody></table>	Expression	Evaluates to	(Total * DepRate) + 1.5	249.50	RIGHT(Description, 7)	"(small)"	(LENGTH(Description) - 8) > 16	TRUE	NUM_TO_STR(INT(DepRate * 10)) & '%'	"20%"	4
Expression	Evaluates to											
(Total * DepRate) + 1.5	249.50											
RIGHT(Description, 7)	"(small)"											
(LENGTH(Description) - 8) > 16	TRUE											
NUM_TO_STR(INT(DepRate * 10)) & '%'	"20%"											

52. O/N 23/P22/Q2/b

2(b)	One mark per point: 1 Name: (pre / post) conditional loop 2 Justification: the number of iterations is not known // loop ends following a specific input (in the loop)	2
------	---	----------

53. O/N 23/P22/Q4/a

4(a)	Example Solution <pre> PROCEDURE Count() DECLARE COdd, CEven, ThisNum : INTEGER COdd ← 0 CEven ← 0 INPUT ThisNum WHILE ThisNum <> 99 IF ThisNum MOD 2 = 1 THEN COdd ← COdd + 1 ELSE CEven ← CEven + 1 ENDIF INPUT ThisNum ENDWHILE OUTPUT "Count of odd and even numbers: ", COdd, CEven ENDPROCEDURE </pre> Mark as follows Max 6 marks: 1 Procedure heading and ending 2 Local COdd, CEven and ThisNum declared as integers 3 Conditional loop while ThisNum <> 99 4 Input ThisNum in a loop 5 Check ThisNum is odd or even in a loop 6 Increment appropriate count in a loop, both counts must have been initialised before loop 7 After the loop output COdd and CEven with a suitable message following a reasonable attempt at counting	6
------	---	----------

54. O/N 23/P22/Q6/b(i,ii)

6(b)(i)	Function	1
6(b)(ii)	FUNCTION CheckFiles (NameRoot : STRING) RETURNS INTEGER	1

6(a)	Example Solution <pre> PROCEDURE CreateFiles(NameRoot : STRING, NumFiles : INTEGER) DECLARE FileName, Suffix : STRING DECLARE Count : INTEGER FOR Count ← 1 TO NumFiles Suffix ← NUM_TO_STR(Count) WHILE LENGTH(Suffix) <> 3 Suffix ← '0' & Suffix ENDWHILE FileName ← NameRoot & '.' & Suffix OPENFILE FileName FOR WRITE WRITEFILE FileName, "This is File " & FileName CLOSEFILE FileName NEXT Count ENDPROCEDURE </pre> Mark as follows: 1 Procedure heading, including parameters, and ending 2 Loop for NumFiles iterations 3 Attempt to create filename suffix using NUM_TO_STR() in a loop 4 Completely correct filename 5 OPENFILE in WRITE mode and subsequent CLOSE in a loop 6 WRITE initial first line to the file in a loop	6
------	---	----------

55. O/N 23/P22/Q8/b

8(b)	Example solution <pre> PROCEDURE ProcessMsg(ThisMsg : STRING) DECLARE ThisDest : STRING DECLARE Response : BOOLEAN DECLARE StackNum : INTEGER IF LENGTH(ThisMsg) >= 4 THEN ThisDest ← LEFT(ThisMsg, 3) IF ThisDest = MyID THEN // It's for this computer StackNum ← 1 ELSE StackNum ← 2 ENDIF Response ← StackMsg(ThisMsg, StackNum) IF Response = FALSE THEN OUTPUT "Message discarded - no room on stack" ENDIF ENDIF ENDPROCEDURE </pre> Mark as follows: 1. Ignore message if data field is empty 2. Extract <code>ThisDest</code> from <code>ThisMsg</code> 3. Test if destination is this computer 4. Attempt to use <code>StackMsg()</code> 5. Fully correct use of <code>StackMsg()</code> for both cases / stacks 6. Test <code>StackMsg()</code> return value for both cases 7. Following a reasonable attempt at MP6 output warning if <code>StackMsg()</code> returns <code>FALSE</code>	7
------	---	---

56. O/N 23/P23/Q1/b,c

1(b)			4
	Expression	Evaluates to	
	Fraction >= 0.2 AND NOT Active	FALSE	
	INT((Fraction1 * 100) + 13.3)	33	
	STR_TO_NUM(MID(Code, 4, 2)) + 5	28	
	LENGTH("TRUE" & Code)	11	
1(c)	The use of a program <u>library</u> (routines)		1

57. O/N 23/P23/Q4/a,b

4(a)	<pre> PROCEDURE RandList() DECLARE Count, BaseNum, ThisNum : INTEGER CONSTANT StepVal = 10 BaseNum ← 0 FOR Count ← 1 TO 25 ThisNum ← BaseNum + INT(RAND(StepVal)) OUTPUT ThisNum BaseNum ← BaseNum + StepVal NEXT Count ENDPROCEDURE </pre> MP1 Procedure heading and ending MP2 <u>Local</u> loop counter Count as integer MP3 Loop to iterate 25 times or more for each unique number MP4 'Attempt' to generate a random number including use of INT() in a loop MP5 Ensure that number generated is greater than previous and change 'previous' MP6 Output random number after an attempt at MP5 in a loop	6
4(b)	One mark for simplified logical expression: MP1 Result[x + 1] <= Result[x] ALTERNATIVE SOLUTION: Result[x] >= Result[x + 1]	1

58. O/N 23/P23/Q6

6	<pre> FUNCTION TestNum(ThisNum : STRING) RETURNS INTEGER IF LEFT(ThisNum,3) = RIGHT(ThisNum 3) THEN RETURN 3 ENDIF IF RIGHT(ThisNum, 3) = "000" THEN RETURN 2 ENDIF IF MID(ThisNum, 4, 1) = MID(ThisNum, 5, 1)___ AND MID(ThisNum, 5, 1) = MID(ThisNum, 6, 1) THEN RETURN 1 ENDIF RETURN 0 ENDFUNCTION </pre> MP1 Function heading and ending including parameter and return type MP2 Test for Condition 1 MP3 Test for Condition 2 MP4 Test for Condition 3 MP5 Return the highest value if more than one condition is satisfied MP6 Return zero if no condition matched	6
---	---	---

59. O/N 23/P23/Q7/b(ii)

7(b)(ii)	<pre> PROCEDURE Sub-9 (BYREF Param : MyType) </pre> MP 1 One mark for BYREF MP 2 One mark for the rest of the statement	2
----------	---	---

60. O/N 23/P23/Q8/c

8(c)	<pre> PROCEDURE Chat(Destination : STRING, Port : INTEGER) DECLARE Data : STRING DECLARE Finished : BOOLEAN CONSTANT Terminator = "Bye" CONSTANT STX = CHR(2) CONSTANT ETX = CHR(3) Finished ← FALSE REPEAT Data ← GetData() OUTPUT Data IF Data = Terminator THEN Finished ← TRUE ENDIF IF NOT Finished THEN //about to reply INPUT Data Transmit(STX & Destination & MyID & Data & ETX, Port) IF Data = Terminator THEN Finished ← TRUE ENDIF ENDIF UNTIL Finished = TRUE ENDPROCEDURE </pre> Conditional loop MP1 Conditional loop MP2 Test for terminator in both cases MP3 Use <code>GetData()</code> to get the data from the message MP4 OUTPUT the data in a loop MP5 INPUT the data reply MP6 'Attempted' use of <code>Transmit</code> to send it in a loop MP7 Correct formation of parameters to <code>Transmit()</code>	7
------	---	---

61. M/J 23/P21/Q1/b,c

1(b)	One mark for correct description of: Error 1: Brackets mismatch // 2/value should be added after brackets/function // Addition between a string and a number is not valid // <code>STR_TO_NUM</code> / the function needs to be passed a string / not an integer Error 2: No Error Error 3: <code>MONTH()</code> returns an integer and this is being compared with a character/string // Integer cannot be compared to a string // 6 should not be in quotes	3
------	---	---

1(c)(i)			2
		Expression	Evaluation
	1	(Points > 99) OR Active	TRUE
	2	(Points MOD 2 = 0) OR Exempt	FALSE
	3	(Points <= 75) AND (Active OR Exempt)	TRUE
	4	(Active OR NOT Active) AND NOT Exempt	TRUE
One mark for any two rows correct Two marks for all rows correct			
1(c)(ii)	NOT Exempt		1

62. M/J 23/P21/Q2/b

2(b)	One mark for declaration: <u>DECLARE</u> <u>StartDate</u> : <u>DATE</u> One mark for each underlined part of assignment: <u>StartDate</u> ← <u>SETDATE</u> (15, 11, 2005)	3
------	---	----------

63. M/J 23/P21/Q4

4	<pre>Function Replace(OldString : STRING, Char1, Char2 : CHAR) __ RETURNS : STRING DECLARE NewString : STRING DECLARE ThisChar : CHAR DECLARE Index : INTEGER NewString ← "" FOR Index ← 1 TO LENGTH(OldString) ThisChar ← MID(OldString, Index, 1) IF ThisChar = Char1 THEN ThisChar ← Char2 ENDIF NewString ← NewString & ThisChar NEXT Index RETURN NewString ENDFUNCTION</pre> Mark as follows: - Function heading and ending, including parameters and return type - Declaration of local variables used including loop counter - Loop for length of OldString - Extract char and test in a loop - Use of concatenate to build NewString replace char if necessary, in a loop - Return NewString after reasonable attempt	6
---	---	----------

64. M/J 23/P21/Q8/a

8(a)	<pre> FUNCTION IsNewSupp(ThisString : STRING) RETURNS BOOLEAN DECLARE Index : INTEGER DECLARE ThisChar : CHAR IF LENGTH(ThisString) <> 5 THEN RETURN FALSE // invalid SupplierCode length ENDIF IF SuppExists(ThisString) THEN RETURN FALSE // SupplierCode already exists ENDIF FOR Index ← 1 TO 5 ThisChar ← TO_LOWER(MID(ThisString, Index, 1)) IF ThisChar < 'a' OR ThisChar > 'z' THEN RETURN FALSE ENDIF NEXT Index RETURN TRUE ENDFUNCTION </pre> Mark as follows: 1 Check <code>ThisString</code> is exactly 5 characters in length 2 Use of <code>SuppExists()</code> with a string of 5 characters as a parameter 3 Loop for 5 iterations // loops for length of string parameter 4 Extract a character in a loop 5 Test if char is alphabetic in a loop 6 ...catering for both upper and lower case 7 Return Boolean following a reasonable attempt	7
------	---	----------

65. M/J 23/P22/Q1/a

1(a)(i)	Use of constants	1
1(a)(ii)	One mark per bullet point (or equivalent to max 3): 1 Postal rates are entered once only 2 Avoids input error / changing the cost accidentally // avoids different values for postal rates at different points in the program 3 When required, the constant representing the postal rate value is changed once only // easier to maintain the program when the postal rates change 4 Makes the program easier to understand Note: Max 3 marks	3

66. M/J 23/P22/Q2/a,b

2(a)	<code>MyDOB ← SETDATE(17, 11, 2007)</code>	1
------	--	----------

2(b)	<u>NumMonths</u> $\leftarrow$ <u>12 - MONTH(MyDOB)</u>	2
	One mark per underlined part	

67. M/J 23/P22/Q4

4	<pre> Function GetNum(ThisString : STRING, ThisChar : CHAR) RETURNS INTEGER DECLARE Index, Count : INTEGER Count $\leftarrow$ 0 FOR Index $\leftarrow$ 1 TO LENGTH(ThisString) IF MID(ThisString, Index, 1) = ThisChar THEN Count $\leftarrow$ Count + 1 ENDIF NEXT Index RETURN Count ENDFUNCTION </pre> Mark as follows: 1 Function heading and end, including parameters and return type 2 Declare local Integers for Index and Count 3 Loop for length of ThisString 4 Extract a character and compare with <u>parameter</u> in a loop 5 Increment Count if match in a loop 6 Return Count after loop	6
---	---	----------

68. M/J 23/P22/Q6

6	Example of iterative solution: <pre> PROCEDURE Square(Dim : INTEGER) DECLARE Count : INTEGER DECLARE ThisChar : CHAR DECLARE StringA, StringB : STRING CONSTANT FILLER = '*' StringA ← "" ThisChar ← NUM_TO_STR(Dim) FOR Count ← 1 TO Dim StringA ← StringA & ThisChar //build up first & last line NEXT Count StringB ← ThisChar FOR Count ← 1 TO Dim - 2 StringB ← StringB & FILLER //build up intermediate line NEXT Count StringB ← StringB & ThisChar // add final digit OUTPUT StringA FOR Count ← 1 TO Dim - 2 OUTPUT StringB NEXT Count IF Dim <> 1 THEN OUTPUT StringA ENDIF ENDPROCEDURE </pre> For loop-based solutions, mark as follows: 1 Procedure heading and ending including parameter 2 Loop using parameter, containing attempt to construct first line / last line 3 Construct first line / last line 4 Attempt at loop to construct intermediate line 5 Output first / last line of square when <code>Dim > 2</code> 6 Output all intermediate lines in a loop 7 Correct output of first two squares Note: Max 6 marks	6
---	---	---

6

Example of selection-based solution:

```

PROCEDURE Square(Dim : INTEGER)
  DECLARE Count : INTEGER

  CASE OF Dim
    1 : OUTPUT "1"
    2 : OUTPUT "22"
      OUTPUT "22"
    3 : OUTPUT "333"
      OUTPUT "3*3"
      OUTPUT "333"
    4 : OUTPUT "4444"
      FOR Count ← 1 TO 2
        OUTPUT "4**4"
      NEXT Count
      OUTPUT "4444"
    5 : OUTPUT "55555"
      FOR Count ← 1 TO 3
        OUTPUT "5***5"
      NEXT Count
      OUTPUT "55555"
    6 : OUTPUT "666666"
      FOR Count ← 1 TO 4
        OUTPUT "6****6"
      NEXT Count
      OUTPUT "666666"
    7 : OUTPUT "7777777"
      FOR Count ← 1 TO 5
        OUTPUT "7*****7"
      NEXT Count
      OUTPUT "7777777"
    8 : OUTPUT "88888888"
      FOR Count ← 1 TO 6
        OUTPUT "8*****8"
      NEXT Count
      OUTPUT "88888888"
    9 : OUTPUT "999999999"
      FOR Count ← 1 TO 7
        OUTPUT "9*****9"
      NEXT Count
      OUTPUT "999999999"

  ENDCASE

ENDPROCEDURE

```

For in-line / selection-based solutions, mark as follows:

- 1 Procedure heading and ending including parameter
- 2 Correct use of parameter to select all required squares
- 3 Correct output of first two squares
- 4 At least six squares correctly output
- 5 At least one loop to output lines with multiple asterisks
- 6 All number squares output correctly

69. M/J 23/P22/Q8/a

8(a)	<pre> FUNCTION CheckInfo(NewLine: STRING) RETURNS BOOLEAN DECLARE ThisNum : STRING DECLARE Index : INTEGER IF LENGTH(NewLine) < 19 THEN RETURN FALSE ENDIF FOR Index ← 1 TO 4 IF NOT IS_NUM(MID(NewLine, Index, 1)) THEN RETURN FALSE ENDIF NEXT Index ThisNum ← LEFT(Newline, 4) IF ThisNum < "0001" OR ThisNum > "5999" THEN RETURN FALSE ENDIF IF NOT OnlyAlpha(MID(Newline, 5, 3)) THEN RETURN FALSE ENDIF RETURN TRUE ENDFUNCTION </pre> Mark as follows: 1 Test length of parameter 2 Extract first 4 characters of parameter (as ItemNum) 3 Test first four characters are all numeric 4 Test ItemNum in range "0001" to "5999" 5 Extract characters 5 to 7 of parameter (as SupplierCode) 6 Use of OnlyAlpha() with extracted SupplierCode 7 Return BOOLEAN value correctly in all cases, must have been declared as local	7
------	--	----------

70. M/J 23/P23/Q2/a

2(a)	One mark for each underlined part <pre> IF <u>DAYINDEX(MyDOB)</u> <u>= 5</u> THEN </pre>	2
------	--	----------

71. M/J 23/P23/Q4

4	<pre> FUNCTION MakeString(Count : INTEGER, AChar : CHAR) RETURNS STRING DECLARE MyString : STRING DECLARE Index : INTEGER IF Count < 1 THEN MyString ← "ERROR" ELSE MyString ← "" FOR Index ← 1 TO Count MyString ← MyString & AChar NEXT Index ENDIF RETURN MyString ENDFUNCTION </pre> MP1 Function heading and end including parameters and return type MP2 Declaration of locals <code>Index</code> and <code>MyString</code> MP3 Test for <code>Count < 1</code> and if true, assign "ERROR" to <code>MyString</code> / Immediate RETURN MP4 Loop for <code>Count</code> iterations MP5 Use of concatenate – must have been initialised in a loop MP6 Return STRING (correctly in both cases)	6
---	--	---

72. M/J 23/P23/Q6/a,b

6(a)	Max 7 marks <pre>PROCEDURE Select(Start, End : INTEGER) DECLARE ThisNum, Total: INTEGER DECLARE ThisString : STRING DECLARE Char1, Char2 : CHAR FOR ThisNum ← Start+1 TO End-1 ThisString ← NUM_TO_STR(ThisNum) Char1 ← RIGHT(ThisString, 1) Char2 ← LEFT(RIGHT(ThisString, 2), 1) Total ← STR_TO_NUM(Char1) + STR_TO_NUM(Char2) IF Total = 6 THEN OUTPUT ThisString ENDIF NEXT ThisNum ENDPROCEDURE</pre> MP1 Procedure heading and ending MP2 (Count-controlled) Loop MP3 with correct range from Start+1 to End-1 MP4 Convert ThisNum (loop counter) to a string MP5 Extract the last two/first/second 'character digit(s)' required in a loop MP6 Extract the second individual 'character digit' required in a loop MP7 Calculate the sum of the last two digits MP8 If sum = 6 then OUTPUT the number (either string or integer) in a loop	7				
6(b)	Max 4 marks MP1 The function will take two integer parameters - the number and the (required) total MP2 ... and return a Boolean OR: <table><tr><td><u>CheckNum(Number, Total : INTEGER)</u></td><td><u>RETURNS BOOLEAN</u></td></tr><tr><td>MP1</td><td>MP2</td></tr></table> Two marks for the advantages: MP3 CheckNum() can be called repeatedly as and when required MP4 CheckNum() is designed and tested once (then used repeatedly) MP5 Any subsequent change to <u>CheckNum()</u> needs to be made once only // is easier to maintain/modify	<u>CheckNum(Number, Total : INTEGER)</u>	<u>RETURNS BOOLEAN</u>	MP1	MP2	4
<u>CheckNum(Number, Total : INTEGER)</u>	<u>RETURNS BOOLEAN</u>					
MP1	MP2					

73. O/N 22/P21/Q1/a,c

1(a)	One mark per point: 1 They are tried and tested so free from errors 2 They perform a function that you may not be able to program yourself (for example encryption) 3 They are readily available / speed up development time	3
------	---	---

1(c)	One mark per row: <table><tr><th>Statement</th><th>Error</th></tr><tr><td>IF EMPTY ← "" THEN</td><td>Should be "=" not ←</td></tr><tr><td>Status ← IS_NUM(-23.4)</td><td>Parameter should be a string (or char) // should not be a real</td></tr><tr><td>X ← STR_TO_NUM("37") + 5</td><td>NO ERROR</td></tr><tr><td>Y ← STR_TO_NUM("37" + "5")</td><td>Wrong operator – should be & or Parameter is not a string</td></tr></table>	Statement	Error	IF EMPTY ← "" THEN	Should be "=" not ←	Status ← IS_NUM(-23.4)	Parameter should be a string (or char) // should not be a real	X ← STR_TO_NUM("37") + 5	NO ERROR	Y ← STR_TO_NUM("37" + "5")	Wrong operator – should be & or Parameter is not a string	4
Statement	Error											
IF EMPTY ← "" THEN	Should be "=" not ←											
Status ← IS_NUM(-23.4)	Parameter should be a string (or char) // should not be a real											
X ← STR_TO_NUM("37") + 5	NO ERROR											
Y ← STR_TO_NUM("37" + "5")	Wrong operator – should be & or Parameter is not a string											

74. O/N 22/P21/Q6/a

6(a)	One mark for any part correct (accept equivalent wording) (Max 1): • Condition evaluates to TRUE if bracket contents evaluate to FALSE: • Bracket contents evaluate to FALSE if: • Dots: zero / less than one or • Ats: not equal to one or • Others: less than nine	1
------	---	----------

75. O/N 22/P21/Q7/a

7(a)(i)	One mark per point (Max 7) as follows: 1 Declaration of local variables for Par1 Par2 and Par3 2 Loop to end of (parameter) string // until operator is found 3 Extract a character in a loop... 4 Attempt at extraction of three parts of expression using substring functions 5 Completely correct extraction of all three parts of expression 6. Convert string to Integer using STR_TO_NUM(<something sensible>) 7 Attempt to interpret at least two operators (Par2): + - * / 8 Corresponding correct calculation (all operators) and final Output of result <pre> PROCEDURE Calculate(Expression : STRING) DECLARE Val1, Val2, Index : INTEGER DECLARE Result : REAL DECLARE Par1, Par2, Par3 : STRING CONSTANT PLUS = '+' CONSTANT MINUS = '-' CONSTANT MULTIPLY = '*' CONSTANT DIVIDE = '/' FOR Index ← 1 TO LENGTH(Expression) //search for operator ThisChar ← MID(Expression, Index, 1) IF IS_NUM(ThisChar) = FALSE THEN Par1 ← LEFT(Expression, Index - 1) Par2 ← ThisChar Par3 ← RIGHT(Expression, LENGTH(Expression) - Index) ENDIF NEXT Index Val1 ← STR_TO_NUM(Par1) Val2 ← STR_TO_NUM(Par3) CASE OF Par2 PLUS : Result ← Val1 + Val2 MINUS : Result ← Val1 - Val2 MULTIPLY : Result ← Val1 * Val2 DIVIDE : Result ← Val1 / Val2 ENDCASE OUTPUT Result ENDPROCEDURE </pre>	7
7(a)(ii)	FUNCTION Calculate(Expression : STRING) RETURNS REAL	1

76. O/N 22/P21/Q8/b

8(b)	<code>CALL CountErrors("Jim01Prog.txt", 20)</code> One mark for each: 1 Module name, at least one parameter in brackets and one parameter correct 2 Completely correct statement	2
------	---	----------

77. O/N 22/P22/Q1/a,c

1(a)(i)	One mark for each point (Max 2): • When a task which is repeated / reused / performed in several places • When a part of an algorithm performs a specific task • Reduces complexity of program / program is simplified // subroutine already available • Testing / debugging / maintenance is easier	2
---------	---	----------

1(a)(ii)	One mark for each part: Term: Parameter(s) Use: to pass values / arguments to the procedure	2
----------	---	----------

1(c)	One mark per row:		3
	Expression	Evaluation	
	MID(CharList, MONTH(FlagDay), 1)	'D'	
	INT(Count / LENGTH(CharList))	4	
	(Count >= 99) AND (DAY(FlagDay) > 23)	FALSE	

78. O/N 22/P22/Q4/b

4(b)	Pre-condition (loop) / count-controlled (loop)	1
------	--	----------

79. O/N 22/P22/Q6/a

6(a)	Example by repeated multiplication: Mark as follows (multiplication solution), (Max 7): 1 Function heading and ending including parameter and return type 2 Declaration and initialisation of local Num 3 Any conditional loop 4 Conditional loop until ThisValue found or Try out of range 5 Multiply Try by Num in a loop 6 Compare Try with ThisValue and set termination if the same in a loop 7 Increment Num and repeat in a loop 8 Attempt to Return Num if ThisValue is a factorial or -1 otherwise <pre> FUNCTION FindBaseNumber(ThisValue : INTEGER) RETURNS INTEGER DECLARE Num, Try : INTEGER DECLARE Found : BOOLEAN Num ← 0 Found ← FALSE Try ← 1 WHILE Try <= ThisValue AND Found = FALSE Num ← Num + 1 Try ← Try * Num IF Try = ThisValue THEN //BaseNumber found Found ← TRUE ENDIF ENDWHILE IF Found = TRUE THEN RETURN Num ELSE RETURN -1 ENDIF ENDFUNCTION </pre>	7
------	---	----------

6(a)	Alternative FOR LOOP solution. Mark as follows: 1 Function heading and ending including parameter and return type 2 Declaration of local Integer value for Num and Try 3 Count-controlled Loop from 1 to ThisValue 4 Multiply Try by Num in a loop 5 Compare Try with ThisValue in a loop 6 ...Immediate return of Num if they are the same in a loop 7 Return -1 if ThisValue not found after loop <pre> FUNCTION FindBaseNumber(ThisValue : INTEGER) RETURNS INTEGER DECLARE Num, Try : INTEGER Try ← 1 FOR Num ← 1 TO ThisValue Try ← Try * Num IF Try = ThisValue THEN //BaseNumber found RETURN Num ENDIF NEXT Num RETURN -1 ENDFUNCTION </pre>	
------	---	--

80. O/N 22/P23/Q5/a,b(ii)

5(a)(i)	One mark per point: 1 Procedure heading and ending including four parameters... 2 ...and use of BYREF for the three extracted values 3 Extract and assign SID 4 Extract and assign SDesc 5 Calculation of length of SCost (remainder of string) 6 Extract and assign SCost following an attempt at MP5 <pre> PROCEDURE UnPack(BYVAL TLine : STRING, BYREF SID, SDesc, SCost : STRING) SID ← LEFT(TLine, 5) SDesc ← MID(TLine, 6, 32) SCost ← RIGHT(TLine, LENGTH(TLine) - 37) ENDPROCEDURE </pre>	6
5(a)(ii)	One mark each (Max 2): 1 Provides a mechanism to allow calling program to pass data 2 Defines the four parameters of Unpack() 3 ... giving their <u>data type and order</u>	2

5(b)(ii)	One mark per point (Max 2): • The new function will return an item of type <code>StockItem</code> • Need to declare/use a (local) variable of type <code>StockItem</code> • <code>Costfield</code> needs to be converted from a string to a real	2
----------	--	----------

81. O/N 22/P23/Q6/b

6(b)	One mark per point: 1 Function heading and ending including parameters 2 Declaration of local variable for Result / alt mechanism 3 Check for Reject 4 Check for Accept 5 Check for Recheck (or just default to third option) 6 Return Result following a reasonable attempt <pre> Function Status(Actual, Min, Max : INTEGER) RETURNS CHAR DECLARE Result : CHAR CONSTANT Accept = 'A' CONSTANT Reject = 'R' CONSTANT ReCheck = 'C' Result ← ReCheck //Check if reject IF Actual > Max + 2 OR Actual < Min - 2 THEN Result ← Reject ENDIF //Check if acceptable IF Actual < Max - 2 AND Actual > Min + 2 THEN Result ← Accept ENDIF RETURN Result ENDFUNCTION </pre>	6
------	--	----------

82. O/N 21/P21/Q6/c

6(c)	<pre> FUNCTION GetCentreCol(ThisRow : INTEGER) RETURNS INTEGER DECLARE StartCol, EndCol, CentreCol : INTEGER StartCol ← SearchInRow(ThisRow, 1) IF StartCol = -1 THEN CentreCol ← StartCol ELSE EndCol ← SearchInRow(ThisRow, 1280) CentreCol ← INT((StartCol + EndCol)/2) ENDIF RETURN CentreCol ENDFUNCTION </pre> Mark as follows: 1 Declaration of local INTEGER for return value 2 Use SearchInRow() with correct parameters and check for -1 3 Use SearchInRow(ThisRow, 1) and SearchInRow(ThisRow, 1280) 4 Calculate centre column 5 Use of INT() function // use of DIV 6 Return -1 or centre value Max 5 marks if function heading, including return type, and ending is incorrect or incomplete	6
------	--	----------

83. O/N 21/P22/Q1/c,d

1(c)	<table><tr><th>Expression</th><th>Evaluates to</th></tr><tr><td>ASC('C')</td><td>67</td></tr><tr><td>2 * STR_TO_NUM ("27")</td><td>54</td></tr><tr><td>INT(27 / 2)</td><td>13</td></tr><tr><td>"Sub" & MID("Abstraction" , 4 , 5)</td><td>"Subtract"</td></tr></table>		Expression	Evaluates to	ASC('C')	67	2 * STR_TO_NUM ("27")	54	INT(27 / 2)	13	"Sub" & MID("Abstraction" , 4 , 5)	"Subtract"	4
	Expression	Evaluates to											
	ASC('C')	67											
	2 * STR_TO_NUM ("27")	54											
	INT(27 / 2)	13											
	"Sub" & MID("Abstraction" , 4 , 5)	"Subtract"											
One mark per row													
Function names must be exactly as shown													

1(d)	Expression	Evaluates to	2
	PressureOK AND HiFlow	FALSE	
	PumpOn OR PressureOK	TRUE	
	NOT PumpOn OR (PressureOK AND NOT HiFlow)	TRUE	
	NOT (PumpOn OR PressureOK) AND NOT HiFlow	FALSE	
1 mark for any two rows correct 2 marks for all rows correct.			

84. O/N 21/P22/Q4/a,c

4(a)	Answer		5
	A line number containing a variable being incremented	19 / 21 / 23	
	The type of loop structure	pre-condition	
	The number of functions used	3	
	The number of parameters passed to function STR_TO_NUM()	1	
	The name of a procedure other than Check()	Result	
4(c)(i)	One mark per point: Structure: A count-controlled loop Justification: The number of iterations is known // repeats for the length of InString		2
4(c)(ii)	15, 23 One mark for both line numbers		1

85. O/N 21/P22/Q6/b,c(ii),d

6(b)	One mark for: (A flag is used to) exit the loops // iteration is terminated - as soon as a Screen element with value 1 is found	2
------	--	----------

6(c)(ii)	One mark for advantage and one for disadvantage: Advantage: (max 1) • Only have to change one module if specification changes • Less repetitive code / fewer lines of code • Aids re-usability Disadvantage: (max 1) • Single module more complex / more error prone / more difficult to debug ... • Single module cannot be split among programmers / teams Max 2	2
6(d)	<pre> PROCEDURE GetCentre () DECLARE StartRow, EndRow, StartCol, EndCol : INTEGER StartRow ← FirstRowSet() IF StartRow = -1 THEN CentreRow ← -1 // no 'touch' detected ELSE EndRow ← LastRowSet() StartCol ← FirstColSet() EndCol ← LastColSet() CentreRow ← INT((StartRow + EndRow)/2) CentreCol ← INT((StartCol + EndCol)/2) ENDIF ENDPROCEDURE </pre> Mark as follows: 1 Call <any Set function> and check for -1 // check for no element set 2 ...and if so set CentreRow to -1 3 Call all 4 Set functions to get 'extremity' values 4 Calculate centre row and centre column 5 Use of INT() function or DIV operator on values from MP4 6 Assign calculated values to CentreRow and CentreCol Note: Max 5 if procedure heading and ending missing or incorrect (ignore array if passed as a parameter) or any local variables are undefined or of incorrect type	6

6(a)	<pre> FUNCTION FirstRowSet() RETURNS INTEGER DECLARE Row, Col : INTEGER DECLARE Found : BOOLEAN // array is 1280 x 800 Row ← 1 Found ← FALSE WHILE Row ≤ 800 AND Found = FALSE // top to bottom Col ← 1 WHILE Col ≤ 1280 AND Found = FALSE // left to right IF Screen[Row,Col] = 1 THEN Found ← TRUE // end function as soon as first // found ENDIF Col ← Col + 1 ENDWHILE Row ← Row + 1 ENDWHILE IF Found = FALSE THEN // nothing found Row ← 0 ENDIF RETURN Row - 1 ENDFUNCTION </pre> Mark as follows: 1 Function heading and ending and return type 2 (Conditional) outer loop 1 to 800 (row) 3 (Conditional) inner loop 1 to 1280 // 1280 to 1 (column) 4 Reference <code>Screen</code> element and test for = 1 // <> 0 5 and if true save row number and exit loops 6 Increment index variables in both inner and outer loop 7 Return Row number or -1, following a reasonable attempt	7
------	---	---

6(c)	<pre> FUNCTION GetCentreCol(ThisRow : INTEGER) RETURNS INTEGER DECLARE StartCol, EndCol, CentreCol : INTEGER StartCol ← SearchInRow(ThisRow, 1) IF StartCol = -1 THEN CentreCol ← StartCol ELSE EndCol ← SearchInRow(ThisRow, 1280) CentreCol ← INT((StartCol + EndCol)/2) ENDIF RETURN CentreCol ENDFUNCTION </pre> Mark as follows: 1 Declaration of local INTEGER for return value 2 Use SearchInRow() with correct parameters and check for -1 3 Use SearchInRow(ThisRow, 1) and SearchInRow(ThisRow, 1280) 4 Calculate centre column 5 Use of INT() function // use of DIV 6 Return -1 or centre value Max 5 marks if function heading, including return type, and ending is incorrect or incomplete	6
------	--	---

87. M/J 21/P21/Q1/b

1(b)

Statement	Error
Result ← 2 & 4	Should be arithmetic operator (not &) // 2 and 4 should be CHAR / STRING
SubString ← MID("pseudocode", 4, 1)	NO ERROR
IF x = 3 OR 4 THEN	Not Boolean values / incorrect operator // Condition incorrect
Result ← Status AND INT(X/2)	INT(X/2) doesn't evaluate to a boolean value / incorrect operator
Message ← "Done" + LENGTH(MyString)	Can't add string to number / "Done" is not a number

One mark for each row

5

88. M/J 21/P21/Q2/b

2(b)	<u>PROCEDURE LoanReturn (LoanID, BookID : STRING,</u> <u>BYREF Fine : REAL)</u>	2
	One mark for each underlined part	

89. M/J 21/P21/Q4/b

4(b)	Loop structure: A count-controlled loop Justification: The number of iterations is known One mark per point	2
------	---	----------

90. M/J 21/P21/Q7/a,c

7(a)	<pre> FUNCTION GetStart (WordNum : INTEGER) RETURNS INTEGER DECLARE Index, ThisPos, NumFound : INTEGER DECLARE ThisChar : Char CONSTANT SPACECHAR = ' ' Index ← -1 Numfound ← 0 ThisPos ← 1 IF WordNum = 1 THEN // if looking for word 1... Index ← 1 // Word 1 always starts at index // position 1 ELSE // Otherwise start counting spaces... WHILE ThisPos <= LENGTH(FNString) AND Index = -1 ThisChar ← MID(FNString, ThisPos, 1) IF ThisChar = SPACECHAR THEN NumFound ← NumFound + 1 IF NumFound = WordNum - 1 THEN Index ← ThisPos + 1 // the start of the // required word ENDIF ENDIF ThisPos ← ThisPos + 1 ENDWHILE ENDIF RETURN Index ENDFUNCTION </pre> 1 mark for each of the following: 1 Function heading, including return type and function end 2 Loop counting spaces until word found or end of FNString 3 extract a character from FNString in a loop 4 compare with SPACECHAR and increment count if equal in a loop 5 compare count with WordNum - 1 (depending on initialisation value) in a loop 6 if equal then set flag or Index to ThisPos + 1 in a loop 7 Return Index (correctly in all cases / following a reasonable attempt) 8 Works for special case when looking for word 1 Note: Max 7 marks	7
------	---	---

7(c)	FUNCTION GetWord (Index : INTEGER) RETURNS STRING DECLARE NextWord : STRING DECLARE Done : BOOLEAN DECLARE ThisChar : CHAR DECLARE Index : INTEGER CONSTANT SPACECHAR = ' ' NextWord ← "" Done ← FALSE REPEAT ThisChar ← MID(FNString, Index, 1) IF ThisChar <> SPACECHAR THEN NextWord ← NextWord & ThisChar // build up NextWord ENDIF IF ThisChar = SPACECHAR OR Index = LENGTH(FNString) THEN Done ← TRUE ENDIF Index ← Index + 1 UNTIL Done = TRUE RETURN NextWord ENDFUNCTION 1 mark for each of the following: 1 Conditional loop 2 Extract char from FNString and compare with SPACECHAR in a loop 3 Concatenate with NextWord if not SPACECHAR in a loop 4 Exit loop when SPACECHAR encountered or when end of FNString reached 5 Return NextWord (after reasonable attempt at forming, and must have been initialised)	5
------	--	----------

7(c)	The 'length and substring' solution: <pre> FUNCTION GetWord (Index : INTEGER) RETURNS STRING DECLARE Done : BOOLEAN DECLARE ThisChar : CHAR DECLARE Index, NextPos : INTEGER CONSTANT SPACECHAR = ' ' Done ← FALSE NextPos ← Index // must be at least one character in // the required word REPEAT ThisChar ← MID(FNString, NextPos, 1) IF ThisChar = SPACECHAR OR NextPos = LENGTH(FNString) THEN Done ← TRUE ELSE NextPos ← NextPos + 1 ENDIF UNTIL Done = TRUE IF NextPos = LENGTH(FNString) THEN NextPos ← NextPos - 1 // special case when last word ENDIF RETURN MID(FNString, Index, NextPos - Index) ENDFUNCTION </pre> 1 mark for each of the following: 1 Conditional loop 2 ...extract char from FNString and compare with SPACECHAR in a loop 3 .. increment count if word continues 4 Exit loop when SPACECHAR encountered or when end of FNString reached 5 Apply substring function and Return	
------	--	--

91. M/J 21/P22/Q5/b

5(b)(i)	Answer		4
	Give a line number containing an example of an initialisation statement.	07	
	Give a line number containing the start of a repeating block of code.	09 / 10	
	Give a line number containing a logic statement.	12	
	Give the number of parameters of function MID () .	3	
One mark for each row			

5(b)(ii)	IF (NextChar >= 'a') AND (NextChar <= 'z') THEN	2
	One mark for IF ... AND ... One mark for both conditions	

92. M/J 21/P22/Q8/b

8(b)	<pre> Procedure GetInitials() DECLARE NewString, NextWord : STRING DECLARE ThisWordNum, Index : INTEGER ThisWordNum ← 0 NewString ← "" REPEAT ThisWordNum ← ThisWordNum + 1 Index ← GetStart(ThisWordNum) IF Index <> -1 THEN //if there is ThisWordNum NextWord ← GetWord(Index) IF IgnoreWord(NextWord) = FALSE THEN NewString ← NewString & UCASE(LEFT(NextWord, 1)) ENDIF ENDIF UNTIL Index = -1 OUTPUT NewString ENDPROCEDURE </pre> 1 mark for each of the following: 1 Declare <code>NewString</code> and initialise to empty string 2 Conditional loop to pick out all words from <code>FNString</code> 3 Evaluate result of <code>GetStart()</code> in a loop 4 Test result <code><> -1</code> and if not: 5 Assign result of <code>GetWord()</code> to a variable in a loop 6 Test result of <code>IgnoreWord()</code> in a loop 7 If not ignored, add the next initial letter to <code>NewString</code> in a loop 8 Increment <code>ThisWordNum</code> (must have been initialised) in a loop 9 Output <code>NewString</code> (must be all upper case) outside loop Note: Max 8 marks	8
------	--	---

93. M/J 21/P23/Q1/b

1(b)	Statement	Error	5
	Result $\leftarrow$ 2 & 4	Should be arithmetic operator (not &) // 2 and 4 should be CHAR / STRING	
	SubString $\leftarrow$ MID("pseudocode", 4, 1)	NO ERROR	
	IF x = 3 OR 4 THEN	Not Boolean values / incorrect operator // Condition incorrect	
	Result $\leftarrow$ Status AND INT(X/2)	INT(X/2) doesn't evaluate to a boolean value / incorrect operator	
	Message $\leftarrow$ "Done" + LENGTH(MyString)	Can't add string to number / "Done" is not a number	
One mark for each row			

94. M/J 21/P23/Q2/b

2(b)	<u>PROCEDURE LoanReturn (LoanID, BookID : STRING,</u> <u>BYREF Fine : REAL)</u>	2
One mark for each underlined part		

95. M/J 21/P23/Q4/b

4(b)	Loop structure: A count-controlled loop Justification: The number of iterations is known One mark per point	2
------	---	----------

96. M/J 21/P23/Q7/a,c

7(a)	<pre> FUNCTION GetStart (WordNum : INTEGER) RETURNS INTEGER DECLARE Index, ThisPos, NumFound : INTEGER DECLARE ThisChar : Char CONSTANT SPACECHAR = ' ' Index ← -1 Numfound ← 0 ThisPos ← 1 IF WordNum = 1 THEN // if looking for word 1... Index ← 1 // Word 1 always starts at index // position 1 ELSE // Otherwise start counting spaces... WHILE ThisPos <= LENGTH(FNString) AND Index = -1 ThisChar ← MID(FNString, ThisPos, 1) IF ThisChar = SPACECHAR THEN NumFound ← NumFound + 1 IF NumFound = WordNum - 1 THEN Index ← ThisPos + 1 // the start of the // required word ENDIF ENDIF ThisPos ← ThisPos + 1 ENDWHILE ENDIF RETURN Index ENDFUNCTION </pre> 1 mark for each of the following: 1 Function heading, including return type and function end 2 Loop counting spaces until word found or end of FNString 3 extract a character from FNString in a loop 4 compare with SPACECHAR and increment count if equal in a loop 5 compare count with WordNum - 1 (depending on initialisation value) in a loop 6 if equal then set flag or Index to ThisPos + 1 in a loop 7 Return Index (correctly in all cases / following a reasonable attempt) 8 Works for special case when looking for word 1 Note: Max 7 marks	7
------	---	---

7(c)	FUNCTION GetWord (Index : INTEGER) RETURNS STRING DECLARE NextWord : STRING DECLARE Done : BOOLEAN DECLARE ThisChar : CHAR DECLARE Index : INTEGER CONSTANT SPACECHAR = ' ' NextWord ← "" Done ← FALSE REPEAT ThisChar ← MID(FNString, Index, 1) IF ThisChar <> SPACECHAR THEN NextWord ← NextWord & ThisChar // build up NextWord ENDIF IF ThisChar = SPACECHAR OR Index = LENGTH(FNString) THEN Done ← TRUE ENDIF Index ← Index + 1 UNTIL Done = TRUE RETURN NextWord ENDFUNCTION 1 mark for each of the following: 1 Conditional loop 2 Extract char from FNString and compare with SPACECHAR in a loop 3 Concatenate with NextWord if not SPACECHAR in a loop 4 Exit loop when SPACECHAR encountered or when end of FNString reached 5 Return NextWord (after reasonable attempt at forming, and must have been initialised)	5
------	--	----------

7(c)	The 'length and substring' solution: <pre> FUNCTION GetWord (Index : INTEGER) RETURNS STRING DECLARE Done : BOOLEAN DECLARE ThisChar : CHAR DECLARE Index, NextPos : INTEGER CONSTANT SPACECHAR = ' ' Done ← FALSE NextPos ← Index // must be at least one character in // the required word REPEAT ThisChar ← MID(FNString, NextPos, 1) IF ThisChar = SPACECHAR OR NextPos = LENGTH(FNString) THEN Done ← TRUE ELSE NextPos ← NextPos + 1 ENDIF UNTIL Done = TRUE IF NextPos = LENGTH(FNString) THEN NextPos ← NextPos - 1 // special case when last word ENDIF RETURN MID(FNString, Index, NextPos - Index) ENDFUNCTION </pre> 1 mark for each of the following: 1 Conditional loop 2 ...extract char from FNString and compare with SPACECHAR in a loop 3 .. increment count if word continues 4 Exit loop when SPACECHAR encountered or when end of FNString reached 5 Apply substring function and Return	
------	--	--